



UNIVERSITÀ DEGLI STUDI DI TRENTO

Dipartimento di Ingegneria e Scienza dell'Informazione

Appunti di Calcolatori

Secondo parziale

A cura di:

Mirco Negri

Versione Light Mode. Disponibile anche la [versione Dark Mode](#).

luglio 2026

Indice

1	Cenni ad Assembly Intel	5
1.1	Architettura Intel: Caratteristiche Generali	5
1.2	I Registri	5
1.2.1	Registri General Purpose	5
1.2.2	Registri Speciali	6
1.3	Convenzioni di Chiamata (ABI)	6
1.4	Modalità di Indirizzamento	6
1.4.1	Accesso alla Memoria	7
1.5	Set di Istruzioni Comuni	7
1.5.1	Trasferimento Dati	7
1.5.2	Operazioni Logico-Aritmetiche	7
1.5.3	L'istruzione LEA (Load Effective Address)	8
1.5.4	Confronti e Salti	8
1.5.5	Chiamate a Funzione	8
2	Cenni ad Assembly ARM	8
2.1	Caratteristiche Generali dell'Architettura ARM	8
2.2	I Registri	9
2.3	Convenzioni di Chiamata (ABI)	9
2.4	Modalità di Indirizzamento	9
2.4.1	Accesso alla Memoria	9
2.5	Il Set di Istruzioni	10
2.5.1	Istruzioni Aritmetiche e Logiche	10
2.5.2	Istruzioni di Controllo e Movimento	10
2.6	Accesso Avanzato alla Memoria: Load/Store Multipli	10
3	Esempi di Programmi Assembly: RISC-V, Intel x86-64 e ARM	11
3.1	Richiamo: Registri e Convenzioni	11
3.2	Semplici Istruzioni Aritmetiche	11
3.2.1	Traduzione RISC-V	12
3.2.2	Traduzione Intel x86-64	12
3.2.3	Traduzione ARM	12
3.3	Accesso alla Memoria	12
3.3.1	Traduzione RISC-V	12
3.3.2	Traduzione Intel x86-64	12
3.3.3	Traduzione ARM	13
3.4	Blocchi Condizionali	13
3.4.1	Traduzione RISC-V	13
3.4.2	Traduzione Intel x86-64 (Ottimizzazione con Conditional Move)	13
3.4.3	Traduzione ARM (La potenza dell'esecuzione condizionale)	13
3.5	Cicli (While Loop)	14
3.5.1	Traduzione RISC-V	14
3.5.2	Traduzione Intel x86-64	14
3.5.3	Traduzione ARM	15
3.6	Funzioni e Gestione dello Stack	15
3.6.1	Traduzione RISC-V	15
3.6.2	Traduzione Intel x86-64	15

3.6.3	Traduzione ARM	16
3.7	Manipolazione Ottimizzata di Array (Copia Stringhe)	16
3.7.1	Traduzione RISC-V	16
3.7.2	Traduzione Intel x86-64	16
3.7.3	Traduzione ARM	17
4	Toolchain e Generazione degli Eseguibili	17
4.1	Dal Linguaggio ad Alto Livello al Linguaggio Macchina	17
4.2	Allocazione della Memoria (Esempio RISC-V)	17
4.3	Il Compilatore (Es. gcc)	18
4.4	L'Assembler e le Pseudo-Istruzioni	18
4.5	La Struttura del File Oggetto (.o)	18
4.6	Il Linker (ld)	19
4.7	Librerie: Statiche vs Dinamiche	19
4.8	Esempio Pratico di Linking	20
5	Il Processore: Architettura e Datapath	20
5.1	Obiettivi e Insieme di Istruzioni	20
5.2	Fasi di Esecuzione delle Istruzioni	20
5.3	Progettazione del Datapath (Percorso dei Dati)	21
5.3.1	Fase di Fetch e Aggiornamento del PC	21
5.3.2	Istruzioni Aritmetico-Logiche (Tipo R)	21
5.3.3	Istruzioni di Accesso alla Memoria (Load e Store)	22
5.3.4	Istruzioni di Salto Condizionato (Branch, Tipo SB)	22
5.4	Composizione del Datapath (Multiplexer e Controllo)	22
6	L'Unità di Controllo e l'Implementazione del Datapath	23
6.1	Progetto di un'Unità di Elaborazione a Ciclo Singolo	23
6.2	Panoramica dei Segnali di Controllo Principali	23
6.3	Decodifica a Livelli Multipli e Controllo dell'ALU	24
6.4	Decodifica dell'Istruzione	24
6.5	Esecuzione Passo-Passo sul Processore	25
6.5.1	Istruzione di Tipo R (es. add x5, x6, x7)	25
6.5.2	Istruzione di Load (es. ld x5, offset(x6))	25
6.5.3	Istruzione di Salto Condizionato (es. beq x5, x6, offset)	25
6.6	Sintesi dei Segnali per l'Unità di Controllo	26
6.7	Considerazioni Conclusive sull'Architettura a Singolo Ciclo	26
7	La Pipeline	26
7.1	Limiti dell'Architettura a Singolo Ciclo	26
7.2	L'Idea di Base: L'Analogia della Catena di Montaggio	27
7.3	La Pipeline nel Processore RISC-V	27
7.3.1	Perché RISC-V è ottimizzato per la Pipeline?	28
7.4	I Conflitti o Alee (<i>Hazards</i>)	28
7.4.1	1. Conflitti Strutturali (<i>Structural Hazards</i>)	28
7.4.2	2. Conflitti sui Dati (<i>Data Hazards</i>)	28
7.4.3	3. Conflitti di Controllo (<i>Branch Hazards</i>)	29
8	La Gerarchia di Memoria	29

8.1	Organizzazione e Concetti di Base	29
8.2	Terminologia Fondamentale	30
8.3	Interfaccia della Memoria Principale	30
8.4	Memorie Statiche (SRAM)	31
8.5	Memorie Dinamiche (DRAM)	31
8.5.1	Multiplazione degli Indirizzi	31
8.6	Evoluzione della DRAM	32
8.7	Metriche di Prestazione	32
9	Memoria Cache: Organizzazione e Funzionamento	32
9.1	L'Illusione di una Memoria Ideale e le Località	32
9.2	Terminologia Fondamentale	33
9.3	Politiche di Mappatura della Cache	33
9.3.1	1. Cache a Mappatura Diretta (<i>Direct Mapped</i>)	33
9.3.2	2. Cache Completamente Associativa (<i>Fully Associative</i>)	33
9.3.3	3. Cache Set-Associativa (<i>N-vie</i>)	34
9.4	Struttura di un Indirizzo e Calcolo	34
9.5	Il Trade-off sulla Dimensione del Blocco	34
9.6	Gestione delle Miss e delle Scritture	35
10	Sistemi di Input/Output e Interfacciamento	35
10.1	La Necessità di Comunicare e Classificazione	35
10.2	Tipologie di Dispositivi di Storage	36
10.2.1	I Dischi Magnetici (Hard Disk)	36
10.2.2	Memorie Flash e SSD (Solid State Drive)	36
10.3	I Bus di Comunicazione e la Sincronizzazione	36
10.4	Interfacciamento: Memory Mapped I/O	37
10.5	Gestione Asincrona e il Problema del Polling	37
10.5.1	Il Polling (Busy-Waiting)	38
11	Gestione delle Interruzioni e delle Eccezioni	38
11.1	Limiti del Polling e Introduzione agli Interrupt	38
11.2	Classificazione delle Eccezioni	39
11.3	Identificazione dell'Handler: Salto Diretto vs Vettorizzato	39
11.4	Supporto Hardware in RISC-V (I Registri CSR)	40
11.4.1	Istruzioni per manipolare i CSR	40
11.5	La Transizione di Privilegio (User → Supervisor)	41
11.6	Eccezioni nella Pipeline Hardware	41
12	Esercizi di Riepilogo	42
12.1	Esercizio 1: Reti Logiche e Algebra di Boole	42
12.2	Esercizio 2: Memoria Cache a Mappatura Diretta	42
12.3	Esercizio 3: Calcolo del CPI Reale (Prestazioni della CPU)	43
13	Tutorato: Esercizi di Traduzione C-Assembly (RISC-V, Intel x86-64, ARM)	44
13.1	Esercizio 1: Espressioni Aritmetiche Semplici	44
13.1.1	Traduzione RISC-V	45
13.1.2	Traduzione Intel x86-64	45

13.1.3 Traduzione ARM	45
13.2 Esercizi 2 e 3: Memoria e Moltiplicazioni Ottimizzate	45
13.3 Esercizi 4, 5, 6 e 7: Ottimizzazione dei Salti (Branchless)	46
13.3.1 La simulazione dell'If-Else: Intel vs ARM	46
13.4 Esercizio 8: L'Estrema Ottimizzazione Bit-a-Bit in ARM	47
13.5 Esercizio 9: Cicli While e Indirizzamento Avanzato	47
14 Soluzioni Mockup Complete	48
15 Preparazione all'Esame Orale: Domande Teoriche e Approfondimenti	83
15.1 1. Spiegare la gerarchia della memoria	83
15.2 2. Definire i principi di località spaziale e temporale	83
15.3 3. Le varie tipologie di cache e la lettura Set-Associativa	84
15.4 4. Il concetto di Pipeline: vantaggi, stadi ed Hazard	84
15.5 5. Differenza tra Interrupt e Traps	85
15.6 6. Polling vs I/O Asincrono e Bus	85
15.7 7. Spiegare lo schema del processore (RISC-V) e la <code>beq</code>	86
15.8 8. Istruzione <code>lea</code> in Assembly Intel x86-64	86
15.9 9. Toolchain: fasi ed esecuzione	87
15.10 10. Write-back in Assembly ARM (e implementazione in RISC-V)	87

1 Cenni ad Assembly Intel

1.1 Architettura Intel: Caratteristiche Generali

L'architettura Intel è impiegata nella stragrande maggioranza dei laptop, desktop e server moderni. È caratterizzata da una storia molto lunga che parte dagli anni '70 con processori a 8 bit (come l'Intel 8080) e arriva fino ai giorni nostri con le moderne architetture a 64 bit (come i processori Intel Core i7, i9, ecc.).

- **Retrocompatibilità:** Lo sviluppo ha seguito un'evoluzione lenta ma costante. Le moderne CPU a 64 bit sono ancora in grado di eseguire codice datato progettato per sistemi a 8 bit (es. MS-DOS). Questo comporta enormi complicazioni e una notevole complessità hardware.
- **CISC vs RISC:** A differenza dell'architettura RISC-V (che si basa su un set ridotto di istruzioni, *Reduced Instruction Set Computer*), Intel utilizza un'architettura **CISC** (*Complex Instruction Set Computer*), che mette a disposizione un numero molto elevato di istruzioni complesse e svariate modalità di indirizzamento.

1.2 I Registri

1.2.1 Registri General Purpose

L'architettura a 64 bit dispone di 16 registri di uso generale. In Assembly con sintassi GNU (gas), i registri sono sempre preceduti dal prefisso %.

- **Registri base:** %rax, %rbx, %rcx, %rdx, %rsi, %rdi, %rbp, %rsp.
- **Registri aggiuntivi:** %r8, %r9, ..., %r15.

Alcuni di questi hanno scopi specifici per convenzione:

- **%rsp:** Stack Pointer (punta alla cima dello stack).
- **%rbp:** Base Pointer (usato per puntare allo stack frame corrente).
- **%rsi e %rdi:** Nati originariamente per la copia di array (*source index* e *destination index*).

Sottodivisione dei registri: Una particolarità derivante dalla retrocompatibilità è la possibilità di accedere a frazioni (sottoregistri) dei registri a 64 bit:

- **64 bit:** %rax
- **32 bit:** %eax (la metà inferiore di %rax)
- **16 bit:** %ax (la metà inferiore di %eax)
- **8 bit:** %ah (parte alta di %ax) e %al (parte bassa di %ax).

Questa struttura si ripete per i primi quattro registri (a, b, c, d). Per i registri da %r8 a %r15, i sottoregistri si indicano con suffissi: d (32 bit, es. %r8d), w (16 bit, es. %r8w), b (8 bit, es. %r8b).

1.2.2 Registri Speciali

- **%rip** (*Instruction Pointer*): Contiene l'indirizzo della prossima istruzione da eseguire (corrisponde al Program Counter).
- **%rflags** (*Flags register*): Contiene i flag settati dalle istruzioni logico-aritmetiche. I principali sono:
 - **CF (Carry Flag)**: Settato a 1 se si verifica un riporto (carry-out) o un overflow su operazioni *unsigned*.
 - **ZF (Zero Flag)**: Settato a 1 se il risultato dell'operazione è zero.
 - **SF (Sign Flag)**: Settato a 1 se il risultato è negativo.
 - **OF (Overflow Flag)**: Settato a 1 se si verifica un overflow in operazioni *signed* (in complemento a 2).

1.3 Convenzioni di Chiamata (ABI)

Le convenzioni di chiamata stabiliscono le regole di interfaccia tra funzioni (come passare i parametri e chi deve salvare i registri). Non fanno parte dell'hardware (ISA), ma sono specifiche del sistema (ABI).

- **Passaggio dei parametri**: I primi 6 argomenti vengono passati nei registri, rigorosamente in questo ordine: `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`. Dal settimo argomento in poi, i parametri vengono caricati sullo stack.
- **Valori di ritorno**: Vengono memorizzati nei registri `%rax` e (se necessario) `%rdx`.
- **Registri preservati (Callee-saved)**: `%rbp`, `%rbx`, `%r12`, `%r13`, `%r14`, `%r15`. Se una funzione chiamata (callee) vuole usare questi registri, ha l'obbligo di salvarne il valore originario sullo stack e ripristinarlo prima di ritornare.
- **Registri non preservati (Caller-saved)**: `%rax`, `%r10`, `%r11` e i registri usati per i parametri. Possono essere sovrascritti liberamente da una subroutine.

1.4 Modalità di Indirizzamento

Le istruzioni Intel generalmente utilizzano un formato a 2 operandi, in cui il secondo funge implicitamente anche da destinazione (sintassi: **operazione sorgente, destinazione**). Questo differisce da RISC-V, dove si possono specificare tre registri distinti. Gli operandi possono essere:

- **Immediati**: Valori costanti prefissati dal simbolo `$` (es. `$20`).
- **In Registro**: Valore contenuto in un registro (es. `%rax`).
- **In Memoria**: Valore in una specifica locazione di memoria. Intel permette di scrivere o leggere direttamente in memoria come parte di operazioni logico-aritmetiche, non limitandosi alle sole *load* e *store* come in RISC-V.

Vincolo fondamentale: Un'istruzione non può avere entrambi gli operandi in memoria. Almeno uno dei due (sorgente o destinazione) deve essere un registro o un valore immediato.

1.4.1 Accesso alla Memoria

L'indirizzamento alla memoria ha una forma sintattica generale molto potente:

$$\text{displacement}(\%base, \%indice, scala)$$

L'indirizzo effettivo calcolato sarà: $\text{displacement} + \text{base} + (\text{indice} \times \text{scala})$.

- **displacement:** Valore costante di offset (a differenza di RISC-V, può essere a 8, 16 o 32 bit).
- **base:** Registro che contiene l'indirizzo di partenza.
- **indice:** Registro usato per l'iterazione.
- **scala:** Moltiplicatore costante per l'indice (può valere solo 1, 2, 4 o 8, facilitando l'accesso ad array i cui elementi hanno queste dimensioni in byte).

Si possono omettere elementi se non necessari. Ad esempio, `(%rbx)` usa solo la base, mentre `4(%rbx)` usa base e spiazzamento.

1.5 Set di Istruzioni Comuni

In Assembly GNU, le istruzioni spesso terminano con un carattere che indica la dimensione dei dati trattati: **b** (byte, 8 bit), **w** (word, 16 bit), **l** (long, 32 bit), **q** (quadword, 64 bit).

1.5.1 Trasferimento Dati

- **mov:** Copia dati dalla sorgente alla destinazione.
- **movsx / movzx:** Copia i dati estendendo il segno (*sx*) o riempiendo con zeri (*zx*) quando si passa da un operando più piccolo a uno più grande.
- **pushq %reg:** Inserisce il valore di un registro a 64 bit sullo stack. Decrementa `%rsp` di 8 byte e scrive in memoria.
- **popq %reg:** Estrae il valore dalla cima dello stack, lo salva in `%reg` e incrementa `%rsp` di 8 byte.

1.5.2 Operazioni Logico-Aritmetiche

- **Aritmetiche:** **add** (somma), **sub** (sottrazione), **mul/imul** (moltiplicazione unsigned/signed), **div/idiv** (divisione unsigned/signed), **inc** (incrementa di 1), **dec** (decrementa di 1), **neg** (nega in complemento a 2).
- **Logiche e Bit-a-bit:** **and**, **or**, **xor**, **not**.
- **Shift e Rotazioni:** **sal/shl** (shift logico/aritmetico a sinistra), **sar/shr** (shift aritmetico/logico a destra), **rol/ror** (rotazione).

Nota su inc/dec: Queste istruzioni esistono, rispetto ad un normale `add $1`, perché in un'architettura CISC permettono di codificare l'operazione usando meno bit (non dovendo memorizzare il valore costante 1 nell'istruzione).

1.5.3 L'istruzione LEA (Load Effective Address)

L'istruzione **lea** *src*, *dest* ha la sintassi di un accesso in memoria, ma *non accede* alla memoria. Sfrutta semplicemente i circuiti hardware dell'ALU dedicati al calcolo degli indirizzi per calcolare il risultato e salvarlo nel registro destinazione.

- **Vantaggio:** Permette di sommare fino a due registri e una costante, potendo persino moltiplicare un registro per 2, 4 o 8, il tutto in una sola istruzione e salvando il risultato in un *terzo* registro senza modificare i flag.
- Esempio: `leaq 80(%rdx, %rcx, 2), %rax` calcola l'espressione $rdx + (rcx \cdot 2) + 80$ e salva il risultato in *rax*.

1.5.4 Confronti e Salti

- **cmp** *arg1*, *arg2*: Esegue la sottrazione $arg2 - arg1$ (senza salvare il risultato) unicamente per aggiornare il registro `%rflags`.
- **test** *arg1*, *arg2*: Esegue l'AND logico $arg2 \& arg1$ per aggiornare i flag (es. `test %eax, %eax` si usa per verificare se un registro è zero o negativo).
- **jmp**: Salto incondizionato a una label.
- **Salti Condizionati**: Saltano solo se i flag rispettano certe condizioni. Esempi: `je` / `jz` (salta se uguale/zero), `jne` / `jnz` (se diverso/non zero), `jg` (maggiore signed), `j1` (minore signed), `ja` (maggiore unsigned).

1.5.5 Chiamate a Funzione

- **call** *label*: Spinge (push) l'indirizzo dell'istruzione successiva (*indirizzo di ritorno*) sullo stack e aggiorna il Program Counter (`%rip`) all'indirizzo della label della funzione.
- **ret**: Estrae (pop) l'indirizzo di ritorno dallo stack, lo inserisce in `%rip` e riprende l'esecuzione del chiamante.

2 Cenni ad Assembly ARM

2.1 Caratteristiche Generali dell'Architettura ARM

L'architettura ARM (originariamente *Acorn Risc Machine*, poi *Advanced Risc Machine*) è nata negli anni '80 ed è oggi ampiamente utilizzata nella maggior parte di tablet, smartphone e sistemi embedded.

- **Evoluzione lineare:** A differenza dell'architettura Intel, ha avuto un'evoluzione meno complicata. Nata come architettura a 32 bit, è passata solo recentemente ai 64 bit, rinunciando però a una rigida retrocompatibilità col passato.
- **Filosofia RISC "pragmatica":** Sebbene sia una macchina RISC (*Reduced Instruction Set Computer*), ARM possiede un set di istruzioni molto ricco e potenti modalità di indirizzamento, prendendo spunto anche dai punti di forza del mondo CISC.

- **Obiettivi primari:** L'architettura è stata sviluppata per garantire un'alta densità del codice e, soprattutto, un forte risparmio energetico.

2.2 I Registri

L'architettura a 32 bit mette a disposizione 16 registri principali, da **r0** a **r15**. In Assembly GNU, a differenza di Intel, i nomi dei registri non presentano alcun prefisso. Sebbene siano (quasi) tutti *general purpose*, alcuni assumono funzioni specifiche:

- **r13 (sp):** Utilizzato come Stack Pointer.
- **r14 (lr):** *Link Register*. Salva l'indirizzo di ritorno quando si invoca una subroutine (analogo a **ra** in RISC-V).
- **r15 (pc):** Contiene il *Program Counter* e, nei bit dal 28 al 31, ospita anche i *Flags* di stato.

I **Flags Register** (*apsr/cpsr*) vengono aggiornati dalle istruzioni logico-aritmetiche (se provviste del suffisso **s**) o da istruzioni di confronto. Permettono l'esecuzione condizionale delle istruzioni (es. **eq** per uguale, **ne** per diverso, **mi** per negativo, **vs** per overflow).

2.3 Convenzioni di Chiamata (ABI)

Le convenzioni di chiamata specificano l'utilizzo dei registri per l'interazione tra funzioni. In ARM:

- **Passaggio parametri:** I primi 4 argomenti vengono passati nei registri da **r0** a **r3**. Dal quinto argomento in poi, si utilizza lo stack.
- **Registri non preservati (*Caller-saved*):** **r0**, **r1**, **r2**, **r3** (utilizzabili anche come registri temporanei) e **r12**.
- **Registri preservati (*Callee-saved*):** Da **r4** a **r11** (il salvataggio di **r9** dipende dalla specifica ABI).
- **Valori di ritorno:** Vengono memorizzati in **r0** e **r1**.

2.4 Modalità di Indirizzamento

Le istruzioni ARM tipicamente accettano 3 argomenti: un registro destinazione, un primo operando sorgente (registro) e un secondo operando (che può essere un valore immediato oppure un registro, a sua volta shiftato o ruotato). I valori immediati (costanti) si indicano col prefisso **#**. Non sono concessi operandi in memoria per le operazioni logico-aritmetiche.

2.4.1 Accesso alla Memoria

La memoria è accessibile solo tramite istruzioni di Load e Store (**ldr/str** e **ldm/stm**). L'indirizzamento è estremamente potente e sfrutta un registro base con tre possibili metodologie:

- **Pre-Indexed:** L'indirizzo effettivo è la somma del registro base e di un offset/indice. La sintassi è `[rb, #offset]` oppure `[rb, ro, shift]`. Se è presente il simbolo `!` alla fine (`[rb, #offset]!`), la CPU eseguirà il *writeback*: dopo aver calcolato l'indirizzo ed effettuato l'accesso in memoria, il registro base `rb` verrà aggiornato col nuovo indirizzo calcolato.
- **Post-Indexed:** Si accede prima alla memoria utilizzando l'indirizzo base originale contenuto in `rb`. Subito dopo, si somma l'offset (o indice) a `rb` aggiornandolo. La sintassi è `[rb], #offset` oppure `[rb], ro, shift`. È estremamente utile per scorrere sequenzialmente degli array.

2.5 Il Set di Istruzioni

Ogni istruzione in ARM può essere **eseguita condizionalmente** apponendo un suffisso specifico al mnemonic dell'istruzione (es. `addeq` esegue la somma solo se il flag Z è asserito).

2.5.1 Istruzioni Aritmetiche e Logiche

- **Somma e Sottrazione:** `add`, `adc` (somma col riporto), `sub`, `sbc` (sottrazione in prestito). ARM fornisce anche `rsb` (*Reverse Subtract*) e `rsc`. La `rsb` calcola $r2 - r1$ invece di $r1 - r2$; è fondamentale poiché il terzo operando in ARM può subire shift hardware, permettendo operazioni combinate complesse in un singolo ciclo di clock.
- **Logiche:** `and`, `orr` (OR), `eor` (XOR). Esiste inoltre l'istruzione `bic` (*Bit Clear*) che esegue l'operazione `r1 AND NOT r2`, usata per resettare a 0 specifici bit del primo operando.
- **Moltiplicazione:** `mul` e `mla` (*Multiply Accumulate*, che calcola $r1 \cdot r2 + r3$). Va notato che molte CPU ARM non forniscono nativamente un'istruzione hardware di divisione; viene spesso implementata via software.
- **Confronto:** `cmp` (sottrazione), `tst` (AND), `teq` (XOR), `cmn` (somma). Eseguono la rispettiva operazione aritmetico-logica ma *non* salvano il risultato, aggiornando esclusivamente il registro dei Flag (`apsr`).

2.5.2 Istruzioni di Controllo e Movimento

- **Movimento dati:** `mov` ricopia un valore in un registro. `mvn` (*Move Not*) copia il complemento a uno di un registro.
- **Salti:** `b` (salto incondizionato o condizionato tramite suffisso), `bl` (*Branch and Link*, usato per chiamare subroutine salvando il pc in `lr`).

2.6 Accesso Avanzato alla Memoria: Load/Store Multipli

Le istruzioni `ldm` (Load Multiple) e `stm` (Store Multiple) permettono di leggere o scrivere *un insieme* di registri in blocco, migliorando l'efficienza nel trasferimento dati o durante il prologo/epilogo delle funzioni.

Possono operare con quattro diverse modalità di avanzamento dell'indirizzo :

- **IA (*Increment After*)**: L'indirizzo base viene incrementato *dopo* ogni operazione di trasferimento.
- **IB (*Increment Before*)**: L'indirizzo base viene incrementato *prima* di trasferire il dato.
- **DA (*Decrement After*)**: L'indirizzo decrementa dopo l'accesso.
- **DB (*Decrement Before*)**: L'indirizzo decrementa prima dell'accesso.

Ad esempio, `ldmia r0!, {r1, r2, r3}` carica i tre registri a partire dall'indirizzo `r0`, incrementando l'indirizzo ad ogni passo e aggiornando il valore finale in `r0` grazie al suffisso `!`.

Per operare su parti inferiori ai 32 bit, le normali istruzioni di load e store accettano suffissi per la dimensione dei dati: `ldrb` e `strb` operano sui singoli Byte (8 bit), mentre `ldrh` e `strh` sulle Half-Word (16 bit).

3 Esempi di Programmi Assembly: RISC-V, Intel x86-64 e ARM

In questa sezione analizzeremo la traduzione di vari costrutti di programmazione ad alto livello in linguaggio Assembly, confrontando l'architettura RISC-V, l'architettura Intel x86-64 e quella ARM. Questo confronto ci permetterà di apprezzare come le peculiarità di ciascun processore (come l'esecuzione condizionale in ARM o l'indirizzamento complesso in Intel) semplifichino e differenzino notevolmente la stesura del codice.

3.1 Richiamo: Registri e Convenzioni

Prima di procedere con gli esempi, ricordiamo brevemente l'uso dei registri nelle tre architetture:

- **RISC-V**: Dispone di 32 registri (`x0-x31`). `x0` è costantemente zero. I parametri passano in `a0-a7` (`x10-x17`), i valori di ritorno in `a0-a1`. Lo stack pointer è `sp` (`x2`), l'indirizzo di ritorno è `ra` (`x1`). I registri `s0-s11` sono preservati dal chiamato (*callee-saved*).
- **Intel x86-64**: Dispone di 16 registri general-purpose a 64 bit. I parametri passano in `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`, mentre il valore di ritorno va in `%rax`. Lo stack pointer è `%rsp` e il base pointer è `%rbp`. I registri `%rbx`, `%rbp`, `%r12-%r15` sono preservati dal chiamato (*callee-saved*).
- **ARM**: Dispone di 16 registri (`r0-r15`). I primi 4 (`r0-r3`) sono usati per gli argomenti e come temporanei (*caller-saved*). I registri da `r4` a `r11` sono preservati (*callee-saved*). I registri speciali sono `r13` (`sp`, Stack Pointer), `r14` (`lr`, Link Register per l'indirizzo di ritorno) e `r15` (`pc`, Program Counter).

3.2 Semplici Istruzioni Aritmetiche

Consideriamo l'espressione C: `f = (g + h) - (i + j);`

3.2.1 Traduzione RISC-V

Supponendo g , h , i , j nei registri da $x19$ a $x22$ e il risultato f in $x23$:

```
add x23, x19, x20    // x23 = g + h
add x6,  x21, x22    // x6  = i + j
sub x23, x23, x6     // x23 = x23 - x6 (f = ...)
```

3.2.2 Traduzione Intel x86-64

Supponiamo g , h , i , j passati nei registri `%edi`, `%esi`, `%edx`, `%ecx`. Il risultato f andrà in `%eax`.

```
leal (%rdi, %rsi), %eax    // %eax = g + h
addl %ecx, %edx            // %edx = i + j
subl %edx, %eax            // %eax = %eax - %edx (f = ...)
```

3.2.3 Traduzione ARM

La logica è identica a RISC-V, differiscono solo i nomi dei registri. L'uso della **s** finale (es. `adds`) serve per far aggiornare i flag di stato alla ALU, anche se per questa specifica espressione non è strettamente necessario.

```
adds r1, r0, r1          // r1 = g + h
adds r3, r2, r3          // r3 = i + j
subs r0, r1, r3          // r0 = r1 - r3 (f = ...)
```

3.3 Accesso alla Memoria

Consideriamo l'assegnamento: `a[12] = h + a[8];`

3.3.1 Traduzione RISC-V

Supponiamo h in $x21$ e l'indirizzo base di a in $x22$. Essendo gli elementi interi a 32 bit (4 byte), l'indice va moltiplicato per 4: $a[8]$ si trova all'offset 32, $a[12]$ all'offset 48.

```
lw   x9, 32(x22)        // x9 = a[8]
addw x9, x21, x9         // x9 = h + a[8]
sw   x9, 48(x22)        // a[12] = x9
```

3.3.2 Traduzione Intel x86-64

Assumiamo h in `%esi` e l'indirizzo base dell'array a in `%rdi`. Poiché gli offset (32 e 48) sono costanti statiche, il compilatore le pre-calcola e le usa come *displacement* diretto.

```
movl 32(%rdi), %eax      // %eax = a[8]
addl %esi, %eax          // %eax = h + a[8]
movl %eax, 48(%rdi)      // a[12] = %eax
```

3.3.3 Traduzione ARM

L'indirizzamento è molto simile a RISC-V (pre-indexed senza writeback). Assumiamo **h** in **r0** e base **a** in **r1**.

```
ldr r3, [r1, #32]    // r3 = a[8]
add r0, r3, r0        // r0 = h + a[8]
str r0, [r1, #48]    // a[12] = r0
```

3.4 Blocchi Condizionali

Vediamo come le architetture gestiscono i salti. Codice C:

```
if (i == j)
    f = g + h;
else
    f = g - h;
```

3.4.1 Traduzione RISC-V

Richiede l'uso esplicito di istruzioni di *branch* (salto) per deviare il flusso di esecuzione:

```
    bne x22, x23, L2    // se i != j salta a L2 (ramo else)
    add x19, x20, x21    // Ramo IF: f = g + h
    beq x0, x0, L3      // Salto incondizionato alla fine
L2: sub x19, x20, x21    // Ramo ELSE: f = g - h
L3:                      // Uscita
```

3.4.2 Traduzione Intel x86-64 (Ottimizzazione con Conditional Move)

Assumiamo **i**, **j**, **g**, **h** in **%edi**, **%esi**, **%edx**, **%ecx**. Intel x86-64 può evitare i salti svuota-pipeline calcolando entrambi i rami speculativamente e usando un **cmov** (Conditional Move).

```
leal (%rdx, %rcx), %eax    // %eax = g + h (Calcola sempre il Ramo IF)
movl %edx, %r8d
subl %ecx, %r8d            // %r8d = g - h (Calcola sempre il Ramo ELSE)
cmpl %esi, %edi            // Confronta i e j (imposta i flag)
cmovne %r8d, %eax          // Se i != j (Not Equal), sovrascrivi %eax con %r8d
```

3.4.3 Traduzione ARM (La potenza dell'esecuzione condizionale)

ARM evita i salti condizionati grazie ai suffissi applicati alle istruzioni (**eq** per *equal*, **ne** per *not equal*). Questo approccio, senza salti, previene gli stalli nella pipeline del processore.

```
cmp    r2, r3            // Confronta i e j, aggiorna i flag
addeq  r0, r0, r1        // Esegue r0 = g + h SOLO se i flag indicano uguaglianza (eq)
subne  r0, r0, r1        // Esegue r0 = g - h SOLO se i flag indicano
                        // disuguaglianza (ne)
```

La stessa logica si applica per le **disuguaglianze** (**if (i < j)**):

- **RISC-V**: Usa prima **blt** (Branch Less Than) o **slt** (Set Less Than).

- **Intel x86-64:** Usa `cmpl` seguito da `cmovge` (Move if Greater or Equal).
- **ARM:** Esegue una `cmp r2, r3` seguita da `addlt` (Less Than) e `subge` (Greater or Equal).

3.5 Cicli (While Loop)

Codice C:

```
i = 0;
while (a[i] == k)
    i += 1;
```

3.5.1 Traduzione RISC-V

Si gestisce un contatore `i` esplicito, lo si moltiplica per 4 a ogni iterazione e lo si somma all'indirizzo base per ottenere l'elemento corrente.

```

        add  x22, x0, x0      // i = 0
L1:     slli x10, x22, 2      // x10 = i * 4 (offset)
        add  x10, x10, x25    // x10 = indirizzo di a[i]
        lw   x9, 0(x10)      // x9 = a[i]
        bne  x9, x24, L2     // se a[i] != k esci dal ciclo
        addi x22, x22, 1     // i = i + 1
        beq  x0, x0, L1      // torna all'inizio del loop
L2:
```

3.5.2 Traduzione Intel x86-64

Sfruttiamo la potente modalità di indirizzamento AGU (`base, indice, scala`) in hardware, che ci risparmia il calcolo manuale dello shift per il contatore. Assumiamo `a` in `%rdi`, `k` in `%edx` e il contatore `i` in `%rax`.

```

        movq $0, %rax        // i = 0
L1:     cmpl %edx, (%rdi, %rax, 4) // k == a[i]? (Risolve addr = rdi + rax * 4)
        jne  L2              // Se diversi, esci dal ciclo
        addq $1, %rax        // i += 1
        jmp  L1              // Torna all'inizio del loop
L2:
```

3.5.3 Traduzione ARM

Usiamo il **pre-indexing con writeback (!)** di ARM. Invece di calcolare l'indice *i* moltiplicandolo per 4 ad ogni iterazione, aggiorniamo direttamente il puntatore *r1* di 4 byte alla volta.

```
    ldr r3, [r1]          // Carica a[0] (prima del loop)
    cmp r0, r3            // k == a[0]?
    bne L2                // Se no, esci subito
    mov r3, #0            // Contatore i = 0
L1: add r3, r3, #1        // i = i + 1
    ldr r2, [r1, #4]!     // PRE-INDEXED WRITEBACK: r1=r1+4, poi carica a[i] in r2
    cmp r2, r0            // a[i] == k?
    beq L1                // Se uguali, cicla
    b L3
L2: mov r3, #0
L3: mov r0, r3            // Salva il risultato (i) in r0
```

3.6 Funzioni e Gestione dello Stack

La differenza sostanziale emerge nel prologo e nell'epilogo delle funzioni che chiamano altre funzioni (non-foglia).

Codice C:

```
int inc(int n) { return n + 1; }
int f(int x)   { return inc(x) - 4; }
```

3.6.1 Traduzione RISC-V

La funzione *f* deve salvare *ra* (Return Address) sullo stack manualmente prima di eseguire la *call* (che sovrascriverebbe *ra*).

```
f:  addi sp, sp, -16      // Prologo: alloca stack
    sd   ra, 8(sp)       // Salva l'indirizzo di ritorno
    jal  ra, inc          // call inc()
    addiw a0, a0, -4      // Elabora ritorno: a0 = a0 - 4
    ld   ra, 8(sp)       // Epilogo: recupera indirizzo di ritorno
    addi sp, sp, 16       // Dealloca stack
    ret
```

3.6.2 Traduzione Intel x86-64

L'istruzione *call* in Intel salva automaticamente l'indirizzo di ritorno sullo stack. Il parametro *x* si trova già in *%edi*, che è lo stesso registro richiesto dalla convenzione di chiamata per passarlo in avanti a *inc*. Serve solo riallineare lo stack pointer.

```
f:  subq $8, %rsp         // Allinea lo stack pointer (a 16 byte boundary)
    call inc              // Chiama inc(x), risultato in %eax
    subl $4, %eax         // Sottrae 4 al valore di ritorno
    addq $8, %rsp         // Ripristina lo stack pointer
    ret
```


3.6.3 Traduzione ARM

ARM impiega le formidabili istruzioni di load/store multiplo (`stmfd` / `ldmfd`) per salvare e ripristinare contestualmente più registri. In particolar modo, caricando l'indirizzo originario (precedentemente salvato da `lr`) direttamente nel Program Counter (`pc`) all'interno della pop, la funzione effettua contemporaneamente l'epilogo e il ritorno (`ret`).

```
f:  stmfd sp!, {r4, lr} // Push di r4 e del Link Register. sp si aggiorna (!)
    bl    inc          // Branch and Link (chiama inc)
    sub   r0, r0, #4    // a0 = a0 - 4
    ldmfd sp!, {r4, pc} // Pop: ripristina r4 e mette l'indirizzo salvato in PC.
                          // (RITORNO AUTOMATICO)
```

Nota: `stmfd` (Store Multiple Full Descending) è un alias di `stmdb` (Decrement Before), e `ldmfd` è alias di `ldmia` (Increment After).

3.7 Manipolazione Ottimizzata di Array (Copia Stringhe)

Codice C:

```
while ((d[i] = s[i]) != '\0') {
    i += 1;
}
```

3.7.1 Traduzione RISC-V

Si deve gestire esplicitamente un contatore e calcolare l'offset per le istruzioni `lbu` (Load Byte Unsigned) e `sb` (Store Byte).

```
LoopCopia:
    add x5, x19, x11    // x5 = indirizzo di s[i]
    lbu x6, 0(x5)       // x6 = s[i]
    add x7, x19, x10    // x7 = indirizzo di d[i]
    sb  x6, 0(x7)       // d[i] = x6
    beq x6, x0, Fine    // se carattere nullo ('\0'), esci
    addi x19, x19, 1    // i++
    jal x0, LoopCopia   // cicla
```

Fine:

3.7.2 Traduzione Intel x86-64

Sfruttiamo l'indirizzamento complesso sui singoli byte. Manteniamo il contatore `i` in `%rcx`, l'array destinazione `d` in `%rdi` e la sorgente `s` in `%rsi`.

```
    movq $0, %rcx      // i = 0
L1:  movb (%rsi, %rcx, 1), %al // %al = s[i] (carica 1 byte)
    movb %al, (%rdi, %rcx, 1) // d[i] = %al (salva 1 byte)
    testb %al, %al      // Controlla se %al è 0 ('\0')
    je   Fine           // Se è nullo, esci dal ciclo
    addq $1, %rcx       // i += 1
    jmp  L1             // cicla
```

Fine:

3.7.3 Traduzione ARM

Sfruttando l'istruzione **post-indexed con writeback**, i due puntatori **r0 (d)** e **r1 (s)** si aggiornano automaticamente di 1 byte al termine del load e dello store. Non c'è alcun contatore esplicito, né si calcola la somma per trovare l'indirizzo. L'efficienza è massima.

```
L3: ldrb r3, [r1], #1    // Carica il byte puntato da r1 in r3.
                          // POI r1 = r1 + 1 (Post-Indexed)
    strb r3, [r0], #1    // Salva r3 all'indirizzo r0.
                          // POI r0 = r0 + 1 (Post-Indexed)
    cmp  r3, #0          // Era il terminatore di stringa '\0'?
    bne  L3              // Se no, cicla
    bx   lr              // Ritorna
```

4 Toolchain e Generazione degli Eseguibili

4.1 Dal Linguaggio ad Alto Livello al Linguaggio Macchina

I processori comprendono esclusivamente il **linguaggio macchina**, ovvero sequenze binarie di 0 e 1 di bassissimo livello. Il linguaggio Assembly (basato su codici mnemonici come **add**, **addi**) è più leggibile per l'uomo, ma è comunque complesso per lo sviluppo di software moderno. Di norma, si programma in linguaggi di alto livello (come il C) e si utilizza una **Toolchain** (catena di strumenti) per arrivare all'eseguibile. Il processo di trasformazione si divide in quattro fasi principali:

1. **Preprocessore:** Analizza il codice sorgente (es. un file **.c**) e risolve le direttive (come **#include** o **#define**), operando essenzialmente delle sostituzioni di testo.
2. **Compilatore:** Traduce il codice di alto livello in linguaggio Assembly (generando un file **.s**).
3. **Assembler:** Traduce il codice Assembly nel linguaggio macchina vero e proprio, generando un **file oggetto** (file **.o**).
4. **Linker:** Prende uno o più file oggetto e li unisce insieme alle funzioni di libreria, risolvendo i riferimenti incrociati e generando il programma **eseguibile** finale.

Inoltre, per avviare il programma, interviene il **Loader**, un componente del Sistema Operativo (su Unix associato alle system call della famiglia **exec()**), che carica l'eseguibile in memoria e avvia l'esecuzione.

4.2 Allocazione della Memoria (Esempio RISC-V)

Quando il programma viene caricato in memoria, il suo spazio di indirizzamento viene diviso in segmenti specifici (tipici in architetture come RISC-V):

- **Stack:** Inizia da indirizzi alti (es. **0x0000 003f ffff fff0**) e cresce verso il basso. Contiene le variabili locali e i record di attivazione delle funzioni.
- **Dati Dinamici (Heap):** Cresce verso l'alto ed è utilizzato per le allocazioni a runtime (es. tramite **malloc**).

- **Dati Statici (.data):** Posizionato più in basso (es. da 0x0000 0000 1000 0000), contiene le variabili globali e statiche allocate per tutta la durata del programma.
- **Testo (.text):** Contiene il codice eseguibile in linguaggio macchina. Su RISC-V parte tipicamente dall'indirizzo 0x0000 0000 0040 0000.
- **Area Riservata:** La primissima parte della memoria (a partire dall'indirizzo 0) è riservata per il sistema.

4.3 Il Compilatore (Es. gcc)

Il comando `gcc` (GNU Compiler Collection) funge da driver: invoca automaticamente preprocessore (`cpp`), compilatore (`cc`), assembler (`as`) e linker (`ld`).

- Eseguendo `gcc -S file.c`, il processo si ferma al compilatore, generando il file Assembly (`file.s`).
- Eseguendo `gcc -c file.c`, il processo si ferma all'assembler, generando il file oggetto (`file.o`).

Il compilatore conosce intimamente l'architettura bersaglio e spesso genera codice più efficiente di quello scritto a mano. Supporta vari livelli di ottimizzazione: da `-O0` (nessuna ottimizzazione, più facile da debuggare) fino a `-O1`, `-O2`, `-O3` (il compilatore riordina pesantemente le istruzioni per sfruttare al meglio l'hardware).

4.4 L'Assembler e le Pseudo-Istruzioni

L'Assembler converte l'Assembly in file oggetto. Oltre a tradurre i mnemonici in bit e a gestire i numeri (da esadecimali/decimanti a binari), l'assembler svolge un ruolo chiave nella gestione delle **pseudo-istruzioni**. Queste sono istruzioni non previste nativamente dall'hardware, ma fornite per comodità del programmatore, che l'assembler traduce in vere istruzioni:

- `mv x10, x11` diventa `addi x10, x11, 0`
- `li x9, 123` (Load Immediate) diventa `addi x9, x0, 123`
- `j LABEL` diventa `jal x0, LABEL`

Inoltre, l'assembler gestisce le *label* testuali e, se la destinazione di un salto è troppo lontana rispetto al Program Counter, converte automaticamente l'istruzione in una che carica l'indirizzo in un registro seguita da un salto indiretto (es. `jr`).

4.5 La Struttura del File Oggetto (.o)

Il file oggetto non è ancora eseguibile. È un blocco logico che contiene:

- **Header:** Specifica la dimensione e la posizione dei vari segmenti.
- **Segmento Text (.text):** Il codice macchina.
- **Segmento Data (.data):** Le costanti e le variabili statiche/globali.

- **Tabella dei Simboli (*Symbol Table*):** Associa nomi di variabili o funzioni a indirizzi (ancora relativi). Distingue tra simboli *definiti* (presenti in questo file) e *non definiti* (utilizzati qui, ma definiti in altri file). Include anche i simboli locali (esportati ma non visibili ad altri moduli).
- **Tabella di Rilocazione (*Relocation Table*):** Un elenco di istruzioni in linguaggio macchina il cui campo dell'indirizzo è momentaneamente vuoto o temporaneo e deve essere "patchato" (corretto) dal linker quando si saprà l'indirizzo di memoria definitivo.

4.6 Il Linker (1d)

Il Linker prende uno o più file oggetto e costruisce il vero file eseguibile (es. `a.out` o `.exe`). Il suo lavoro si articola in 3 passi fondamentali:

1. **Disposizione in Memoria:** Mette insieme i vari segmenti. Accoda tutti i segmenti `.text` dei vari file oggetto in un unico blocco e fa lo stesso per i segmenti `.data`.
2. **Risoluzione dei Simboli:** Avendo deciso la posizione in memoria, calcola e assegna un indirizzo assoluto a ciascun simbolo, combinando le varie Tabelle dei Simboli. Se un simbolo risulta invocato ma non è presente in alcun file oggetto né in alcuna libreria, si ottiene un *errore di linking*.
3. **Rilocazione (Patching):** Scorrendo la Tabella di Rilocazione, il linker corregge materialmente i bit delle singole istruzioni (inserendo gli offset corretti per istruzioni come i salti e gli accessi alla memoria). Una volta completata, le tabelle dei simboli e di rilocazione non sono più strettamente necessarie per l'esecuzione.

4.7 Librerie: Statiche vs Dinamiche

Le librerie evitano di dover re-inventare funzioni comuni (come `printf`). Possono essere caricate in due modi:

- **Librerie Statiche (file `.a`):** Il linker copia letteralmente il codice macchina delle funzioni di libreria utilizzate all'interno del file eseguibile.
 - *Pro:* L'eseguibile è "autocontenuto" (non dipende da file esterni a runtime) ed è facile da avviare per il Sistema Operativo.
 - *Contro:* Se molti programmi usano la `printf`, ogni eseguibile ne avrà una copia, sprestando enorme spazio su disco. Inoltre, se la libreria si aggiorna, bisogna ricompilare tutto.
- **Librerie Dinamiche (file `.so` su Linux, `.dll` su Windows):** Il linker inserisce nell'eseguibile solo un "riferimento" alla libreria e un richiamo a un linker dinamico (es. `/lib/ld-linux.so`). Al momento del lancio, il SO esegue il linker dinamico che carica le librerie in RAM (condividendole tra più programmi) e fa il linking finale.
 - *Pro:* Eseguibili molto più piccoli; possibilità di aggiornare una libreria senza toccare gli eseguibili.
 - *Contro:* Il programma smette di funzionare se la libreria non è presente sul sistema. Il caricamento iniziale è più complesso.

- **Lazy Linking:** Per non rallentare l'avvio, molti linker dinamici operano il caricamento della libreria "pigramente" (lazy), ovvero risolvono i riferimenti e caricano il codice solo la primissima volta che quella specifica funzione viene effettivamente chiamata a runtime.

4.8 Esempio Pratico di Linking

Immaginiamo di dover collegare `file1.o` e `file2.o`. `file1.o` contiene la funzione `func_1` che legge la variabile globale `x` e chiama `func_2`. `file2.o` contiene `func_2` che scrive la variabile globale `y` e chiama `func_1`.

- Il Linker per prima cosa affianca le dimensioni. Supponiamo che il testo di `file1` sia grosso 0x100 byte e quello di `file2` sia 0x200. Il segmento testo totale sarà di 0x300 byte.
- Inizia a mappare in memoria: `file1.text` parte da 0x400000. Di conseguenza `file2.text` partirà dall'indirizzo successivo disponibile, ovvero 0x400100. Allo stesso modo posiziona le variabili del segmento `.data` (ad es. `x` a 0x10000000 e `y` a 0x10000020).
- **Calcolo dei salti (es. `jal`):** Siccome le istruzioni di branch usano indirizzamenti relativi al Program Counter (PC), se da 0x400004 dobbiamo saltare a `func_2` in 0x400100, l'assembler scriverà nell'istruzione il salto relativo: $0x400100 - 0x400004 = 252$ in decimale.
- **Calcolo degli offset (es. `ld/sd`):** Se le load/store utilizzano un registro base (ad esempio `x3` impostato all'inizio della memoria dati 0x10000000), per caricare `y` che si trova all'indirizzo 0x10000020, il linker compilerà la store indicando l'offset 32 decimale (che corrisponde a 0x20).

5 Il Processore: Architettura e Datapath

5.1 Obiettivi e Insieme di Istruzioni

In questa sezione si analizza l'architettura interna di un processore e come questo esegua le istruzioni. I concetti si legano a quanto visto sulle reti logiche, mostrando come i circuiti fisici implementino il set di istruzioni. Per semplicità, faremo riferimento all'implementazione di un sottoinsieme ridotto dell'ISA RISC-V, che comprende tre classi fondamentali:

- **Istruzioni di accesso alla memoria:** `ld` (load doubleword), `sd` (store doubleword).
- **Istruzioni aritmetiche e logiche:** `add`, `sub`, `and`, `or` (tutte appartenenti al Formato R).
- **Istruzioni di salto condizionato:** `beq` (branch if equal).

5.2 Fasi di Esecuzione delle Istruzioni

Tutte le istruzioni, per come è progettata l'architettura RISC-V, condividono una struttura esecutiva comune. In particolare, le prime due fasi sono identiche per ogni istruzione:

1. **Instruction Fetch (Prelievo):** L'istruzione viene prelevata dalla memoria istruzioni utilizzando l'indirizzo contenuto nel Program Counter (PC).
2. **Decode e Lettura dei Registri:** Il processore decodifica l'istruzione ed estrae i valori di uno o due registri operandi dal banco dei registri (*Register File*), leggendo i campi specificati nell'istruzione stessa.

I passi successivi dipendono dalla specifica classe dell'istruzione:

3. **Esecuzione / Calcolo dell'indirizzo / Salto:** La ALU (Unità Aritmetico Logica) viene utilizzata per l'esecuzione vera e propria, per calcolare un indirizzo di memoria (nel caso di load/store) o per verificare la condizione di un salto.
4. **Accesso alla Memoria:** Le istruzioni di load leggono il dato dalla memoria dati, mentre le store lo scrivono.
5. **Write-Back (Scrittura del risultato):** Il risultato dell'operazione (che provenga dalla ALU o dalla Memoria Dati) viene scritto nel registro destinazione all'interno del Register File. Se l'istruzione era un salto, il PC viene aggiornato con l'indirizzo di destinazione.

5.3 Progettazione del Datapath (Percorso dei Dati)

Il *datapath* è l'insieme delle unità funzionali che elaborano i dati. È composto da elementi di stato (memorie, registri, che necessitano di un segnale di clock per aggiornarsi) ed elementi combinatori (ALU, multiplexer, sommatore, la cui uscita dipende esclusivamente dagli ingressi attuali).

5.3.1 Fase di Fetch e Aggiornamento del PC

L'esecuzione inizia dal Program Counter (PC), un registro a 64 bit che contiene l'indirizzo dell'istruzione corrente.

- Il PC fornisce l'indirizzo alla **Memoria Istruzioni**, che restituisce l'istruzione a 32 bit da eseguire.
- Contemporaneamente, un Sommatore (Adder) dedicato incrementa il valore del PC di 4 (poiché le istruzioni RISC-V sono sempre lunghe 32 bit, ovvero 4 byte) per calcolare l'indirizzo dell'istruzione successiva: $PC = PC + 4$.

5.3.2 Istruzioni Aritmetico-Logiche (Tipo R)

Le istruzioni come **add**, **sub**, **and**, **or** operano esclusivamente sui registri.

- **Lettura:** Vengono letti due registri sorgente (**rs1** e **rs2**) dal *Register File*. Il Register File è dotato di due porte di lettura che permettono di estrarre entrambi i valori simultaneamente nel corso dello stesso ciclo di clock.
- **Esecuzione:** I due dati letti (a 64 bit) vengono inviati agli ingressi della ALU, la quale esegue l'operazione specificata dai bit di controllo dell'istruzione.
- **Scrittura:** Il risultato dell'ALU viene reinviato al Register File e scritto nel registro destinazione (**rd**) attraverso l'unica porta di scrittura disponibile.

5.3.3 Istruzioni di Accesso alla Memoria (Load e Store)

Le istruzioni **ld** e **sd** richiedono il calcolo di un indirizzo di memoria base più un offset.

- **Calcolo dell'indirizzo:** L'indirizzo base viene letto da un registro sorgente (**rs1**). L'offset è un valore immediato a 12 bit contenuto direttamente nell'istruzione.
- **Estensione del Segno (*Sign Extension*):** Poiché i registri e le operazioni dell'ALU lavorano a 64 bit, l'offset a 12 bit deve essere esteso a 64 bit mantenendone il segno. A questo provvede un'unità dedicata (*Imm Gen*).
- **Uso dell'ALU:** L'ALU somma l'indirizzo base letto dal registro e l'offset esteso in segno, generando l'indirizzo fisico di memoria a 64 bit.
- **Load (**ld**):** Il dato viene letto dalla Memoria Dati all'indirizzo calcolato e scritto nel registro destinazione (**rd**) del Register File.
- **Store (**sd**):** Un secondo registro (**rs2**) viene letto dal Register File e il suo contenuto viene scritto nella Memoria Dati all'indirizzo calcolato.

5.3.4 Istruzioni di Salto Condizionato (Branch, Tipo SB)

L'istruzione **beq** (branch if equal) confronta due registri e, se sono uguali, salta a un indirizzo calcolato.

- **Confronto:** Due registri sorgente (**rs1** e **rs2**) vengono letti dal Register File e inviati all'ALU. L'ALU esegue una sottrazione. Se il risultato è zero (segnalato dall'uscita hardware *Zero flag* dell'ALU), i due registri sono uguali e il salto deve essere effettuato (*branch taken*).
- **Calcolo dell'Indirizzo di Salto:** L'indirizzo di destinazione è *relativo al PC corrente*. Si calcola sommando l'attuale PC a un offset immagazzinato nell'istruzione a 12 bit.
- **Estensione e Shift:** L'offset a 12 bit viene esteso in segno a 64 bit. L'architettura RISC-V stabilisce che questo offset indichi uno spiazzamento in termini di "mezze parole" (half-words, 2 byte). Pertanto, l'offset esteso deve essere **shiftato a sinistra di 1 bit** (moltiplicato per 2) prima di essere sommato al PC tramite un secondo sommatore dedicato.
- **Aggiornamento del PC:** Se il salto è preso (*Zero flag* = 1), il PC viene aggiornato con il nuovo indirizzo calcolato. Altrimenti, il PC assumerà il normale valore sequenziale $PC + 4$.

5.4 Composizione del Datapath (Multiplexer e Controllo)

Costruendo un processore *uniciclo* (dove ogni istruzione viene completata in un solo ciclo di clock), i vari componenti (ALU, Memoria, Register File) devono essere condivisi dalle diverse istruzioni. Poiché non possiamo fisicamente collegare più cavi agli stessi ingressi, si utilizzano i **Multiplexer (Mux)**, controllati dall'Unità di Controllo. Le principali scelte di instradamento (multiplexing) nel datapath sono:

1. **Ingresso della ALU (Sorgente 2):** Per le istruzioni Tipo R, il secondo ingresso dell'ALU è un registro (**rs2**). Per le Load/Store, il secondo ingresso è il valore immediato esteso in segno. Un Mux seleziona quale dei due dati debba entrare nella ALU.
2. **Dato da scrivere nel Register File:** Per le istruzioni Tipo R, il dato da scrivere nel registro **rd** proviene dall'uscita della ALU. Per le istruzioni di Load, il dato proviene dall'uscita della Memoria Dati. Un Mux decide quale dei due valori inviare alla porta di scrittura del Register File.
3. **Prossimo valore del PC:** Un Mux finale determina se il prossimo PC dovrà essere il PC sequenziale ($PC + 4$) oppure l'indirizzo di salto calcolato dal sommatore dedicato (se si tratta di una **beq** con condizione verificata).

6 L'Unità di Controllo e l'Implementazione del Datapath

6.1 Progetto di un'Unità di Elaborazione a Ciclo Singolo

Il nostro obiettivo iniziale è progettare un processore in cui ogni istruzione venga eseguita interamente in **un solo ciclo di clock** (architettura a ciclo singolo). Questo vincolo progettuale impone una regola hardware fondamentale: *non è possibile utilizzare una stessa unità funzionale più di una volta nello stesso ciclo di clock*. Da questo derivano due conseguenze principali per l'architettura:

1. **Separazione delle memorie:** È necessario avere due memorie fisicamente distinte (o due cache separate), una per le istruzioni (*Memoria Istruzioni*) e una per i dati (*Memoria Dati*), poiché non potremmo accedervi due volte nello stesso ciclo (una volta per fare il fetch dell'istruzione e una per caricare/salvare il dato).
2. **Uso dei Multiplexer:** Le unità funzionali, come la ALU, devono essere condivise il più possibile. Per instradare i dati corretti verso queste unità a seconda dell'istruzione corrente, si inseriscono dei **Multiplexer (Mux)**, governati da appositi segnali di controllo.

6.2 Panoramica dei Segnali di Controllo Principali

Il funzionamento del processore è guidato dall'**Unità di Controllo Principale**, che decodifica l'istruzione e attiva una serie di segnali hardware (solitamente a 1 bit) che configurano il datapath. Di seguito i segnali fondamentali:

- **RegWrite:** Se asserito (1), il dato presente sulla linea "Dato Scritto" viene memorizzato nel registro di destinazione all'interno del Register File. Se non asserito (0), non avviene alcuna scrittura.
- **ALUSrc:** Determina il secondo operando della ALU. Se 0, l'operando proviene dal Register File (Registro 2). Se 1, l'operando è il valore immediato estratto dall'istruzione (esteso in segno).

- **PCSrc**: Controlla il Multiplexer del Program Counter (PC). Se 0, il prossimo PC sarà $PC + 4$. Se 1, il prossimo PC sarà l'indirizzo di salto calcolato dal sommatore dedicato ai branch.
- **MemRead**: Se asserito (1), abilita la lettura dalla Memoria Dati all'indirizzo fornito.
- **MemWrite**: Se asserito (1), abilita la scrittura nella Memoria Dati.
- **MemtoReg**: Determina quale dato debba essere scritto nel Register File. Se 0, passa il risultato calcolato dalla ALU. Se 1, passa il dato appena letto dalla Memoria Dati.

6.3 Decodifica a Livelli Multipli e Controllo dell'ALU

Per semplificare la progettazione hardware, la generazione dei comandi è divisa in **due livelli**. Il primo livello (Unità di Controllo Principale) genera un segnale intermedio a 2 bit chiamato **ALUOp**. Il secondo livello (Unità di Controllo dell'ALU) prende **ALUOp** e i bit di funzione dell'istruzione per decidere l'operazione esatta della ALU.

La ALU deve poter eseguire: SOMMA (0010), SOTTRAZIONE (0110), AND (0000) e OR (0001). I 2 bit di **ALUOp** indicano la classe dell'istruzione:

- **ALUOp = 00** (Load/Store): La ALU deve sommare l'indirizzo base all'offset. L'operazione della ALU sarà una somma.
- **ALUOp = 01** (Branch condizionato, beq): La ALU deve confrontare i due registri, quindi effettua una sottrazione (se il risultato è zero, i registri sono uguali).
- **ALUOp = 10** (Istruzioni di Tipo R): La ALU deve eseguire un'operazione specifica. L'unità di controllo dell'ALU leggerà i campi **funct7** (bit 30) e **funct3** (bit 14-12) dell'istruzione per determinare se fare add, sub, and o or.

Il controllore della ALU usa una logica combinatoria ottimizzata (tramite valori "don't care" o wildcards indicati con X) per limitare il numero di porte logiche necessarie: non serve mappare tutti i possibili 4096 input (12 bit), ma solo analizzare i bit rilevanti a seconda di **ALUOp**.

6.4 Decodifica dell'Istruzione

L'Unità di Controllo Principale analizza unicamente il campo **Opcode (codice operativo)**, localizzato sempre nei bit [6:0] dell'istruzione a prescindere dal formato. L'architettura RISC-V posiziona i campi dei registri sempre negli stessi bit per semplificare l'hardware:

- **rs1** (registro sorgente 1) è sempre nei bit [19:15].
- **rs2** (registro sorgente 2) è sempre nei bit [24:20] (usato nei Tipi R, S, SB).
- **rd** (registro destinazione) è sempre nei bit [11:7] (usato nei Tipi R, I).

Questo permette al Register File di iniziare a leggere i registri immediatamente, contemporaneamente all'azione di decodifica dell'Unità di Controllo.

6.5 Esecuzione Passo-Passo sul Processore

Vediamo cosa succede nei vari componenti del datapath durante un singolo ciclo per le istruzioni principali:

6.5.1 Istruzione di Tipo R (es. `add x5, x6, x7`)

1. **Prelievo:** Si preleva l'istruzione dalla Memoria Istruzioni tramite il PC e si incrementa il PC di 4.
2. **Lettura e Decodifica:** Si leggono i registri `x6` e `x7`. L'Unità di Controllo setta `ALUSrc = 0` (la ALU usa i due registri), `MemtoReg = 0` (il risultato ALU va al registro), e `RegWrite = 1`. `ALUOp = 10`.
3. **Esecuzione:** La ALU, comandata dal controller secondario (che legge i bit di funzione per `add`), somma i due registri.
4. **Scrittura:** Il risultato oltrepassa la Memoria Dati (che è disattivata) e arriva al Mux finale che lo instrada per scriverlo in `x5`.

6.5.2 Istruzione di Load (es. `ld x5, offset(x6)`)

1. **Prelievo e Lettura:** Come sopra, ma si legge solo `x6`.
2. **Esecuzione (Calcolo Indirizzo):** L'Unità di Controllo setta `ALUSrc = 1`, instradando il valore dell'offset a 12 bit (esteso in segno a 64 bit) nel secondo ingresso della ALU. `ALUOp = 00`, per cui la ALU fa una somma tra `x6` e l'offset.
3. **Accesso in Memoria:** L'output della ALU diventa l'indirizzo per la Memoria Dati. `MemRead = 1`, quindi il dato viene letto.
4. **Scrittura:** `MemtoReg = 1`, quindi il Multiplexer finale instrada il dato appena letto verso il Register File, per essere scritto in `x5`.

La `sd` funziona similmente, ma `MemWrite = 1`, `RegWrite = 0`, e il dato contenuto in `rs2` viene inviato alla porta "Dato da scrivere" della memoria.

6.5.3 Istruzione di Salto Condizionato (es. `beq x5, x6, offset`)

1. **Lettura e Confronto:** Vengono letti `x5` e `x6`. La ALU (con `ALUOp = 01`) sottrae i due registri.
2. **Calcolo dell'Indirizzo:** Contemporaneamente, il sommatore del salto somma l'attuale PC all'offset immediato. (Nota architettónica: l'offset viene esteso in segno e fatto scorrere a sinistra di 1 bit (*shift left logical* di 1, non di 2, a causa delle istruzioni compresse di RISC-V a 16 bit)).
3. **Aggiornamento PC:** L'Unità di Controllo setta `Branch = 1`. Se la ALU certifica l'uguaglianza dei registri, la sua uscita `Zero` diventa 1. Un operatore logico AND hardware tra il segnale `Branch` e `Zero` pilota il Multiplexer `PCSrc`, decidendo se scrivere `PC+4` o il nuovo indirizzo di salto. `RegWrite` e `MemWrite` sono rigorosamente a 0.

6.6 Sintesi dei Segnali per l'Unità di Controllo

Riassumiamo la configurazione completa dei segnali generati dall'Unità di Controllo Principale a seconda del tipo di istruzione:

Istruzione	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
Tipo R	0	0	1	0	0	0	1	0
ld	1	1	1	1	0	0	0	0
sd	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

Tabella 1: Tabella della Verità per i segnali dell'Unità di Controllo Principale.

L'implementazione fisica in silicio deriverà semplicemente le porte logiche (AND, OR) partendo da questa tabella, leggendo i 7 bit dell'Opcod.

6.7 Considerazioni Conclusive sull'Architettura a Singolo Ciclo

Sebbene il processore a singolo ciclo sia semplice ed elegante dal punto di vista didattico, **non è utilizzato nei computer moderni**. Le motivazioni principali sono:

- **Frequenza di Clock vincolata:** L'istruzione più lunga (ovvero la Load, che deve percorrere Memoria Istruzioni, Register File, ALU, Memoria Dati e di nuovo Register File) stabilisce il *cammino critico*. Il periodo del clock deve essere sufficientemente lungo per accomodare l'istruzione peggiore in assoluto.
- **Istruzioni complesse:** Introducendo istruzioni hardware molto lente (ad esempio operazioni in virgola mobile, floating point), il periodo del clock crollerebbe in modo inaccettabile, rallentando drasticamente tutte le altre istruzioni più semplici che avrebbero terminato molto prima.
- **Mancanza di ottimizzazione:** Non è possibile ottimizzare pesantemente le operazioni svolte in modo sequenziale.

Per risolvere queste drammatiche inefficienze temporali, le architetture reali abbandonano l'approccio a ciclo singolo in favore del **Pipelining**, dove l'esecuzione viene spezzata su più cicli brevi, sovrapponendo istruzioni diverse in varie fasi.

7 La Pipeline

7.1 Limiti dell'Architettura a Singolo Ciclo

Nella lezione precedente abbiamo analizzato un processore in cui ogni istruzione viene eseguita interamente in un singolo ciclo di clock. Questo approccio, seppur semplice, non viene più utilizzato nelle architetture moderne per diversi motivi:

- **Frequenza del clock limitata:** Il periodo di clock (che deve essere comune a tutte le istruzioni) è dettato dall'istruzione più lenta, tipicamente quella di accesso alla memoria (ld).

- **Inefficienza con istruzioni complesse:** Aggiungendo operazioni hardware lente (come la virgola mobile), il tempo di ciclo peggiorerebbe drasticamente per tutte le istruzioni, comprese quelle veloci.
- **Mancanza di ottimizzazione:** Non si riesce a sfruttare il parallelismo per velocizzare le operazioni eseguite più frequentemente.

7.2 L'Idea di Base: L'Analogia della Catena di Montaggio

Per superare questi limiti, si applica il concetto della **catena di montaggio** (introdotto da Henry Ford). Possiamo comprenderne i vantaggi con l'analogia del bucato. Supponiamo che fare il bucato richieda 4 fasi: Lavatrice (30 min), Asciugatrice (40 min), Piegatura (20 min) e Sistemazione nell'armadio (10 min).

- **Esecuzione Sequenziale:** Se abbiamo 4 carichi di bucato, eseguendoli uno dopo l'altro senza sovrapposizioni impiegheremmo $4 \times 100 \text{ min} = 400 \text{ min}$.
- **Esecuzione in Pipeline:** Appena la lavatrice ha finito il primo carico e lo passa all'asciugatrice, possiamo inserire immediatamente il secondo carico nella lavatrice. In questo modo, le diverse fasi lavorano in parallelo su carichi diversi. Il tempo totale per 4 carichi scende a 190 min.

Lezioni fondamentali della Pipeline:

1. La pipeline *non migliora la latenza* (il tempo necessario a completare un singolo carico resta 100 min).
2. La pipeline **migliora il throughput** (la quantità di lavoro completato nell'unità di tempo).
3. Lo *Speedup* (fattore di accelerazione) teorico è pari al numero di stadi della pipeline.
4. La velocità della pipeline è **limitata dallo stadio più lento** (nell'esempio, l'asciugatrice da 40 min).
5. Gli stadi non perfettamente bilanciati e i tempi di "riempimento" e "svuotamento" della pipeline riducono lo speedup rispetto al valore ideale.

7.3 La Pipeline nel Processore RISC-V

Applichiamo questa idea al processore. L'esecuzione di un'istruzione RISC-V viene tradizionalmente suddivisa in **5 stadi temporali**:

1. **IF (*Instruction Fetch*):** Prelievo dell'istruzione dalla memoria.
2. **ID (*Instruction Decode & Register Read*):** Decodifica dell'istruzione e lettura dei registri operandi.
3. **EX (*Execute*):** Esecuzione dell'operazione logico-aritmetica o calcolo dell'indirizzo.
4. **MEM (*Memory Access*):** Lettura o scrittura nella memoria dati.
5. **WB (*Write Back*):** Scrittura del risultato nel banco dei registri.

Vantaggio temporale (Esempio): Supponiamo che i tempi dei singoli stadi siano: IF=200ps, ID=100ps, EX=200ps, MEM=200ps, WB=100ps. In un processore a singolo ciclo, il clock deve durare la somma di questi tempi: **800 ps**. In un processore con pipeline, il clock deve durare solo quanto lo stadio più lento: **200 ps**. Le istruzioni si susseguono entrando una dopo l'altra ogni 200 ps.

7.3.1 Perché RISC-V è ottimizzato per la Pipeline?

- **Istruzioni a lunghezza fissa:** Tutte lunghe 32 bit, rendendo il fetch semplice e predicibile (nella prima fase).
- **Pochi formati di istruzione:** I campi dei registri sono sempre negli stessi bit, permettendo alla fase ID di leggere i registri contemporaneamente alla decodifica.
- **Architettura Load/Store:** Solo ld e sd accedono alla memoria. La fase MEM non interferisce con l'aritmetica.

7.4 I Conflitti o Alee (*Hazards*)

La sovrapposizione delle istruzioni può causare problemi quando un'istruzione dipende da qualcosa non ancora completato dall'istruzione precedente. Questi problemi si chiamano **Hazards** e si dividono in tre categorie.

7.4.1 1. Conflitti Strutturali (*Structural Hazards*)

Si verificano quando due istruzioni diverse tentano di usare la **stessa risorsa hardware** nello stesso ciclo di clock.

- **Esempio Memoria:** Se avessimo un'unica memoria per dati e istruzioni, l'istruzione N (che fa fetch in IF) colliderebbe con l'istruzione $N - 3$ (che fa load/store in MEM). *Soluzione:* Architettura Harvard con memorie fisiche o cache separate per Dati e Istruzioni.
- **Esempio Register File:** Una scrittura in WB colliderebbe con una lettura in ID. *Soluzione:* Scrivere nel registro durante la prima metà del ciclo di clock e leggere nella seconda metà.

7.4.2 2. Conflitti sui Dati (*Data Hazards*)

Avvengono quando un'istruzione ha bisogno del risultato di un'istruzione precedente che è ancora in viaggio nella pipeline (non ha ancora raggiunto la fase WB).

```
add x19, x0, x1
sub x2, x19, x3
```

La **sub** necessita di **x19** nella sua fase EX, ma la **add** non lo scriverà fino alla sua fase WB.

- **Propagazione (Forwarding o Bypassing):** Aggiunta di hardware dedicato che "intercetta" il risultato appena esce dalla ALU o dalla Memoria (nei registri di pipeline EX/MEM o MEM/WB) e lo re-invia (bypassa) all'ingresso della ALU per l'istruzione successiva, risolvendo il conflitto senza rallentamenti.

- **Load-Use Hazard:** Il Forwarding non basta se l'istruzione precedente è una `ld`. Il dato viene letto dalla memoria solo alla fine della fase MEM. È impossibile mandare indietro nel tempo il dato per farlo arrivare all'inizio della fase EX dell'istruzione immediatamente successiva.
- **Stallo (Bolla):** L'hardware rileva il Load-Use hazard e *mette in pausa* (stalla) la pipeline inserendo un'istruzione nulla (bolla) per 1 ciclo di clock, dando tempo al dato di uscire dalla memoria ed essere propagato.
- **Code Scheduling (Pianificazione del Codice):** Per evitare lo stallo, il compilatore cerca di riordinare il codice macchina. Inserisce tra la `ld` e l'istruzione dipendente un'altra istruzione indipendente, nascondendo la latenza e mantenendo in funzione la pipeline.

7.4.3 3. Conflitti di Controllo (*Branch Hazards*)

Nascono dalle istruzioni di salto condizionato (`beq`). Quando si esegue il fetch dell'istruzione successiva a una `beq`, non sappiamo ancora se il salto verrà preso oppure no, perché la condizione viene calcolata solo nelle fasi successive (solitamente ID o EX).

- **Stallo su salto:** Aspettare il calcolo perdendo interi cicli di clock. Fortemente inefficiente.
- **Predizione del Salto (*Branch Prediction*):** Il processore "tira a indovinare" l'esito del salto e continua a fare il fetch sulla base della previsione.
 - Se la predizione è **corretta**, non si perde alcun ciclo.
 - Se la predizione è **sbagliata**, la pipeline deve essere "svuotata" (Flush): le istruzioni caricate per errore vengono scartate (trasformate in bolle) e si riprende dall'indirizzo corretto, pagando una penalità di cicli.
- **Tecniche di predizione:**
 - *Statica:* Ad esempio, ipotizzare sempre che il salto "non venga effettuato" (Not Taken).
 - *Dinamica:* Si usa circuiteria aggiuntiva basata sulla storia recente (History Buffer). I cicli (*loops*) vengono generalmente eseguiti molte volte e non presi solo alla fine; la predizione dinamica impara questi schemi e diventa molto accurata.

8 La Gerarchia di Memoria

8.1 Organizzazione e Concetti di Base

Un elaboratore non può funzionare senza memoria. Poiché il sogno di avere una memoria infinitamente grande e con accessi ultrarapidi si scontra con i limiti fisici ed economici, i moderni sistemi adottano un'organizzazione basata su compromessi tra **costo**, **prestazioni** e **dimensione**.

Le memorie si dividono fondamentalmente in base all'indirizzamento:

- **Ad indirizzamento diretto (Memoria principale e Cache):** Le informazioni sono accessibili al processore in qualsiasi momento. Si tratta di memoria *volatile* (il cui contenuto viene perso se si spegne l'elaboratore) ed è limitata dallo spazio di indirizzamento nativo del processore.
- **Ad indirizzamento indiretto (Memoria periferica/di massa):** È di tipo *permanente* (mantiene il contenuto anche senza alimentazione) e ha uno spazio di indirizzamento "software" teoricamente non limitato dal processore. Tuttavia, le informazioni devono prima essere trasferite nella memoria principale per essere lette, tramite un'operazione mediata dal software (tipicamente dal Sistema Operativo).

8.2 Terminologia Fondamentale

Per valutare e classificare le memorie si utilizzano parametri specifici:

- **Tempo di accesso:** Il tempo richiesto per completare una singola operazione di lettura o scrittura.
- **Tempo di ciclo:** Il tempo che deve intercorrere tra l'inizio di due operazioni consecutive verso locazioni diverse (è sempre superiore al tempo di accesso per via dei ritardi di ripristino).
- **Accesso Casuale (RAM - Random Access Memory):** Non vi è alcun ordine fisico stringente nei dati; il tempo di accesso è indipendente dalla posizione dell'informazione. È tipico delle memorie a semiconduttori, che sono leggibili e scrivibili.
- **ROM (Read Only Memory):** Memorie a semiconduttori di sola lettura (ad accesso casuale o sequenziale).
- **Accesso Sequenziale:** L'accesso è ordinato e il tempo necessario dipende dalla posizione fisica del dato (tipico dei dischi e dei nastri).

8.3 Interfaccia della Memoria Principale

Il processore (Master) si interfaccia con la memoria (Slave) tramite dei registri specifici e un set di bus:

- **MAR (Memory Address Register):** Invia l'indirizzo della locazione di memoria a cui accedere tramite il *bus degli indirizzi* a k bit (indirizzando fino a 2^k locazioni).
- **MDR (Memory Data Register):** Invia o riceve il dato effettivo tramite il *bus dei dati* a n bit (che definisce la word-length).
- **Linee di Controllo:** Trasportano i segnali come $R/\overline{W}$ (che indica alla memoria se leggere o scrivere) e il segnale *MFC (Memory Function Completed)*, inviato dalla memoria per confermare la fine dell'operazione.

Una memoria di capacità N può avere un'organizzazione basata su un **parallelismo** differente. Ad esempio, una memoria da 512 Kbit può essere configurata come $512K \times 1$ bit, $128K \times 4$ bit, oppure $64K \times 8$ bit. Questa architettura influenza direttamente il numero di pin di Input/Output del circuito integrato.

8.4 Memorie Statiche (SRAM)

Le SRAM conservano il loro valore indefinitamente finché il circuito è alimentato.

- **Prestazioni e Costi:** Sono estremamente veloci (il tempo di accesso è nell'ordine dei nanosecondi), e consumano poca corrente. Costano però molto care poiché la densità è bassa (ogni cella è formata da molti transistor).
- **Struttura della cella:** Utilizzano un circuito detto *latch a doppio NOT* (due porte NOT collegate in anello chiuso) che memorizza i punti stabili X e Y . La cella si connette a due **Bit lines** (b e b') tramite due transistor che fungono da interruttori, pilotati dalla **Word line**.
- **Lettura e Scrittura:** L'accesso è mediato dai *sense/write circuit*. In scrittura, il valore e il suo negato vengono posti su b e b' , la Word Line si alza, e il valore forza il latch. In lettura, le bit line restano in alta impedenza, la Word Line chiude il circuito, e il valore interno polarizza le bit lines. La ridondanza di avere sia il bit che il suo negato (b e b') minimizza il tasso di errori.

8.5 Memorie Dinamiche (DRAM)

Le DRAM sono le memorie più diffuse per la RAM dei PC per via del loro insuperabile rapporto capacità/prezzo.

- **Struttura della cella:** La cella base ha un'incredibile semplicità: utilizza un solo transistor (T) come interruttore e un piccolo **condensatore** (C) per memorizzare il bit sotto forma di carica elettrica.
- **Lettura e Scrittura:** In scrittura, il valore chiude l'interruttore e carica o scarica il condensatore. In lettura, la flebile carica sul condensatore altera la tensione della bit line. Un **Sense Amplifier** rileva questo calo o aumento: se supera la soglia, spinge la bit line all'alimentazione (ricaricando C); se è sotto soglia, mette a terra la linea (scaricando C).
- **Il problema del Refresh:** Le correnti parassite fanno sì che il condensatore si scarichi col tempo. Per evitare che i dati "svaniscano", è obbligatorio effettuare un ciclo continuo di **rinfresco (refresh)** tramite letture periodiche governate da circuiti dedicati, motivo per cui le DRAM presentano consumi più elevati.

8.5.1 Multiplazione degli Indirizzi

Per via dell'altissima densità di locazioni, avere pin hardware sufficienti per l'intero indirizzo è irrealizzabile. Si **multiplano nel tempo** gli indirizzi di riga e di colonna sugli stessi fili fisici:

- **RAS (Row Address Strobe):** Segnale che avvisa la DRAM che i bit sul bus rappresentano l'indirizzo della riga (Row).
- **CAS (Column Address Strobe):** Segnale che avvisa che i bit sul bus rappresentano l'indirizzo della colonna (Col).

8.6 Evoluzione della DRAM

Per aumentare l'efficienza nel dialogo con la CPU, sono state sviluppate varie architetture DRAM:

- **FPM (Fast Page Mode):** Spesso i dati vengono prelevati sequenzialmente. Con la FPM, invece di ri-assestare tutta la circuiteria per ogni accesso, si seleziona la riga (pagina) una sola volta (tramite RAS). Finché gli accessi rimangono nella stessa riga, si inviano solo i segnali CAS di colonna, risparmiando un tempo preziosissimo.
- **SDRAM (Synchronous DRAM):** Le vecchie DRAM sono *asincrone* e richiedono attese precise sui ritardi elettrici. Aggiungendo dei registri (latch) si sincronizzano del tutto gli accessi con il **clock** della CPU, disaccoppiando le letture dai cicli di rinfresco e consentendo burst continui in Fast Page Mode in perfetta sintonia.
- **DDR-SDRAM (Double Data Rate):** Questa SDRAM permette il trasferimento dei dati in FPM sia sul fronte positivo (**salita**) che su quello negativo (**discesa**) del segnale di clock. La *latenza* resta immutata, ma la *banda* passante risulta doppia. Il trucco consiste nell'avere due banchi separati interni: uno per i dati pari e uno per i dispari, accedendo ad essi in maniera interlacciata.

8.7 Metriche di Prestazione

- **Latenza:** È il tempo di accesso a una singola parola. È la misura fondamentale che dà l'indicazione di quanto, nel caso peggiore, il processore resterà "in stallo" attendendo l'inizio dell'arrivo di un blocco di dati.
- **Velocità (Banda):** È la velocità massima di trasferimento dati in blocco (FPM/DDR). Risulta cruciale per alimentare rapidamente le memorie Cache interne al processore e per trasferimenti rapidi con dispositivi di periferica tramite accesso diretto (DMA).

9 Memoria Cache: Organizzazione e Funzionamento

9.1 L'Illusione di una Memoria Ideale e le Località

Il problema fondamentale dell'architettura dei calcolatori, noto fin dai tempi di Von Neumann, è il desiderio di avere una memoria infinitamente grande e al tempo stesso accessibile istantaneamente. Poiché fisicamente impossibile, si ricorre a una **gerarchia di memoria** (spiegata con l'analogia di un impiegato che tiene i documenti di uso frequente sulla scrivania e gli altri in un grande archivio). Questa illusione di avere "tanta memoria veloce" funziona unicamente grazie a due principi:

- **Località Temporale:** Se si fa uso di una locazione di memoria, è altamente probabile che la si riutilizzi a breve. Nel codice, questo si verifica costantemente con le istruzioni all'interno di un ciclo (*loop*) o con le variabili contatore.
- **Località Spaziale:** Se si accede a una locazione, è molto probabile che nei passi successivi si acceda a locazioni con indirizzi vicini. Questo avviene con la lettura sequenziale delle istruzioni in memoria o quando si scorre un array parola per parola.

9.2 Terminologia Fondamentale

La **Cache** è una memoria piccola e veloce, nascosta al programmatore (gestita in modo totalmente trasparente dall'hardware), che si interpone tra CPU e Memoria Principale. Per valutarne le prestazioni si usano metriche specifiche:

- **Blocco (o Linea):** L'unità minima di informazione che viene spostata tra i vari livelli della gerarchia.
- **Hit Rate (Frequenza di successo):** La frazione di accessi in cui il processore trova il dato cercato direttamente nella cache. Il tempo necessario per questo accesso è il **Tempo di Hit**.
- **Miss Rate:** Frazione degli accessi in cui il dato non è presente in cache ($1 - \text{Hit Rate}$).
- **Miss Penalty (Penalità di Miss):** Il tempo necessario per recuperare il blocco mancante dal livello inferiore (memoria principale) e portarlo in cache. Questo tempo è enormemente superiore al tempo di hit, per cui minimizzare i miss è cruciale per le prestazioni.

9.3 Politiche di Mappatura della Cache

Quando la CPU richiede un indirizzo, come fa a sapere se il dato è in cache e dove si trova? Esistono tre architetture principali per mappare i blocchi di memoria nella cache.

9.3.1 1. Cache a Mappatura Diretta (*Direct Mapped*)

Ogni blocco della memoria principale può essere ospitato in **un'unica e specifica posizione** all'interno della cache.

- **Calcolo della posizione:** L'indice della cache si calcola con l'operazione (Indirizzo del Blocco) MODULO (Numero di blocchi nella cache).
- **Il problema dell'identificazione (Tag):** Poiché milioni di indirizzi di memoria condividono lo stesso indice della cache, ogni blocco in cache è accompagnato da un **Tag** (i bit più significativi dell'indirizzo originale) per confermare l'identità del dato presente.
- **Bit di Validità (V):** Un singolo bit che indica se i dati contenuti in quel blocco di cache sono validi o se contengono solo "spazzatura" residua dall'accensione.

9.3.2 2. Cache Completamente Associativa (*Fully Associative*)

Un blocco di memoria principale può essere posizionato in **qualsiasi** blocco vuoto della cache.

- **Funzionamento:** Non essendoci un indice fisso, l'indirizzo richiesto deve essere confrontato simultaneamente con *tutti* i Tag presenti nella cache.
- **Pro e Contro:** Minimizza i *miss* (poiché non c'è conflitto su indici specifici), ma richiede un hardware estremamente costoso (un comparatore per ogni blocco), rendendola fattibile solo per cache piccolissime.

9.3.3 3. Cache Set-Associativa (*N-vie*)

È il compromesso universalmente adottato. La cache è divisa in "insiemi" (*Set*). Un blocco di memoria viene mappato su uno specifico Set (tramite operazione modulo), ma all'interno di quel Set può essere inserito in **una qualsiasi delle N vie** disponibili.

- Combina la semplicità di calcolo dell'indice della mappatura diretta con la flessibilità dell'associativa (all'interno del singolo Set la ricerca avviene in parallelo).
- **Politiche di rimpiazzo:** Quando un Set è pieno e arriva un nuovo blocco, bisogna decidere quale vecchio blocco scartare. Si usa spesso la politica **LRU** (*Least Recently Used*), che scarta il blocco non utilizzato da più tempo, richiedendo bit aggiuntivi per tenere traccia della "storia" degli accessi.

9.4 Struttura di un Indirizzo e Calcolo

Prendendo ad esempio un'architettura a 64 bit e una cache a mappatura diretta, l'indirizzo generato dalla CPU viene idealmente spezzato in tre campi hardware:

1. **Offset del Byte/Blocco:** I bit meno significativi indicano il singolo byte all'interno della parola e della specifica parola all'interno del blocco.
2. **Indice:** I bit centrali selezionano la riga (il blocco) della cache. Se la cache ha 1024 blocchi, serviranno 10 bit ($2^{10} = 1024$).
3. **Tag:** I restanti bit più significativi. Vengono inviati a un comparatore logico: se coincidono col Tag salvato nella riga indicizzata, e il bit V è a 1, abbiamo una **Hit**.

Esempio di indirizzamento: Dato un indirizzo al byte 1200, e una cache con blocchi da 16 byte. Indirizzo del blocco = $1200/16 = 75$. Se la cache ha 64 blocchi totali, la posizione in cache sarà: $75 \pmod{64} = 11$. Il blocco andrà nella riga 11.

9.5 Il Trade-off sulla Dimensione del Blocco

Aumentare la grandezza del blocco esalta la località spaziale (si portano in cache più dati vicini in un sol colpo), diminuendo inizialmente i miss. Tuttavia, se la dimensione totale della cache è fissa, blocchi più grandi significano averne di meno a disposizione, peggiorando la località temporale e aumentando spaventosamente la penalità di miss (ci vuole molto più tempo per trasferire un blocco gigantesco sul bus). Esiste quindi una dimensione "ottima" che minimizza i miss per una data capienza.

9.6 Gestione delle Miss e delle Scritture

L'integrazione della cache con la Pipeline della CPU introduce alcune logiche di controllo:

- **Gestione del Miss:** Se c'è un miss (es. sull'istruzione), la pipeline va in stallo. Il controller invia l'indirizzo mancante alla memoria, attende l'arrivo del blocco, aggiorna la cache (Dato, Tag e Valid bit), e fa ripartire il fetch dell'istruzione bloccata.
- **Politica Write-Through:** Ad ogni istruzione di *Store*, il dato viene scritto **sia** nella cache **sia** nella Memoria Principale. È semplice e mantiene la memoria sempre coerente, ma è lentissima (si usano *buffer di scrittura* per mitigare l'attesa).
- **Politica Write-Back:** La scrittura avviene **solo** nella cache. La Memoria Principale non viene aggiornata immediatamente. Il blocco modificato verrà riversato in memoria unicamente quando la cache sarà costretta a sovrascriverlo per far spazio a nuovi dati. È molto più efficiente in presenza di scritture ripetute sulla stessa variabile.

10 Sistemi di Input/Output e Interfacciamento

10.1 La Necessità di Comunicare e Classificazione

Un calcolatore, per quanto potente, risulta completamente inutile senza la capacità di caricare o salvare dati e di comunicare con il mondo esterno. I dispositivi di Input/Output (I/O) garantiscono questa interazione e sono caratterizzati dall'essere **espandibili** ed **estremamente eterogenei**.

Esistono diverse categorie di periferiche, classificabili secondo tre criteri principali:

- **Comportamento:**
 - *Input*: dispositivi letti una sola volta (es. tastiera, mouse).
 - *Output*: dispositivi scritti (e raramente riletti), come i display o le stampanti.
 - *Storage*: memorie di massa lette e scritte più volte (es. dischi magnetici, SSD).
- **Partner di interazione:** Interazione con un *umano* (monitor, tastiera, dove contano i tempi di reazione percepiti) o con un'altra *macchina* (reti, hard disk, dove conta massimizzare i dati trasferiti).
- **Prestazioni (Data Rate):** Si va da pochi Byte al secondo (tastiera) fino a svariati GigaByte al secondo (schede video o di rete ad alta velocità). Esistono periferiche per cui è fondamentale minimizzare la **Latenza** (il tempo di risposta al singolo comando) e altre progettate per massimizzare il **Throughput** (quantità di dati trasmessi nell'unità di tempo).

10.2 Tipologie di Dispositivi di Storage

10.2.1 I Dischi Magnetici (Hard Disk)

Rappresentano la tecnologia storica per lo storage di massa. Sono memorie magnetiche non volatili, composte da piatti rotanti e testine mobili. Sono enormemente più lenti della memoria RAM (fino a 10.000 volte) ma anche drasticamente più economici.

Il **tempo di accesso** a un disco magnetico è la somma di diverse componenti ritardanti:

1. **Queuing delay:** Il tempo di attesa in coda prima che il comando venga processato.
2. **Seek time (Tempo di ricerca):** Il tempo meccanico impiegato dalla testina per spostarsi radialmente sulla traccia corretta del disco.
3. **Rotational latency (Latenza rotazionale):** Il tempo di attesa affinché la rotazione del disco porti il settore cercato esattamente sotto la testina (in media si considera il tempo pari a mezza rotazione).
4. **Data transfer time:** Il tempo materiale per leggere i byte dal supporto magnetico alla velocità di trasferimento nominale del disco.
5. **Controller overhead:** Il tempo impiegato dal chip controllore del disco per gestire l'intera operazione logica.

Ad esempio, per un disco a 10.000 RPM (Rotazioni Per Minuto), la rotazione completa richiede 6 ms, quindi la latenza rotazionale media sarà di 3 ms. Se il *seek time* medio è di 6 ms e trasferiamo dati minuscoli (es. un settore da 512 byte), i tempi meccanici dominano in modo schiacciante (oltre 9 ms totali) rispetto al tempo effettivo di trasferimento dati (frazioni di millisecondo).

10.2.2 Memorie Flash e SSD (Solid State Drive)

Sono basate su tecnologia a semiconduttori (EEPROM) e sono completamente prive di parti meccaniche in movimento. Rispetto ai dischi magnetici offrono dimensioni ridotte, bassissimo consumo energetico e un'altissima resistenza agli urti meccanici (motivo per cui dominano il mercato mobile e dei laptop). Costano meno della DRAM, ma più degli Hard Disk tradizionali. Si dividono in:

- **NOR Flash:** Permettono l'accesso casuale a singole word di memoria. Utilizzate prevalentemente per contenere i firmware dei dispositivi.
- **NAND Flash:** Organizzate a blocchi, più dense ed economiche, rappresentano il cuore degli attuali SSD e delle memorie USB.

10.3 I Bus di Comunicazione e la Sincronizzazione

I vari dispositivi sono fisicamente collegati al processore tramite i **Bus**, ovvero canali di comunicazione condivisi su cui viaggiano gli indirizzi, i dati e i segnali di controllo.

- **Vantaggi:** Versatilità (è facile aggiungere nuovi dispositivi) e costi bassissimi, poiché le linee elettriche sono uniche e condivise.
- **Svantaggi:** Costituiscono un enorme collo di bottiglia per le prestazioni. La banda passante massima del bus limita il throughput totale del sistema.

Per regolare la trasmissione tra un *Master* (es. la CPU) e uno *Slave* (es. la memoria o una periferica), il Bus deve impiegare un protocollo di sincronizzazione:

- **Sincrono:** Usa un segnale di clock globale. Tutte le transazioni avvengono a passi di tempo prestabiliti. È molto veloce, ma impone che tutti i dispositivi collegati operino alla stessa velocità o a multipli di essa. Non si adatta bene se si mescolano componenti veloci e lentissimi.
- **Asincrono:** Non c'è alcun clock globale. La comunicazione avviene tramite un protocollo di **Handshaking** (stretta di mano) usando speciali linee di controllo (*Request*, *Ready*, *Acknowledge*).
 - *Esempio di Lettura:* Il Master asserisce le linee di lettura e si dichiara "Pronto". Lo Slave intercetta il comando, decodifica l'indirizzo, prepara i dati sul bus e asserisce il segnale "Acknowledge" (Conferma). Il Master, leggendo la conferma, assorbe i dati e abbassa i segnali, chiudendo la transazione. Questa modalità si adatta dinamicamente alla latenza di ogni periferica.

10.4 Interfacciamento: Memory Mapped I/O

I dispositivi di I/O moderni per comunicare con il processore espongono una serie di registri interni:

- **Registri Dati:** Contengono il byte in ingresso o il dato da emettere in output.
- **Registri di Stato:** Indicano lo stato corrente della periferica (es. "dispositivo occupato", "dato pronto in ricezione", "errore").
- **Registri di Controllo:** Usati dalla CPU per inviare comandi specifici (es. "inizia rotazione").

Il processore accede a questi registri usando il principio del **Memory Mapped I/O** (I/O mappato in memoria). Ad ogni registro della periferica viene assegnato un indirizzo specifico (nello stesso spazio di indirizzamento della RAM). La CPU non ha bisogno di istruzioni Assembly speciali per dialogare con le periferiche: utilizza semplicemente le classiche **lw** (load word) e **sw** (store word) su quegli specifici indirizzi di memoria, ed è l'hardware del bus a intercettare l'indirizzo per recapitare il messaggio al controllore I/O invece che alla memoria RAM. Per sicurezza, questi indirizzi critici sono inaccessibili ai normali programmi utente e possono essere letti o scritti solo dal Sistema Operativo.

10.5 Gestione Asincrona e il Problema del Polling

Le periferiche sono ordini di grandezza più lente della CPU. Quando il processore invia un comando a un disco, non può semplicemente "aspettare" fermo il risultato nel ciclo successivo, perché perderebbe milioni di cicli operativi. L'I/O va quindi gestito in modo asincrono. Come fa la CPU a sapere quando la periferica ha completato il lavoro? Esistono varie strategie:

10.5.1 Il Polling (Busy-Waiting)

La tecnica più banale è il **Polling**. Il processore esegue un loop continuo nel quale legge il Registro di Stato della periferica, finché un certo bit (es. il bit *Ready*) non viene impostato a 1 dal dispositivo. Solo in quel momento il processore legge o scrive il Registro Dati.

- **Pro:** Estremamente semplice da implementare, basta un piccolo ciclo condizionale in Assembly.
- **Contro:** È altamente inefficiente e spreca enormi risorse di calcolo. Durante l'attesa la CPU sta materialmente eseguendo istruzioni a vuoto (*busy-waiting*), impedendo al computer di fare qualsiasi altra operazione utile.

Costo del Polling: Se usiamo il polling per leggere gli spostamenti di un Mouse (che richiede solo 30 letture al secondo), il carico computazionale su una CPU a 500 MHz sarà trascurabile (una frazione infinitesimale dell'1%). Ma se usassimo il polling per gestire un Hard Disk che invia interi blocchi di byte a 8 MB/s, la CPU impiegherebbe il 100% del suo tempo unicamente a fare loop di controllo, saturando l'intero sistema. Per gestire trasferimenti ingenti ed evitare questi sprechi, i processori ricorrono a metodologie ben più sofisticate come gli **Interrupt** e il **DMA (Direct Memory Access)**.

11 Gestione delle Interruzioni e delle Eccezioni

11.1 Limiti del Polling e Introduzione agli Interrupt

Come visto in precedenza, la tecnica del **polling** (attesa attiva) costringe il processore a sprecare un'enorme quantità di cicli macchina eseguendo letture a vuoto per verificare lo stato di una periferica. Questo approccio è accettabile solo per periferiche estremamente veloci o in sistemi dedicati in cui la CPU non ha altri compiti da svolgere. Nella maggior parte dei casi reali, questo spreco è inaccettabile. La soluzione è l'**I/O a interruzione di programma (Interrupt)**. In questo paradigma:

- La periferica lavora in totale autonomia.
- Quando la periferica è pronta (es. ha ricevuto un dato o ha completato una scrittura), invia un segnale hardware asincrono (*Interrupt*) alla CPU.
- Il processore rileva il segnale, interrompe temporaneamente il programma corrente, salva il proprio stato e salta a una specifica routine del Sistema Operativo chiamata **ISR (Interrupt Service Routine)**.
- Terminata la ISR, il processore ripristina lo stato e riprende l'esecuzione del programma interrotto esattamente dal punto in cui era stato sospeso.

L'overhead dell'interruzione (il tempo perso per salvare lo stato e fare il cambio di contesto) si paga *solo* quando la periferica genera effettivamente una richiesta, lasciando la CPU libera di eseguire codice utile per il resto del tempo.

11.2 Classificazione delle Eccezioni

In informatica, si definisce **eccezione** qualsiasi trasferimento non programmato del controllo del programma. A seconda della causa scatenante, si dividono in tre categorie:

1. **Interrupt (Interruzioni Hardware):** Causate da eventi esterni al processore (es. dispositivi di I/O, timer). Sono **asincrone** rispetto all'esecuzione del programma e vengono generalmente gestite tra un'istruzione e la successiva.
2. **Trap o Fault (Eccezioni Interne):** Causate da eventi anomali interni al programma, come overflow aritmetici, divisioni per zero, istruzioni illegali o page fault (accesso a memoria non valida). Sono **sincrone** all'esecuzione e costringono il processore a decidere se abortire il programma o risolvere il problema e riprovare l'istruzione.
3. **Environment Call / Break:** Eccezioni causate *volontariamente* dal programmatore. L'istruzione `ecall` richiede un servizio al Sistema Operativo (es. stampare a video), mentre `ebreak` genera un'eccezione utile per il debugging (es. i breakpoint in GDB).

11.3 Identificazione dell'Handler: Salto Diretto vs Vettorizzato

Quando si verifica un'eccezione, il processore deve sapere *dove* si trova in memoria il codice (Exception Handler) per gestirla. Esistono due approcci hardware:

- **Salto diretto all'indirizzo:** Il processore salta a un unico indirizzo base predefinito. L'handler unico dovrà poi interrogare speciali registri hardware per scoprire la causa dell'eccezione e agire di conseguenza. (Approccio base di RISC-V).
- **Vettore di interruzione (Vectored Interrupts):** Il processore calcola automaticamente l'indirizzo della routine specifica sommando un indirizzo base a un offset derivato dalla causa dell'eccezione ($\text{Indirizzo} = \text{Base} + \text{Causa} \times 4$). Questo metodo è più rapido perché salta direttamente al codice competente, ma richiede una tabella in memoria (Interrupt Vector Table) e un accesso aggiuntivo in memoria per leggere l'indirizzo.

11.4 Supporto Hardware in RISC-V (I Registri CSR)

Per gestire lo stato e i privilegi, RISC-V implementa un banco separato di registri interni chiamati **CSR (Control and Status Registers)**. Sono accessibili solo dal Sistema Operativo (Modalità Supervisor). I principali sono:

- **SEPC** (*Supervisor Exception Program Counter*): Salva l'indirizzo (il PC) dell'istruzione che ha generato l'eccezione o che è stata interrotta.
- **SCAUSE** (*Supervisor Cause*): Contiene un codice numerico che identifica la causa dell'eccezione. Il bit più significativo indica se si tratta di un Interrupt (1) o di una Trap (0). I restanti bit contengono il codice di errore (es. 2 per Istruzione Illegale, 9 per ecall).
- **SSTATUS** (*Supervisor Status*): Tiene traccia dei privilegi e delle abilitazioni globali. I bit fondamentali sono:
 - **SIE (Supervisor Interrupt Enable)**: Se a 1, gli interrupt sono abilitati.
 - **SPIE / SPP**: Salvano lo stato precedente del bit SIE e il livello di privilegio precedente (User o Supervisor) prima che scattasse l'eccezione.
- **STVEC** (*Supervisor Trap Vector*): Contiene l'indirizzo base della routine del Sistema Operativo a cui la CPU deve saltare in caso di eccezione.
- **STVAL** (*Supervisor Trap Value*): In caso di fault di memoria, salva l'indirizzo "sbagliato" che ha causato il crash.
- **SIE e SIP** (*Interrupt Enable / Pending*): Registri per un controllo fine. Permettono di abilitare o mascherare specifici tipi di interruzioni (Esterne, Timer, Software) e di verificare quali sono in attesa di essere servite.

11.4.1 Istruzioni per manipolare i CSR

Le normali istruzioni aritmetiche non possono leggere o scrivere i CSR. RISC-V fornisce istruzioni speciali "atomiche":

- **csrrw rd, csr, rs**: Scrive il valore di **rs** nel **csr**, e salva il vecchio valore del **csr** in **rd**.
- **csrrs rd, csr, rs**: Applica un OR bit-a-bit (Set) per accendere specifici bit del CSR.
- **csrrc rd, csr, rs**: Applica un AND negato (Clear) per spegnere specifici bit del CSR.

11.5 La Transizione di Privilegio (User → Supervisor)

Quando scatta un'eccezione, un complesso meccanismo hardware interviene automaticamente prima di eseguire la prima istruzione dell'Handler:

1. Il PC corrente viene salvato in **SEPC** (se il PC era già stato incrementato nella fase di fetch, l'hardware si assicura di salvare l'indirizzo dell'istruzione originale).
2. Il livello di privilegio corrente viene salvato in **SSTATUS.SPP** e si forza il passaggio alla **Modalità Supervisor**.
3. Il bit di abilitazione **SIE** viene salvato in **SPIE** e poi messo a 0, **disabilitando temporaneamente nuovi interrupt** per evitare loop e sovrascritture dei registri di stato.
4. Il PC viene sovrascritto con l'indirizzo dell'handler (**STVEC**).

Per ritornare al programma utente, l'handler chiama l'istruzione speciale **sret** (Supervisor Return), la quale esegue l'operazione inversa: ripristina il PC da **SEPC**, ripristina il privilegio da **SPP** e riabilita le interruzioni copiando **SPIE** in **SIE**.

11.6 Eccezioni nella Pipeline Hardware

A livello circuitale (Datapath), le eccezioni vengono trattate in modo molto simile agli *Hazard sul controllo* (come i salti mal predetti). Se un'istruzione in fase **EX** si rivela illegale o genera un fault aritmetico:

- È fondamentale impedire che questa istruzione completi il suo percorso e vada a sovrascrivere un registro nella fase **WB** (Write-Back) o scriva in memoria.
- Si utilizzano dei segnali di **Flush** (**ID.Flush**, **EX.Flush**) per svuotare i registri di pipeline, trasformando di fatto l'istruzione in una bolla (*nop*).
- Contemporaneamente, un multiplexer forza l'indirizzo dell'istruzione successiva al valore dell'handler (**STVEC**), dirottando il flusso di esecuzione verso il Sistema Operativo prima che avvengano danni allo stato architetturale.

12 Esercizi di Riepilogo

12.1 Esercizio 1: Reti Logiche e Algebra di Boole

Individuare tra le opzioni seguenti l'espressione logica equivalente a:

$$x \cdot (y + z) + \overline{x + \overline{y}}$$

Le opzioni possibili sono:

- $z \cdot \overline{z} + \overline{x}$
- $x \cdot z + y$
- x
- $x \cdot y + \overline{x} \cdot z$

Soluzione: Semplifichiamo l'espressione di partenza applicando le regole dell'algebra di Boole. Riscriviamo l'espressione usando le leggi di De Morgan ($\overline{A + B} = \overline{A} \cdot \overline{B}$ e $\overline{A \cdot B} = \overline{A} + \overline{B}$), la proprietà distributiva ($A \cdot (B + C) = A \cdot B + A \cdot C$) e la regola della doppia negazione ($\overline{\overline{A}} = A$):

$$x \cdot (y + z) + \overline{x + \overline{y}} = x \cdot y + x \cdot z + \overline{x} \cdot \overline{\overline{y}} = x \cdot y + x \cdot z + \overline{x} \cdot y$$

Raccogliamo la variabile y dal primo e dal terzo termine:

$$y \cdot (x + \overline{x}) + x \cdot z$$

Sapendo per la regola dell'inversa che $x + \overline{x} = 1$, l'espressione si riduce a:

$$y \cdot 1 + x \cdot z = y + x \cdot z$$

Riorganizzando i termini per la proprietà commutativa otteniamo $x \cdot z + y$. Pertanto, la seconda opzione è quella corretta.

12.2 Esercizio 2: Memoria Cache a Mappatura Diretta

Data una cache a mappatura diretta con capacità totale di 4096 byte e blocchi della dimensione di 16 byte, determinare in quale blocco della cache viene mappato l'indirizzo di memoria $0x1F164_{16}$.

Opzioni:

- Nel blocco numero 6
- Nel blocco numero 64
- Nel blocco numero 22
- Nel primo blocco libero

Soluzione: Calcoliamo innanzitutto la suddivisione dei bit dell'indirizzo:

- **Offset del byte:** Dato che ci sono 16 byte per blocco, i bit dedicati all'offset sono $\log_2(16) = 4$ bit.
- **Indice del blocco:** Calcoliamo il numero totale di blocchi presenti nella cache:

$$\text{Numero di blocchi} = \frac{\text{Dimensione Cache}}{\text{Dimensione Blocco}} = \frac{4096}{16} = 256$$

I bit necessari per codificare l'indice del blocco all'interno della cache sono quindi $\log_2(256) = 8$ bit.

Convertiamo l'indirizzo esadecimale $0x1F164_{16}$ nel corrispondente valore binario:

$$0001\ 1111\ 0001\ 0110\ 0100_2$$

Suddividiamo la stringa binaria partendo da destra (bit meno significativi) in base alle dimensioni calcolate:

- **Offset (ultimi 4 bit):** 0100_2
- **Block/Indice (successivi 8 bit):** 00010110_2
- **Tag (bit rimanenti):** 00011111_2

L'indice che determina la posizione in cache è 00010110_2 . Convertendolo in decimale otteniamo:

$$2^4 + 2^2 + 2^1 = 16 + 4 + 2 = 22$$

L'indirizzo è quindi mappato nel **blocco numero 22**. La risposta corretta è la terza.

12.3 Esercizio 3: Calcolo del CPI Reale (Prestazioni della CPU)

Si consideri una CPU dotata di due cache separate per dati ed istruzioni. Il CPI (Clock Cycles per Instruction) ideale della CPU è 4.

- La cache istruzioni ha una frequenza di miss dell'1%.
- La cache dati ha una frequenza di miss del 4%.
- Una cache miss richiede una penalità di 100 cicli di clock per essere servita dalla memoria principale.
- Il 20% delle istruzioni assembly accede a dati in memoria (istruzioni di load/store).

Calcolare il CPI reale della macchina.

Soluzione: Il CPI reale si calcola sommando al CPI ideale i cicli di stallo medi generati dalle latenze delle memorie cache (miss penalty).

$$\text{CPI Reale} = \text{CPI Ideale} + \text{Stalli per Istruzione} + \text{Stalli per Dati}$$

- **Stalli dovuti alla Cache Istruzioni:** Poiché ogni singola istruzione deve essere obbligatoriamente prelevata dalla memoria (fase di fetch), il 100% delle istruzioni accede a questa cache.

$$\text{Stalli Istruzioni} = 1 \cdot (\text{Miss Rate Istruzioni}) \cdot (\text{Miss Penalty}) = 1 \cdot 0.01 \cdot 100 = 1 \text{ ciclo/istruzione}$$

- **Stalli dovuti alla Cache Dati:** Solo il 20% delle istruzioni totali necessita di accedere alla memoria dati.

$$\text{Stalli Dati} = 0.20 \cdot (\text{Miss Rate Dati}) \cdot (\text{Miss Penalty}) = 0.20 \cdot 0.04 \cdot 100 = 0.8 \text{ cicli/istruzione}$$

Sommando i valori calcolati:

$$\text{CPI Reale} = 4 + 1 + 0.8 = 5.8$$

Il CPI effettivo della CPU sarà pari a **5.8 cicli per istruzione**.

13 Tutorato: Esercizi di Traduzione C-Assembly (RISC-V, Intel x86-64, ARM)

Questa sezione analizza in dettaglio gli esercizi proposti nel tutorato, mettendo a confronto diretto le tre architetture studiate (RISC-V, Intel x86-64 e ARM). L'obiettivo è comprendere come lo stesso codice C ad alto livello venga tradotto dai compilatori (ottimizzazione -O1) adattandosi alle peculiarità e ai vincoli hardware di ciascuna architettura.

13.1 Esercizio 1: Espressioni Aritmetiche Semplici

Consideriamo la seguente funzione in C, che esegue una semplice espressione aritmetica:

```
int es_1(int b, int c, int d) {  
    return (b + c) - d;  
}
```

13.1.1 Traduzione RISC-V

I parametri di ingresso `b`, `c`, `d` vengono passati nei registri `a0`, `a1`, `a2`. Il valore di ritorno deve essere posizionato in `a0`.

```
add a0, a0, a1    // a0 = b + c
sub a0, a0, a2     // a0 = a0 - d
ret               // Ritorna al chiamante
```

Spiegazione: RISC-V è un'architettura a 3 operandi. Il compilatore riutilizza intelligentemente il registro `a0` (che conteneva originariamente `b`) per accumulare prima la somma parziale e poi il risultato finale della sottrazione, trovandosi già pronto per il ritorno (`ret`).

13.1.2 Traduzione Intel x86-64

I parametri `b`, `c`, `d` (essendo interi a 32 bit) arrivano in `%edi`, `%esi`, `%edx`. Il ritorno va in `%eax`.

```
leal (%rdi, %rsi), %eax    // %eax = b + c
subl %edx, %eax            // %eax = %eax - d
ret
```

Spiegazione: Poiché Intel usa istruzioni distruttive a 2 operandi (la destinazione sovrascrive un sorgente), il compilatore sfrutta l'istruzione `lea` (Load Effective Address) in modo ingegnoso. Invece di usare una `addl` che sporcherebbe uno dei parametri, usa la circuiteria per il calcolo degli indirizzi di memoria per eseguire la somma `rdi + rsi` salvandola in un registro illibato (`%eax`). Dopodiché sottrae direttamente `%edx`.

13.1.3 Traduzione ARM

I parametri arrivano in `r0`, `r1`, `r2`. Il ritorno va in `r0`.

```
add r0, r0, r1      // r0 = b + c
subs r0, r0, r2     // r0 = r0 - d (e aggiorna i flag)
bx lr               // Ritorna al chiamante
```

Spiegazione: Simile al RISC-V, ARM supporta 3 operandi (in questo caso la destinazione collassa sul primo sorgente). Il compilatore usa `subs` (con la 's' finale) che aggiorna il registro di stato della ALU, un comportamento tipico in ARM anche quando non strettamente necessario per l'istruzione successiva. Il ritorno avviene tramite `bx lr` (Branch and eXchange al Link Register).

13.2 Esercizi 2 e 3: Memoria e Moltiplicazioni Ottimizzate

Analizziamo l'accesso a un array: `A[4] = h + A[2];`. Essendo un array di `int` a 32 bit (4 byte), l'indice 2 corrisponde all'offset logico 8 byte, e l'indice 4 all'offset 16 byte.

- **RISC-V:** Come architettura *Load/Store* pura, richiede esplicitamente di portare i dati nei registri prima di processarli.

```
lw    a5, 8(a0)      // Carica A[2] nel registro a5
add   a5, a5, a1     // a5 = A[2] + h
sw    a5, 16(a0)     // Salva a5 in A[4]
```

- **Intel x86-64:** Essendo CISC, può sommare *direttamente* un valore in memoria a un registro, unendo concettualmente load e add nella prima riga:

```
addl 8(%rdi), %esi   // %esi (h) = h + memoria(A[2])
movl %esi, 16(%rdi) // Salva il nuovo %esi in A[4]
```

Moltiplicazioni per potenze di 2: Nel calcolo $b * 8$, nessuno dei compilatori usa una vera istruzione di moltiplicazione (troppo lenta in cicli di clock).

- **RISC-V / ARM:** Eseguono uno shift logico a sinistra di 3 posizioni ($2^3 = 8$), rispettivamente tramite `slli a0, a0, 3` e `lsls r0, r0, #3`.
- **Intel x86-64:** Usa l'indirizzamento scalato di `leal`: `leal 0(,%rdi,8), %eax` sfrutta il moltiplicatore hardware degli offset per calcolare $b \times 8$.

13.3 Esercizi 4, 5, 6 e 7: Ottimizzazione dei Salti (Branchless)

Una delle maggiori inefficienze dei processori moderni è lo svuotamento della pipeline causato dai salti condizionati (*branch hazard*). I compilatori moderni cercano di implementare costrutti come gli `if-else` senza usare istruzioni di salto.

13.3.1 La simulazione dell'If-Else: Intel vs ARM

Dato il codice: `if (i < j) f = g + h; else f = g - h;`

- **Intel x86-64 (Conditional Move):**

```
leal (%rdx,%rcx), %eax // Calcola a prescindere f = g + h (in %eax)
subl %ecx, %edx        // Calcola a prescindere f = g - h (in %edx)
cmpl %esi, %edi        // Confronta i e j
cmovge %edx, %eax      // SE i >= j, sovrascrivi %eax copiandoci %edx
```

Spiegazione: Il processore esegue il calcolo di **entrambi** i rami (*if* ed *else*) speculativamente. Poi confronta le variabili e usa `cmovge` (Conditional Move if Greater or Equal). Se la condizione dell'*else* si avvera, sposta il risultato corretto nel registro di destinazione. Zero salti eseguiti.

- **ARM (Esecuzione Predicata / ITE):**

```
cmp r0, r1           // Confronta i e j
ite lt              // Inizia blocco: If (Less Than), Then, Else
addlt r0, r2, r3     // Eseguita SOLO se i < j (g + h)
subge r0, r2, r3     // Eseguita SOLO se i >= j (g - h)
```

Spiegazione: ARM possiede blocchi ITE (If-Then-Else). Le istruzioni aritmetiche successive presentano un suffisso condizionale (`lt` o `ge`). L'hardware salta a piè pari (come se fossero *nop*) le istruzioni la cui condizione non è verificata dai flag correnti. Nessun branch necessario.

13.4 Esercizio 8: L'Estrema Ottimizzazione Bit-a-Bit in ARM

Un caso di studio affascinante è il clamp a zero: `a = b + c; if (a < 0) a = 0;`. Il compilatore ARM esegue una vera "magia" bit-a-bit per evitare completamente i confronti:

```
add r0, r0, r1          // r0 = b + c
bic r0, r0, r0, asr #31  // Bit Clear usando Arithmetic Shift Right
```

Spiegazione del trucco: L'istruzione `bic` (Bit Clear) azzerava i bit del primo operando (`r0`) dove il secondo operando (la maschera) presenta degli 1. La maschera è calcolata prendendo `r0` e facendogli uno shift aritmetico a destra di 31 bit (`asr #31`).

- Se `a` è **positivo**, il suo bit di segno è 0. Lo shift di 31 propaga lo zero, creando una maschera `0x00000000`. Il *Bit Clear* contro zeri non fa nulla, lasciando `a` inalterato.
- Se `a` è **negativo**, il bit di segno è 1. Lo shift aritmetico propaga l'1, creando la maschera `0xFFFFFFFF`. Il *Bit Clear* contro una sequenza di uno azzerava completamente il registro, forzando `a = 0`.

13.5 Esercizio 9: Cicli While e Indirizzamento Avanzato

Consideriamo il loop: `while (salva[i] == k) { i += 1; }`

- **Intel x86-64:** Per gestire gli indici in un'architettura a 64 bit, il contatore `i` (a 32 bit in C) viene esteso con segno tramite `movslq %esi, %rax`. Intel gestisce poi l'avanzamento senza dover aggiornare manualmente gli indirizzi fisici: usa il calcolo AGU interno `cmpl %edx, (%rdi, %rcx, 4)`, che in hardware confronta `k` (`%edx`) col dato situato a base (`%rdi`) + indice (`%rcx`) $\times 4$.
- **ARM (Indirizzamento Pre/Post-Indexed):** ARM ottimizza i cicli evitando di tenere traccia separata di "indirizzo base" e "contatore byte".

.L3:

```
adds r0, r0, #1          // i += 1
ldr r1, [r3, #4]!        // Pre-Indexed con Write-Back (Il punto esclamativo)
cmp r1, r2                // salva[i] == k ?
beq .L3                  // Se uguali, continua il loop
```

Spiegazione: L'istruzione chiave è `ldr r1, [r3, #4]!`. Carica nel registro `r1` l'intero che si trova 4 byte più avanti rispetto a `r3`, e il suffisso `!` ordina all'hardware di **aggiornare automaticamente** il puntatore `r3` sommandogli 4 (write-back). In una sola passata si carica il dato e si fa progredire il puntatore per il ciclo successivo.

14 Soluzioni Mockup Complete

1. **Domanda:** Si consideri una CPU in cui le 5 fasi di esecuzione di un'istruzione impiegano rispettivamente 200ps, 300ps, 400ps, 200ps e 100ps. Il massimo incremento di prestazioni che ci si può attendere usando una pipeline è:

- (A) di 2.5 volte
- (B) di 4 volte
- (C) di 3 volte
- (D) di 2 volte
- (E) nessuna delle altre risposte

Risposta corretta: (C)

Perché è corretta: Il tempo di esecuzione di una singola istruzione senza pipeline è la somma dei tempi delle 5 fasi: $200 + 300 + 400 + 200 + 100 = 1200$ ps. In un'architettura a pipeline, il tempo del ciclo di clock è rigidamente determinato dalla fase più lenta, che in questo caso è di 400 ps (costituisce il collo di bottiglia). L'incremento massimo teorico di prestazioni (Speedup) si calcola come il rapporto tra il tempo di esecuzione senza pipeline e il tempo del clock con pipeline: $1200/400 = 3$. Lo speedup atteso è quindi di 3 volte.

Perché scartare le altre: Le opzioni (A) e (D) derivano da calcoli matematici errati del rapporto. La (B) confonde il numero di fasi (pensando magari che la più lenta non incida in questo modo) o assume erroneamente un tempo di clock inferiore per il calcolo.

2. **Domanda:** Si consideri una cache set associative a 4 vie grande 8KB, con blocchi di 16 byte. In quali blocchi di cache può essere mappata la parola all'indirizzo `0x10A24`?

- (A) Nei blocchi dal numero 136 al numero 139
- (B) Nel blocco 34
- (C) Nei blocchi dal numero 68 al numero 71
- (D) Nel primo blocco libero
- (E) Nessuna delle altre risposte

Risposta corretta: (A)

Perché è corretta: Calcoliamo prima i parametri della cache. Dimensione totale = 8 KB = 8192 byte. Il numero totale di blocchi fisici in cache è $8192/16 = 512$ blocchi. Essendo una set-associativa a 4 vie, il numero di Set (insiemi) è $512/4 = 128$ Set. Convertiamo l'indirizzo da esadecimale a decimale: $0x10A24 = 68132_{10}$. L'indirizzo del blocco in memoria si ottiene dividendo l'indirizzo della parola per la dimensione del blocco (16 byte): $\lfloor 68132/16 \rfloor = 4258$. Troviamo l'indice del Set calcolando il modulo con il numero totale di Set: $4258 \bmod 128 = 34$. Il dato verrà quindi mappato nel Set 34. Poiché ogni Set raggruppa 4 blocchi consecutivi, il Set 34 include fisicamente i blocchi che vanno dal numero $34 \times 4 = 136$ fino al blocco $136 + 3 = 139$.

Perché scartare le altre: La (B) indica il numero del Set logico (34), non il numero effettivo dei blocchi fisici della cache. La (C) usa un moltiplicatore errato per le vie (moltiplica per 2 invece che per 4). La (D) descrive il comportamento di una cache *fully-associative*, ma questa specifica cache è set-associativa.

3. **Domanda:** Una CPU genera indirizzi di memoria a 32 bit. Si consideri una cache a mappatura diretta (direct mapped) con capacità totale di 64 KB e linee di cache (blocchi) da 32 byte. Quanti bit dell'indirizzo sono dedicati rispettivamente a Tag, Index e Offset?

- (A) Tag: 17, Index: 10, Offset: 5
- (B) Tag: 16, Index: 11, Offset: 5
- (C) Tag: 15, Index: 12, Offset: 5
- (D) Tag: 16, Index: 12, Offset: 4
- (E) Nessuna delle altre risposte

Risposta corretta: (B)

Perché è corretta: Scomponiamo i 32 bit dell'indirizzo calcolando le tre porzioni necessarie. 1. **Offset:** Ogni blocco è di 32 byte (2^5), quindi i bit necessari per identificare il byte nel blocco sono $\log_2(32) = 5$ bit. 2. **Index:** La cache direct mapped ha un numero di blocchi pari alla capacità diviso la dimensione del blocco: $64 \text{ KB} / 32 \text{ Byte} = 65536 / 32 = 2048$ blocchi totali (2^{11}). Servono quindi $\log_2(2048) = 11$ bit per l'indice (Index). 3. **Tag:** I bit rimanenti dei 32 totali generati dalla CPU sono assegnati al Tag: $32 - 11 - 5 = 16$ bit. La suddivisione corretta è perciò 16 (Tag), 11 (Index), 5 (Offset).

Perché scartare le altre: La (A) presuppone un Index da 10 bit, sottostimando la capienza della cache o sbagliando la formula. La (C) calcola un Index a 12 bit, errato per la dimensione di 64KB. La (D) attribuisce 4 bit di Offset, il che significherebbe avere blocchi da 16 byte invece di 32.

4. **Domanda:** Si consideri una cache direct mapped grande 8KB con blocchi di 32 byte. In che blocco è mappata la parola all'indirizzo 0x3A54?

- (A) Nel blocco numero 210
- (B) Nel blocco numero 84
- (C) Nel blocco numero 54
- (D) Nel blocco numero 120
- (E) Nessuna delle altre risposte

Risposta corretta: (A)

Perché è corretta: Nelle cache direct mapped la mappatura segue la formula $\text{Indirizzo_blocco} \bmod \text{Numero_blocchi_cache}$. Troviamo per prima cosa il numero totale di blocchi della cache: $8192 \text{ byte} / 32 \text{ byte} = 256$ blocchi totali. Convertiamo l'indirizzo in decimale: $0x3A54 = 14932_{10}$. Troviamo a che blocco appartiene l'indirizzo in memoria dividendo per la grandezza del blocco e ignorando il resto (l'offset): $\lfloor 14932 / 32 \rfloor = 466$. Infine, calcoliamo la posizione in cache con il modulo: $466 \bmod 256 = 210$. Il blocco sarà mappato alla riga 210.

Perché scartare le altre: Le opzioni (B), (C) e (D) derivano molto probabilmente dall'aver saltato la divisione iniziale per 32 (l'offset), calcolando direttamente il modulo sull'indirizzo anziché sull'indice di blocco di memoria, o da banali errori di conversione da esadecimale a decimale.

5. **Domanda:** Si consideri una CPU con CPI ideale pari a 2. La cache istruzioni ha un miss rate del 2%, e la cache dati del 5%. Una miss penalizza l'esecuzione di 50 cicli di clock. Sapendo che il 30% delle istruzioni eseguite accede in memoria dati, qual è il CPI reale?
- (A) 4.5
 - (B) 3.75
 - (C) 2.35
 - (D) 5.5
 - (E) Nessuna delle altre risposte

Risposta corretta: (B)

Perché è corretta: Il CPI reale si ottiene sommando al CPI ideale (2) i cicli medi di stallo persi (miss penalty) per ogni singola istruzione, separando istruzioni e dati.

1. **Miss su Istruzioni:** *Tutte* le istruzioni (100%) devono essere lette dalla memoria (fase di fetch). I cicli persi sono: $1 \times 0.02 \times 50 = 1.0$ ciclo/istruzione.

2. **Miss su Dati:** Solo il 30% delle istruzioni fa load/store accedendo alla cache dati. I cicli persi qui sono: $0.30 \times 0.05 \times 50 = 0.75$ cicli/istruzione. La somma finale sarà quindi: $\text{CPI Reale} = \text{CPI Ideale} + \text{Stalli Istruzioni} + \text{Stalli Dati} = 2 + 1.0 + 0.75 = 3.75$.

Perché scartare le altre: La (A) fa l'errore di non pesare la miss rate dei dati per il 30% di frequenza (calcolando $2 + 1 + (1 \times 0.05 \times 50) = 5.5$). La (C) salta completamente la penalità per la cache istruzioni oppure sbaglia le moltiplicazioni. La (D) somma il valore errato di (A).

6. **Domanda:** La funzione `sumV` prende in ingresso l'indirizzo di tre array di interi `u`, `v`, `z`, e la dimensione `size` degli array. Il suo compito è quello di effettuare operazioni sugli elementi di `u` e `v` e di salvare il risultato in `z`. Il compilatore fornisce la seguente traduzione in assembly ARM che è incompleta: le righe **X1** e **X2** sono omesse. (Sono rispettate le convenzioni di chiamata per gli argomenti e per il valore di ritorno specificate dall'ABI). Quale delle coppie **X1** e **X2** proposte corrisponde alle righe corrette?

```
void sumV(int * u, int * v, int * z, unsigned int size) {
    for (int i=0; i < size - 1; i++) {
        *(z+i) = *(v+i) + *(u+i);
    }
    return;
}
```

```
sumV:
    X1
    beq     L9
    sub     r12, r1, #4
    subs    r1, r1, #8
    stmfd   sp!, {r11, lr}
    subs    r0, r0, #4
    add     lr, r1, r3, lsl #2
    subs    r2, r2, #4
L3:
    ldr     r3, [r12, #4]!
    ldr     r1, [r0, #4]!
    cmp     r12, lr
    X2
    str     r3, [r2, #4]!
    bne     L3
    ldmfd   sp!, {r11, pc}
L9:
    bx      lr
```

- (A) X1: `cmp r3, #1` | X2: `add r3, r3, r1`
(B) X1: `cmn r3, #1` | X2: `add r3, r3, r1`
(C) X1: `cmp r3, #1` | X2: `add r3, r3, r0`
(D) X1: `cmn r3, #1` | X2: `add r3, r3, r0`
(E) Nessuna delle altre risposte

Risposta corretta: (A)

Perché è corretta: Dalla traduzione incompleta in assembly notiamo che la seconda istruzione è un `beq L9`, che salta alla fine della funzione (uscita). Il codice C prevede un ciclo dipendente da `size - 1` e il controllo avviene in testa; quindi, se la dimensione (`size`) è 1, il ciclo non deve essere eseguito. L'istruzione per valutare questa condizione (`size == 1`) e impostare i flag per il `beq` deve essere un confronto testuale sulla variabile `size` (contenuta in `r3` per l'ABI). Perciò X1 deve essere `cmp r3, #1`. Per quanto riguarda X2, il ciclo carica in `r3` e `r1` gli elementi degli array, quindi la somma (+) deve avvenire tra i registri `r3` ed `r1`. L'istruzione corretta per X2 è `add r3, r3, r1`.

Perché scartare le altre: Le opzioni (B) e (D) usano `cmn` (Compare Negative), che esegue una somma invece di una sottrazione per aggiornare i flag, rendendo errato il check logico. L'opzione (C) esegue la somma usando `r0` come registro sorgente (`add r3, r3, r0`), ma `r0` contiene l'indirizzo di memoria del primo array e non l'elemento appena estratto, rendendo la somma non valida.

7. **Domanda:** Qual è il risultato della seguente compilazione usando il compilatore gcc: `gcc hello_world.c -o a.out -O2`?

- (A) Genera il file eseguibile `a.out`
- (B) Genera il file assembly `hello_world.s`
- (C) Genera il file oggetto `hello_world.o`
- (D) Genera il file preprocessato `hello_world.i`
- (E) Nessuna delle altre risposte.

Risposta corretta: (A)

Perché è corretta: Il comando esegue l'intera toolchain di GCC sul file sorgente. L'opzione `-o a.out` permette di specificare il nome del file di output generato. Poiché non sono stati inseriti specifici flag per interrompere il processo prima del linking (come `-c` o `-S`), GCC arriva al termine della procedura generando direttamente il programma eseguibile nominato `a.out`[cite: 338, 339, 340].

Perché scartare le altre: Per generare solo il file assembly (B) bisognerebbe usare il flag `-S`. Per generare il solo file oggetto prima del linking (C) occorrerebbe il flag `-c`. Per generare il file espanso e preprocessato (D) serve il flag `-E`.

8. **Domanda:** Quale delle seguenti affermazioni è FALSA:

- (A) Nei circuiti combinatori, l'uscita dipende dallo stato
- (B) Nei circuiti sequenziali, l'uscita dipende dallo stato
- (C) Nei circuiti combinatori, l'uscita dipende dall'ingresso
- (D) Nei circuiti sequenziali, l'uscita dipende dall'ingresso
- (E) Tutte le risposte

Risposta corretta: (A)

Perché è corretta: La domanda chiede di individuare l'affermazione palesemente scorretta. Per definizione hardware, una rete logica combinatoria è "senza memoria". Questo significa che i valori delle sue uscite dipendono *esclusivamente* e puramente dalla combinazione attuale degli ingressi in quello stesso istante[cite: 351, 352]. Non avendo memoria interna, il circuito combinatorio non possiede un concetto di "stato", pertanto affermare che la sua uscita dipenda dallo stato è radicalmente falso.

Perché scartare le altre: Le affermazioni in (B), (C) e (D) descrivono invece concetti validi e veri. I circuiti sequenziali calcolano la loro uscita elaborando sia gli ingressi attuali sia la loro memoria (il loro stato), mentre i combinatori si basano rigorosamente solo sull'ingresso.

9. **Domanda:** Quale delle seguenti alternative rappresenta un esempio di hazard sui dati (Data Hazard)?

- (A) and t0, t1, t3 seguito da or t2, t4, t5
- (B) or t0, t1, t2 seguito da add t2, t4, t5
- (C) add t0, t1, t2 seguito da and t3, t1, t5
- (D) sub t0, t1, t2 seguito da add t3, t4, t0
- (E) Nessuna delle altre risposte.

Risposta corretta: (D)

Perché è corretta: Un hazard (conflitto) sui dati si verifica quando c'è una dipendenza diretta tra istruzioni che si sovrappongono nella pipeline. Nell'opzione (D), la prima istruzione (sottrazione) calcola un valore che viene salvato nel registro t0 come *destinazione*. La seconda istruzione tenta immediatamente di leggere il registro t0 come *sorgente* per l'addizione. Siccome la prima istruzione non ha ancora fatto in tempo a scrivere fisicamente il nuovo valore nel banco dei registri, si crea un conflitto del tipo RAW (Read-After-Write)[cite: 385, 386].

Perché scartare le altre: Nella (A) le due istruzioni non condividono alcun registro. Nella (B) t2 è prima sorgente e solo dopo diventa destinazione, quindi non genera ritardi RAW. Nella (C) il registro t1 è letto contemporaneamente da entrambe come sorgente: più letture dallo stesso registro non causano conflitti nell'hardware.

10. **Domanda:** Quale delle seguenti risposte rappresenta una gerarchia di memoria ordinata per distanza (crescente) dal processore?

- (A) Registri - Cache - RAM - Hard Disk
- (B) Cache - Registri - RAM - Hard Disk
- (C) Registri - RAM - Cache - Hard Disk
- (D) Registri - Cache - Hard Disk - RAM
- (E) Nessuna delle risposte

Risposta corretta: (A)

Perché è corretta: Le gerarchie di memoria sono strutturate su distanze crescenti dalla ALU che si traducono anche in tempi di accesso e costi differenti. Il livello zero "interno" sono i Registri (praticamente istantanei). Subito a ridosso del chip troviamo la memoria Cache (estremamente veloce). Uscendo sul bus si passa alla memoria di massa volatile RAM. All'estremo opposto (il più distante, lento ed economico) risiede l'Hard Disk (memoria permanente).

Perché scartare le altre: La (B) inverte Cache e Registri. La (C) inverte i ruoli di RAM e Cache (la Cache è tra CPU e RAM). La (D) posiziona insensatamente l'Hard Disk come un componente intermedio più vicino/veloce della memoria RAM.

11. **Domanda:** Quale delle seguenti affermazioni riguardanti le traps e le interruzioni è vera?

- (A) Sono entrambi causati da eventi esterni e sono asincroni
- (B) Sono entrambi causati da eventi esterni e sono sincroni all'esecuzione del programma
- (C) Sono entrambi causati da eventi interni e sono asincroni
- (D) Sono entrambi causati da eventi interni e sono sincroni all'esecuzione del programma
- (E) Nessuna delle altre risposte

Risposta corretta: (E)

Perché è corretta: Nessuna delle definizioni fornite dalla (A) alla (D) è valida per accomunare questi due costrutti. Le interruzioni (Interrupts) sono generate da dispositivi hardware esterni (come l'I/O) e sono, per loro natura, *asincrone* rispetto all'esecuzione ordinaria del programma[cite: 408, 409]. Al contrario, le Traps (eccezioni) sono generate direttamente da anomalie della CPU *dall'interno* e sono squisitamente *sincrone* in quanto scatenate da una singola precisa istruzione[cite: 409, 410]. Poiché divergono sia per causa (interna/esterna) sia per tempismo (sincrona/asincrona), è impossibile accorparle e la risposta esatta è la (E).

Perché scartare le altre: Le opzioni A, B, C e D tentano erroneamente di forzare sia i trap sia gli interrupt nella stessa identica scatola, asserendo in modo fallace che abbiano la stessa identica origine causale (entrambi interni / entrambi esterni).

12. **Domanda:** Le seguenti affermazioni descrivono alcuni dei pregi introdotti dalla pipeline nei microprocessori. Individua quale di queste NON è corretta. Nelle architetture pipelined...

- (A) Maggiore è il numero di stadi, potenzialmente maggiori sono le prestazioni in confronto ad un'architettura senza pipeline.
- (B) La frequenza di clock è determinata dall'istruzione più lenta.
- (C) L'ordine con cui sono scritte le istruzioni potrebbe influire sul tempo di esecuzione.
- (D) Non si verificano mai eventi di hazard.
- (E) Nessuna delle altre risposte.

Risposta corretta: (D)

Perché è corretta: L'affermazione è falsa. Nelle architetture a pipeline si verificano sistematicamente gli *hazard* (conflitti sui dati, di controllo e strutturali), che rappresentano proprio il problema principale di questa tecnologia (da risolvere con forwarding o stalli).

Perché scartare le altre: Le opzioni (A), (B) e (C) sono tutte affermazioni *vere*. Lo speedup dipende dal numero di stadi, il clock è vincolato dalla fase più lenta (collo di bottiglia) e l'ordine delle istruzioni influisce enormemente (Code Scheduling per riempire i load-use hazard).

13. **Domanda:** Si consideri una CPU che impiega 600ps per la fase di fetch, 600ps per la fase di decodifica, 500ps per eseguire operazioni con la ALU, 400ps per la fase di accesso alla memoria e 700ps per la fase di scrittura nel register file. Il massimo incremento di prestazioni che ci si può attendere usando una pipeline è:

- (A) Di 2.5 volte
- (B) Di 3 volte
- (C) Di 4 volte
- (D) Di 2 volte
- (E) Nessuna delle altre risposte

Risposta corretta: (C)

Perché è corretta: Calcoliamo il tempo di esecuzione sequenziale di una singola istruzione sommando i tempi delle fasi: $600 + 600 + 500 + 400 + 700 = 2800$ ps. In un'architettura a pipeline, il clock globale deve adattarsi allo stadio più lento per permettere a tutti di finire, che in questo caso è 700 ps. Lo speedup (incremento massimo) si ottiene facendo il rapporto: $2800/700 = 4$. L'incremento atteso è di 4 volte.

Perché scartare le altre: (A), (B) e (D) derivano da un'errata individuazione della fase critica (o dall'utilizzo di una media matematica invece del picco peggiore).

14. **Domanda:** Con riferimento allo schema generale del Datapath, si indichi quale delle seguenti alternative spiega meglio le modalità di esecuzione di un'istruzione logico-aritmetica di tipo R (es. add, sub).

- (A) RegDst = 1, ALUSrc = 0, RegWrite = 1, MemToReg = 0, AluOp = 10
- (B) RegDst = 0, ALUSrc = 0, RegWrite = 1, MemToReg = 0, AluOp = 01
- (C) RegDst = 1, ALUSrc = 1, RegWrite = 1, MemToReg = 0, AluOp = 11
- (D) Nessuna delle scelte proposte

Risposta corretta: (A)

Perché è corretta: In una tipica istruzione R, la destinazione è il registro estratto dal campo specifico dell'istruzione (**RegDst** = 1). Il secondo operando della ALU proviene dal banco dei registri e non dall'immediato (**ALUSrc** = 0). Bisogna ovviamente scrivere il risultato finale (**RegWrite** = 1). Il dato da salvare non proviene dalla memoria, bensì direttamente dalla ALU (**MemToReg** = 0). L'**AluOp** classico assegnato a queste istruzioni dall'Unità di Controllo per abilitare la decodifica fine è 10.

Perché scartare le altre: La (B) usa **AluOp** = 01, che è riservato ai Branch condizionati. La (C) setta erroneamente **ALUSrc** = 1, che farebbe entrare nella ALU il valore immediato esteso in segno (tipico delle istruzioni di Load/Store o I-Type).

15. **Domanda:** Supponiamo che i vari blocchi logici del datapath RISC-V abbiano le seguenti latenze: Mem-I/Mem-D 250ps, Register File 150ps, Mux 25ps, ALU 200ps, Addizionatore 150ps, Lettura PC 40ps, Impostazione/Setup Registro 14ps. Qual è la latenza critica (durata del clock) di un'istruzione di tipo R? (Attenzione ai MUX).

- (A) $40 + 150 + 25 + 200 + 25 + 14 = 454$ ps
(B) $40 \cdot 3 + 250 + 150 + 25 + 200 + 25 + 14 \cdot 3 = 812$ ps
(C) $40 + 250 + 150 + 25 + 200 + 25 + 14 = 704$ ps
(D) Nessuna delle altre risposte

Risposta corretta: (C)

Perché è corretta: Il cammino critico (quello sequenzialmente più lungo in hardware) per un'istruzione di Tipo R su CPU a singolo ciclo è: 1) Lettura del PC (40ps); 2) Estrazione da Memoria Istruzioni (250ps); 3) Lettura dei registri sorgente dal Register File (150ps); 4) Passaggio nel Mux **ALUSrc** (25ps); 5) Calcolo in ALU (200ps); 6) Passaggio nel Mux **MemToReg** saltando la memoria dati (25ps); 7) Tempo di setup per scrivere il risultato nel Register File (14ps). La somma esatta è 704 ps.

Perché scartare le altre: La (A) "dimentica" il tempo biblico di accesso alla Memoria Istruzioni (250ps). La (B) aggiunge assurdamente letture multiple del PC e setup ripetuti ad ogni stadio.

16. **Domanda:** Nel datapath, per le istruzioni di trasferimento dati dalla memoria (**load word** e **store word**):

- (A) La ALU non calcola l'indirizzo di memoria.
(B) La ALU esegue un'operazione AND per calcolare l'indirizzo di memoria.
(C) La ALU deve eseguire una somma per calcolare l'indirizzo di memoria.
(D) La ALU deve eseguire una sottrazione per calcolare l'indirizzo di memoria.

Risposta corretta: (C)

Perché è corretta: Nelle istruzioni di load e store (es. `lw a5, 8(a0)`), l'indirizzo fisico della cella in memoria si ricava *sommando* il valore base del registro (es. `a0`) con l'offset costante contenuto nell'istruzione (es. `8`). È fisicamente la ALU a eseguire questa addizione durante la fase EX.

Perché scartare le altre: Sottrazioni o operatori AND (B, D) porterebbero ad accessi di memoria del tutto insensati. Affermare che la ALU sia bypassata (A) è falso, in quanto non esistono altri sommatore dedicati all'indirizzamento dati se non l'ALU principale.

17. **Domanda:** Si consideri una cache direct mapped grande 16 KB, con blocchi di 64 byte per blocco. In che blocco di cache è mappata la parola che sta in memoria all'indirizzo `0x100400`?

- (A) Nel blocco numero 16
- (B) Nel blocco numero 32 o 33
- (C) Nel blocco numero 0
- (D) Nel primo blocco libero
- (E) Nessuna delle altre risposte

Risposta corretta: (A)

Perché è corretta: 1) Calcoliamo il numero di blocchi nella cache: $16 \text{ KB} / 64 \text{ Byte} = 16384 / 64 = 256$ blocchi totali. 2) Convertiamo l'indirizzo esadecimale in decimale: $0x100400 = 1 \cdot 16^5 + 4 \cdot 16^2 = 1048576 + 1024 = 1049600_{10}$. 3) Troviamo l'indice del blocco di memoria originario dividendo per la grandezza del blocco (offset): $1049600 / 64 = 16400$. 4) Calcoliamo la mappatura in cache usando il modulo: $16400 \bmod 256 = 16$.

Perché scartare le altre: Tutte le altre opzioni derivano da conversioni errate dalla base 16 o dal mancato utilizzo dell'operatore modulo sulla quantità totale di blocchi disponibili nella cache.

18. **Domanda:** Le librerie statiche:

- (A) Possono essere utilizzate da programmi C, ma non da programmi scritti in Assembly.
- (B) Vengono utilizzate dall'Assembler per implementare le macro/pseudo-istruzioni.
- (C) Sono utilizzate in fase di linking, ma non servono per il caricamento o l'esecuzione dell'eseguibile finale.
- (D) Sono effettivamente collegate al programma solo quando esso viene caricato o eseguito (lazy linking).

Risposta corretta: (C)

Perché è corretta: Per definizione, una libreria statica (un file `.a`) contiene codice macchina già compilato che il Linker copia fisicamente e in modo permanente all'interno del file eseguibile. Una volta completato il file `a.out` o `.exe`, questo diventa del tutto indipendente; la libreria non viene più letta dal Sistema Operativo durante il caricamento o l'esecuzione.

Perché scartare le altre: L'opzione (D) descrive esplicitamente il comportamento delle librerie *dinamiche* (es. `.so` o `.dll`). L'opzione (B) fa confusione col ruolo del preprocessore o dell'assembler stesso. La (A) è del tutto infondata.

19. **Domanda:** I simboli non definiti (es. riferimenti a librerie esterne) contenuti in un file oggetto non eseguibile (`.o`):

- (A) Sono associati ad indirizzi di memoria relativi.
- (B) Sono associati ad indirizzi di memoria assoluti.
- (C) Non sono associati ad indirizzi di memoria né assoluti né relativi; l'associazione avverrà solo nella fase di linking.
- (D) Nei file non esiste alcuna nozione di "simboli", definiti o no.

Risposta corretta: (C)

Perché è corretta: Un file oggetto intermedio `.o` contiene codice non ancora rilocato. Se invoca una funzione dichiarata altrove (simbolo non definito), l'Assembler non può calcolare la distanza di salto, lasciando il campo vuoto nella *Tabella di Rilocazione*. L'assegnazione spaziale avverrà unicamente ad opera del Linker, quando tutto il codice e le librerie verranno fusi.

Perché scartare le altre: Un file oggetto non può conoscere le coordinate assolute (B) né quelle relative (A) per variabili esterne. La nozione di Tabella dei Simboli è invece la struttura cardine per descrivere l'esportazione di funzioni, rendendo falsa la (D).

20. **Domanda:** Si dica a quale delle seguenti alternative corrisponde la seguente istruzione in assembly ARM: `add r0, r1, r1, lsl #1`

- (A) $r0 = r0 + (2 * r1)$
- (B) $r1 = r0 + (2 * r1)$
- (C) $r0 = 3 * r1$
- (D) $r0 = r0 * r1$

Risposta corretta: (C)

Perché è corretta: La peculiarità di ARM è di poter ruotare o far scorrere un operando a livello hardware prima di passarlo alla ALU, nello stesso ciclo di clock. L'operando `r1, lsl #1` (Logical Shift Left di 1) equivale a moltiplicare `r1` per $2^1 = 2$. La `add` somma quindi il primo sorgente (`r1`) al secondo sorgente appena shiftato ($2 * r1$). L'equazione algebrica è $r0 = r1 + 2r1$, ovvero $r0 = 3r1$.

Perché scartare le altre: La (A) usa erroneamente `r0` come primo addendo anziché `r1`. La (B) inverte registro destinazione e sorgente. La (D) interpreta falsamente l'addizione shiftata come una generica moltiplicazione tra due registri differenti.

21. **Domanda:** Quale delle seguenti alternative è corretta, se si considera il seguente segmento di codice in assembly ARM:

```
sub r0, r0, #4
mov r3, #0
loop:
str r3, [r0, #4]!
cmp r3, #10
bne loop
```

- (A) Scrive i numeri da 1 a 10 nei primi 10 elementi dell'array puntato da r0
- (B) Scrive i numeri da 0 a 9 nei primi 10 elementi dell'array puntato da r0
- (C) Entra in un ciclo infinito
- (D) È scorretto sintatticamente

Risposta corretta: (C)

Perché è corretta: Il registro r3 viene inizializzato a 0 all'esterno del ciclo. Tuttavia, all'interno del blocco `loop`, la variabile r3 viene unicamente scritta in memoria (`str`) ma non viene *mai incrementata* o alterata in alcun modo. L'istruzione `cmp r3, #10` confronterà per sempre il valore 0 con il valore 10, ottenendo costantemente un risultato di disuguaglianza (`bne Branch if Not Equal` eseguito in `loop`).

Perché scartare le altre: Senza un `add r3, r3, #1` le opzioni (A) e (B) non si verificheranno mai, in quanto il contatore resta congelato. Il codice non ha errori sintattici (D), l'assemblatore lo compila senza problemi, l'errore è unicamente logico.

22. **Domanda:** La funzione C `mathforfun(int i, int j, int q, int s)` possiede al suo interno il ciclo: `for(i=0; i<q; i++) { q=q+s; q=q-j; }`
La traduzione Intel x86-64 incompleta è la seguente:

```
.L3:
    movl -4(%rbp), %eax
    X1
    jge L2
    movl -16(%rbp), %eax
    ... // corpo del ciclo
    X2
    jmp L3
.L2:
    movl -12(%rbp), %eax
    ret
```

Con quali istruzioni completereste le righe X1 e X2 mancanti?

- (A) X1: `addl $1, -4(%rbp)` | X2: `cmpl -12(%rcx), %eax`
- (B) X1: `cmpl -12(%rbp), %eax` | X2: `addl $1, -4(%rbp)`
- (C) X1: `addl $1, -8(%rbp)` | X2: `jmp .L2`
- (D) Nessuna delle altre risposte

Risposta corretta: (B)

Perché è corretta: Il blocco logico inizia caricando la variabile *i* (che per l'ABI dello stack frame è a `-4(%rbp)`) nel registro `%eax`. *X1* si trova immediatamente prima del salto condizionato di uscita `jge L2` ("salta se *i* è maggiore o uguale a *q*"). Pertanto in *X1* dobbiamo confrontare *i* con *q* (il cui offset è `-12(%rbp)`): `cmpl -12(%rbp), %eax`. L'istruzione *X2* si colloca alla fine del corpo, dove avviene l'incremento canonico del `for (i++)`, che corrisponde a sommare uno all'indirizzo `-4(%rbp)`: `addl $1, -4(%rbp)`.

Perché scartare le altre: La (A) inverte la posizione del test logico e dell'incremento, rendendo il programma non funzionante. La (C) usa l'offset errato (`-8`) che modificherebbe la variabile *j* invece del contatore.

23. **Domanda:** L'istruzione Assembly `add $1, %rax`:

- (A) Somma 1 al contenuto di `%rax`
- (B) E' scorretta sintatticamente: non specifica l'ampiezza di `%rax`
- (C) Mescola sintassi MIPS e sintassi INTEL
- (D) Nessuna delle altre risposte
- (E) E' un'istruzione ARM valida

Risposta corretta: (B)

Perché è corretta: In Assembly Intel con sintassi GNU (AT&T), le istruzioni che operano su dati devono specificare esplicitamente la dimensione dell'operando tramite un suffisso. Poiché `%rax` è un registro a 64 bit (Quadword), la sintassi corretta esige il suffisso `q`, rendendo l'istruzione corretta `addq $1, %rax`. Senza il suffisso, l'istruzione è formalmente e sintatticamente incompleta.

Perché scartare le altre: Logicamente l'istruzione servirebbe a sommare 1 (A), ma il compilatore restituirebbe errore prima di poterlo fare. Non mescola MIPS (C) e non c'entra con ARM (E) poiché usa registri col prefisso percentuale tipici della sintassi gas x86.

24. **Domanda:** Con riferimento allo schema del datapath standard, si dica a cosa serve il blocco "Estensione Segno".

- (A) Nessuna delle altre risposte
- (B) Gli operandi immediati sono codificati su 16 bit. Mentre i registri sono a 32, quindi occorre estendere l'operando per effettuare operazioni con il registro preservandone il segno.
- (C) L'estensione occorre per individuare correttamente il registro destinatario.
- (D) Il MIPS consente di utilizzare anche porzioni di registri a 16 bit e in questo caso bisogna estendere l'operando.
- (E) L'estensione è usata solo nell'eventualità di salti condizionati per individuare correttamente il registro da sommare al PC.

Risposta corretta: (B)

Perché è corretta: Nelle architetture RISC classiche (come MIPS o RISC-V), le costanti numeriche (immediati) sono inglobate nei 32 bit dell'istruzione stessa. Di conseguenza, possono occupare solo un sottoinsieme di bit (es. 16 bit nel MIPS o 12 bit nel RISC-V). Poiché la ALU opera fisicamente a 32 o 64 bit, l'hardware deve convertire questo piccolo numero in uno grande, replicando il bit di segno a sinistra per preservarne il valore matematico prima dell'operazione.

Perché scartare le altre: I registri destinatario (C) si selezionano usando Multiplexer e campi a 5 bit, non con l'estensione del segno. L'estensione viene usata in tantissime istruzioni (Load, Store, Addi), non "solo" per i salti condizionati (E).

25. **Domanda:** I dispositivi di I/O possono essere classificati in base a:

- (A) Velocità di trasferimento e comportamento (r/w)
- (B) Comportamento (r/w), tipo di BUS, partner, velocità di trasferimento
- (C) Nessuna delle altre risposte
- (D) Comportamento (r/w) e partner
- (E) Comportamento (r/w), partner, velocità di trasferimento

Risposta corretta: (E)

Perché è corretta: Come visto a lezione, la classificazione canonica delle periferiche I/O si basa su tre pilastri: *Comportamento* (se sono dispositivi di input, output o storage R/W), *Partner di interazione* (se comunicano con umani, come i monitor, o con altre macchine, come le reti) e *Data Rate / Velocità di trasferimento* (che spazia da pochi byte/s della tastiera ai GB/s delle schede video).

Perché scartare le altre: Le opzioni (A) e (D) sono incomplete. L'opzione (B) aggiunge il "tipo di BUS", che riguarda l'interfacciamento hardware di basso livello, non la classificazione logica della periferica stessa.

26. **Domanda:** Nel corso dell'esecuzione di una procedura, il registro `s0` contiene il valore 255.125 espresso secondo lo Standard IEEE754. Quale valore conterrà `s0` dopo aver eseguito le seguenti istruzioni Assembly?

```
addi $t0, $zero, 0x1000
sll $t0, $t0, 19
or $s0, $s0, $t0
```

- (A) Il corrispettivo secondo lo standard di 255.12502
- (B) Il corrispettivo secondo lo standard di 256.125×10^7
- (C) Il corrispettivo secondo lo standard di -255.125
- (D) Nessuna delle altre risposte
- (E) Il corrispettivo secondo lo standard di 256.125

Risposta corretta: (C)

Perché è corretta: Questo esercizio unisce l'Assembly alla rappresentazione Floating Point. Il numero 255.125 è positivo, quindi il suo bit più significativo (il bit di Segno) è 0. Il registro `$t0` viene inizializzato a `0x1000` (che ha un singolo bit 1 in posizione 12). L'istruzione di Shift Left Logic (`sll`) sposta questo bit verso sinistra di 19 posizioni, facendolo atterrare in posizione $12 + 19 = 31$, ovvero esattamente sul bit di Segno. Fare un OR bit-a-bit forza il bit di Segno di `$0` a diventare 1, cambiando il segno del numero da positivo a negativo, senza alterare esponente e mantissa. Il valore diventa -255.125 .

Perché scartare le altre: Le alterazioni aritmetiche su mantissa o esponente (A, B, E) sono impossibili, poiché il bit manipolato è unicamente l'ultimo (il 31-esimo).

27. **Domanda:** Un registro invisibile della CPU è:

- (A) Un registro che può essere acceduto solo usando il linguaggio Assembly
- (B) Un registro che non può essere acceduto direttamente, salvo rare eccezioni
- (C) Nessuna delle altre risposte
- (D) Un registro che non può essere direttamente acceduto da alcun programma
- (E) Un registro che non può contenere dati ma solo istruzioni

Risposta corretta: (D)

Perché è corretta: I registri "invisibili" o di architettura interna (come il MAR - Memory Address Register, l'MDR - Memory Data Register, o i registri di transito della pipeline) sono gestiti esclusivamente e automaticamente dall'hardware della CPU. Non esiste alcuna istruzione Assembly in grado di leggerli o scriverli volontariamente da parte di un programmatore.

Perché scartare le altre: Se fossero accessibili tramite Assembly (A, B), farebbero parte dell'ISA e si chiamerebbero "visibili all'architettura" (come PC o registri generali). La (E) è falsa poiché l'MDR, ad esempio, contiene esplicitamente dati temporanei.

28. **Domanda:** Una CPU funziona nel seguente modo:

- (A) La CPU ha già tutte le istruzioni da eseguire memorizzate in un registro nascosto
- (B) Prima la CPU preleva dalla memoria tutte le istruzioni che compongono un programma, poi le decodifica (incrementando il registro IP) e le esegue.
- (C) Ciclicamente, la CPU prima incrementa il registro IP, poi preleva un'istruzione dalla memoria, la decodifica e la esegue (poiché IP è già stato incrementato, l'esecuzione non può modificarlo).
- (D) Nessuna delle altre risposte.
- (E) Ciclicamente, la CPU preleva un'istruzione (puntata dal registro IP) dalla memoria (incrementando poi il registro IP), decodifica l'istruzione e quindi la esegue (l'esecuzione dell'istruzione può modificare il registro IP). Quindi, ripete tutto da capo.

Risposta corretta: (E)

Perché è corretta: Questa è la descrizione esatta del classico "Ciclo Fetch-Decode-Execute" della macchina di Von Neumann. Si preleva l'istruzione puntata dall'Instruction Pointer (o Program Counter, PC), lo si incrementa subito per puntare

all'istruzione successiva, e poi la si esegue. Crucialmente, l'esecuzione *può* modificare nuovamente l'IP (ad esempio in caso di salti condizionati, branch o jump).

Perché scartare le altre: La CPU non preleva *tutto* il programma in blocco prima di iniziare (B). L'affermazione (C) è falsa perché afferma che l'esecuzione non può alterare l'IP, negando così l'esistenza delle istruzioni di salto.

29. **Domanda:** Un programma assembly:

- (A) È una sequenza di 0 e 1 che la CPU interpreta come istruzioni macchina
- (B) Necessita di un apposito interprete per essere eseguito
- (C) Nessuna delle altre risposte
- (D) È scritto in un linguaggio a basso livello e quindi non deve essere compilato
- (E) È codificato in un file ASCII che contiene la descrizione testuale di istruzioni macchina

Risposta corretta: (E)

Perché è corretta: Un programma Assembly (il file sorgente `.s`) è un normalissimo file di testo codificato in ASCII, che contiene i *mnemonici* (parole leggibili come `add`, `mov`, `beq`) scritti da un essere umano. Questo testo è la pura e semplice descrizione testuale delle istruzioni hardware.

Perché scartare le altre: La (A) descrive il *Linguaggio Macchina* (binario), non l'Assembly. La (B) è falsa, l'Assembly non è un linguaggio interpretato (come Python), ma viene assemblato. La (D) è falsa, perché sebbene non venga "compilato" da un linguaggio ad alto livello, deve comunque passare attraverso il programma Assembler per essere tradotto in file oggetto `.o`.

30. **Domanda:** Con riferimento al blocco addizionale posto in alto a destra nel classico schema del datapath MIPS/RISC-V, si dica quale delle seguenti affermazioni è corretta.

- (A) Il blocco addizionale in alto a destra è usato per implementare i salti condizionati. Il blocco shift a sinistra di due che lo precede è necessario per trasformare l'indirizzo del salto da offset relativo a valore assoluto.
- (B) Il blocco addizionale in alto a destra viene utilizzato per calcolare l'indirizzo da cui prelevare un dato richiesto per un'operazione di lw.
- (C) Il blocco addizionale in alto a destra è usato per implementare i salti condizionati. Il blocco shift a sinistra di due che lo precede è necessario per trasformare l'indicazione dell'indirizzo da word a byte.
- (D) Nessuna delle altre risposte

Risposta corretta: (C)

Perché è corretta: Nello schema del datapath, l'addizionatore ausiliario posto in alto serve unicamente a calcolare l'indirizzo bersaglio per le istruzioni di Branch (salti condizionati). Poiché le istruzioni in memoria sono allineate alla *word* (4 byte), l'offset salvato dentro l'istruzione indica il numero di "parole" da saltare. Prima di sommarlo al PC (che conta in byte), questo offset deve essere moltiplicato per 4. In hardware, moltiplicare per 4 equivale a uno Shift Logico a Sinistra di 2 bit, trasformando l'offset da "word" a "byte".

Perché scartare le altre: L'indirizzo base per i salti condizionati è già relativo (si somma al Program Counter), l'addizionatore e lo shift *non* convertono in assoluto (A). Il calcolo degli indirizzi per la memoria dati (**lw**, **sw**) viene eseguito direttamente dalla ALU principale (B), non dall'addizionatore in alto.

31. **Domanda:** I simboli definiti contenuti in un file oggetto (.o) non eseguibile:
- (A) Non sono associati ad indirizzi di memoria né assoluti né relativi; l'associazione ad un indirizzo di memoria avverrà solo nella fase di linking.
 - (B) Sono associati ad indirizzi di memoria relativi.
 - (C) Nessuna delle altre risposte.
 - (D) Nei file .o non esiste alcuna nozione di "simboli definiti".
 - (E) Sono associati ad indirizzi di memoria assoluti.

Risposta corretta: (B)

Perché è corretta: All'interno della *Symbol Table* di un file oggetto appena generato dall'assemblatore, i simboli **definiti** (ovvero funzioni o variabili globali dichiarate *all'interno* di quel file stesso) sono mappati con un indirizzo **relativo** (o offset) calcolato a partire dall'inizio del segmento di quel singolo file (ad es. offset 0x0000 per la prima funzione, offset 0x0014 per la seconda).

Perché scartare le altre: Diventeranno indirizzi *assoluti* (E) solo al termine della fase di Linking. Affermare che non abbiano alcun indirizzo (A) è vero solo per i simboli *NON definiti* (le funzioni esterne). La (D) è palesemente falsa.

32. **Domanda:** L'istruzione `lea (%rdi, %rsi), %rax`:
- (A) Carica in `%rax` il contenuto della locazione di memoria indicata da `%rdi + %rsi`
 - (B) Nessuna delle altre risposte
 - (C) Somma il contenuto dei registri `%rdi` ed `%rsi` salvando il risultato in `%rax`
 - (D) Non è un'istruzione Assembly valida
 - (E) Salva il contenuto di `%rax` nella locazione di memoria indicata da `%rdi + %rsi`

Risposta corretta: (C)

Perché è corretta: L'istruzione `lea` (Load Effective Address) dell'architettura Intel è mascherata da operazione di memoria (usando le parentesi tonde), ma a livello hardware aggira l'accesso alla RAM. Passa i due registri ai circuiti di calcolo degli indirizzi (AGU), ne esegue la somma ($rdi + rsi$) e salva puramente il risultato matematico nel registro destinazione `%rax`.

Perché scartare le altre: Se la CPU accedesse effettivamente alla memoria per prelevare il dato (A), l'istruzione corretta sarebbe una normale `movq (%rdi, %rsi), %rax`. La (E) descrive una *store*, eseguita invece con `movq %rax, (%rdi, %rsi)`.

33. **Domanda:** Si consideri l'istruzione Assembly Intel: `movb (%rsi, %rax), %b1`. Si dica quale delle seguenti alternative è vera:

- (A) È un'istruzione ARM sbagliata sintatticamente.
- (B) Nessuna delle altre risposte.
- (C) È un'istruzione MIPS sbagliata sintatticamente.
- (D) È un'istruzione INTEL che sposta un byte dall'indirizzo di memoria puntato da `%rsi + %rax` nel sotto-registro `%b1`.
- (E) È un'istruzione INTEL che sposta nel registro `%b1` la word a 64 bit puntata da `%rsi + %rax`.

Risposta corretta: (D)

Perché è corretta: La sintassi appartiene inequivocabilmente ad Assembly x86 GNU (registri prefissati da %, memoria indicata tra parentesi). Il suffisso `b` nel mnemonico `movb` sta per *byte* (8 bit). Il registro destinazione `%b1` è esattamente la porzione a 8 bit bassa del registro base `%rbx`. L'istruzione preleva dunque 1 byte dalla RAM all'indirizzo $rsi + rax$.

Perché scartare le altre: La (E) afferma che viene copiata una word a 64 bit, ma questo è impossibile per due motivi: il suffisso `b` indica esplicitamente un solo byte, e il registro destinatario `%b1` fisicamente non ha lo spazio per contenere 64 bit (servirebbe `%rbx` intero).

34. **Domanda:** Nell'architettura RISC/MIPS, un gestore di eccezione (Exception Handler):

- (A) È pre-programmato in un "coprocessore" e non può essere modificato dall'utente.
- (B) Viene eseguito dalla CPU solo dopo che tutte le fasi di polling sono terminate.
- (C) Può leggere in registri speciali (Cause, EPC ed altri) il motivo dell'eccezione, l'indirizzo della prossima istruzione da caricare dopo aver gestito l'eccezione, etc...
- (D) Non è in grado di capire la causa di un'eccezione.
- (E) Nessuna delle altre risposte.

Risposta corretta: (C)

Perché è corretta: Quando scatta un'eccezione, l'hardware del processore esegue un cambio di contesto forzato passando il controllo al Sistema Operativo e riempiendo automaticamente dei registri speciali di stato (i CSR, o Coprocessor 0 in MIPS). L'handler software inizierà la sua esecuzione proprio leggendo il registro **SCAUSE** (o **Cause**) per capire chi ha generato l'errore o l'interrupt, e userà il registro **SEPC** (o **EPC**) per sapere a quale punto del programma tornare a fine esecuzione.

Perché scartare le altre: Un handler è un normalissimo pezzo di codice software risiedente nel kernel del SO (non inalterabile in hardware) (A). L'interrupt serve proprio ad *evitare* il polling, non attende che finisca (B). È assolutamente in grado di capire la causa (D), altrimenti non saprebbe come gestire l'errore.

35. **Domanda:** I bit di controllo che dicono alla ALU esattamente quale operazione svolgere sono generati da:

- (A) Un'unità di controllo che riceve in ingresso il campo *funct* prelevato dall'istruzione e i due bit detti *ALUOp*.
- (B) Un'unità di controllo che riceve in ingresso solo il campo *funct* prelevato dall'istruzione.
- (C) Un'unità di controllo che riceve in ingresso i campi *funct* e *shamt* prelevati dall'istruzione e i due bit detti *ALUOp*.
- (D) Nessuna delle altre risposte.
- (E) Un'unità di controllo che riceve in ingresso solo i due bit detti *ALUOp*.

Risposta corretta: (A)

Perché è corretta: Nel design del datapath, l'Unità di Controllo Principale decodifica l'Opcode base dell'istruzione e genera un pre-segnaletto riassuntivo di 2 bit chiamato *ALUOp* (ad es. "10" indica che è un'istruzione di tipo R). Questo *ALUOp* viene inviato a una piccola rete combinatoria secondaria (il *Controllore della ALU*) che lo incrocia con i bit del campo *funct* (**funct7** e **funct3**) provenienti dall'istruzione, generando così il segnale a 4 bit finale che accende la circuiteria corretta (es. l'addizionatore, o il moltiplicatore) dentro la ALU.

Perché scartare le altre: Il campo *funct* da solo non basta (B), perché istruzioni con formato diverso potrebbero casualmente avere lo stesso *funct*. Anche gli *ALUOp* da soli non bastano (E) per distinguere, ad esempio, un'addizione da una sottrazione all'interno del formato R. Lo *shamt* (C) serve per l'hardware di traslazione logica, non per pilotare il controllo dell'operatore aritmetico.

36. **Domanda:** Si dica quale delle seguenti affermazioni sulla Memoria è vera.

- (A) La memoria ram statica è velocissima ma necessita di operazioni di rinfresco periodiche per ripristinare i livelli elettrici dei dati.
- (B) La memoria ram statica non può essere usata all'interno di un processore perché è molto ingombrante.
- (C) Una cella di memoria statica necessita di un condensatore e di un CMOS.
- (D) La memoria ram statica è molto veloce ma costosa.
- (E) Nessuna delle altre risposte.

Risposta corretta: (D)

Perché è corretta: La SRAM (Static RAM) è basata su flip-flop ed è caratterizzata da tempi di accesso in nanosecondi (veloce tanto quanto il clock della CPU). Il suo principale svantaggio è l'impiego di ben 6 transistor per cella, il che riduce drasticamente la densità e ne fa schizzare il costo per Gigabyte alle stelle. Per questo viene usata solo per chip microscopici come le memorie Cache.

Perché scartare le altre: È la DRAM (Dinamica) a richiedere le cicliche operazioni di rinfresco (A) e a utilizzare un condensatore per mantenere la carica (C). L'affermazione (B) è falsa in quanto la memoria statica è *esattamente* la memoria primaria implementata dentro ai microprocessori sotto forma di Cache L1/L2/L3.

37. **Domanda:** Riguardo ai Bus di sistema, quale delle seguenti affermazioni è corretta:

- (A) Esistono due tipi di BUS: Bus processore-memoria e BUS I/O.
- (B) Il BUS processore-memoria è generalmente lungo e lento.
- (C) I BUS di I/O sono generalmente collegati alla memoria in maniera diretta.
- (D) Esiste un solo unico BUS parallelo che collega tutto.
- (E) Nessuna delle altre risposte.

Risposta corretta: (A)

Perché è corretta: Nelle architetture moderne i bus sono gerarchici e divisi in due macro-categorie. Il *Bus processore-memoria* (spesso proprietario, cortissimo e sincronizzato col clock ad altissima frequenza) collega unicamente la CPU e la RAM per massimizzare la banda. I *Bus di I/O* (come USB o PCIe) sono standardizzati, più lunghi, intrinsecamente asincroni o lenti per accomodare periferiche eterogenee. I due mondi si parlano tramite un controller bridge.

Perché scartare le altre: Il bus processore-memoria è cortissimo e ultra-veloce, non lungo e lento (B). I bus di I/O non si collegano mai direttamente ai banchi di RAM senza passare da un controllore di interfaccia/DMA (C). L'idea di un unico gigantesco bus per tutto (D) risale ai primi rudimentali computer ed è fisicamente inapplicabile oggi per problemi di carico elettrico e clock.

38. **Domanda:** Quale delle seguenti affermazioni riguardanti i Latch è FALSA?

- (A) Tutti i tipi di Latch sono circuiti Bistabili.
- (B) Tutti i Latch sono di tipo asincrono.
- (C) I Latch di tipo D possono costituire l'unità base di una memoria.
- (D) I Latch di tipo D possono essere usati per realizzare delle macchine a stati sincrone.
- (E) Tutte le affermazioni sono da considerare FALSE.

Risposta corretta: (D)

Perché è corretta: Per progettare una macchina a stati *sincrona* (che aggiorna il suo stato interno solo a passi temporali rigidamente scanditi) è fisicamente obbligatorio utilizzare elementi sensibili unicamente al **fronte** del clock (Edge-triggered Flip-Flops). Un Latch D puro è invece trasparente "a livello": se il segnale di abilitazione resta alto, un feedback combinatorio farebbe ruzzolare i dati incontrollatamente nel circuito per tutta la durata del clock (race condition), rendendolo inidoneo per gestire macchine a stati complesse e sincronizzate.

Perché scartare le altre: Essendo la (D) palesemente falsa, le altre (A e C) sono nozioni vere del mondo dell'elettronica (un Latch è effettivamente un circuito con due stati stabili ed è impiegato per memorizzare bit statici isolati).

39. **Domanda:** Si consideri la seguente istruzione Assembly ARM: `ldr r3, [r0, #8]`. Si dica quale delle seguenti alternative è falsa:

- (A) È una modalità di indirizzamento con offset immediato.
- (B) Il contenuto di `r0` non viene modificato.
- (C) Si tratta di un'istruzione da memoria a registro.
- (D) Il contenuto puntato da `r0+8` (base e spiazzamento) viene copiato in `r3`.
- (E) Il contenuto di `r0` viene incrementato di 1 dopo aver eseguito l'istruzione.

Risposta corretta: (E)

Perché è corretta: L'istruzione in oggetto esegue un indirizzamento *Pre-Indexed* canonico. Si preleva il dato dalla memoria all'indirizzo $r0 + 8$, ma l'indirizzo base dentro al registro `r0` rimane inalterato. Affinché `r0` venisse modificato ed aggiornato hardware con l'offset sommato, l'istruzione avrebbe dovuto presentare il suffisso con il punto esclamativo del write-back: `ldr r3, [r0, #8]!`. Per questo motivo, affermare che `r0` venga incrementato è categoricamente falso (peraltro non si incrementerebbe mai di 1, ma al massimo di 8).

Perché scartare le altre: Tutte le opzioni A, B, C e D sono descrizioni perfette e vere del comportamento di questa Load da memoria su architettura ARM.

40. **Domanda:** Quali delle seguenti singole istruzioni assembly RISC-V equivale alle due istruzioni seguenti?

```
add x6, x5, x10
ld x7, 0(x6)
```

- (A) `mv x6, x7`
- (B) `add x6, x7, x5`
- (C) `ld x7, x10(x5)`
- (D) Tutte le risposte si equivalgono
- (E) Nessuna delle altre risposte

Risposta corretta: (E)

Perché è corretta: Il frammento esegue una somma tra due registri (x5 e x10) salvando il risultato in x6, e poi usa questo nuovo indirizzo per caricare un dato dalla memoria in x7. Architetture come ARM o Intel x86 permettono di fare questo in una singola istruzione (usando l'indirizzamento base+indice, es. `ldr r7, [r5, r10]`). Tuttavia, l'architettura **RISC-V non supporta l'indirizzamento registro+registro** per le operazioni in memoria. In RISC-V l'unico modo per accedere alla RAM è sommare un registro a un *offset immediato* costante (es. `ld x7, 8(x5)`). Pertanto, la sintassi `ld x7, x10(x5)` è illegale e inesistente in RISC-V. Non potendo fondere le due istruzioni, la risposta è "Nessuna delle altre".

Perché scartare le altre: La (C) rappresenta una sintassi non valida per il set di istruzioni base del RISC-V. Le opzioni (A) e (B) eseguono operazioni completamente diverse (spostamento di valori o somme logiche) che non coinvolgono minimamente la memoria e la fase di Load.

41. **Domanda:** Si consideri la seguente funzione nel linguaggio C chiamata "swap", il cui scopo è quello di scambiare il valore dell'elemento di un vettore v (di tipo `long long int`) in posizione k con l'elemento successivo k+1.

```
void swap (long long int v[], long long int k) {
    long long int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

Quale delle implementazioni in assembly RISC-V della funzione **swap** è corretta tra quelle proposte? (Si assuma l'indirizzo base v in x10 e l'indice k in x11).

(A) `slli x6, x11, 4`
`add x6, x10, x6`
`ld x5, 0(x6)`
`ld x7, 8(x6)`
`sd x7, 0(x6)`
`sd x5, 8(x6)`
`jalr x0, 0(x1)`

(B) `slli x6, x11, 3`
`add x6, x10, x6`
`ld x5, 0(x6)`
`ld x7, 8(x6)`
`sd x7, 0(x6)`
`sd x5, 8(x6)`
`jalr x0, 0(x1)`

(C) `slli x6, x11, 3`
`add x6, x10, x6`
`lw x5, 0(x6)`
`lw x7, 4(x6)`
`sw x7, 0(x6)`

```
sw x5, 4(x6)
jalr x0, 0(x1)
```

(D) Nessuna delle altre risposte

Risposta corretta: (B)

Perché è corretta: Questa è la traduzione perfetta della funzione `swap`. Essendo l'array di tipo `long long int`, ogni elemento occupa 8 byte.

- L'indice `k` (in `x11`) viene moltiplicato per 8 con uno shift a sinistra di 3 posizioni (`slli x6, x11, 3`).
- Si somma l'offset all'indirizzo base (`add x6, x10, x6`) ottenendo il puntatore fisico a `v[k]`.
- Si estraggono i due elementi in registri temporanei: `v[k]` a offset 0 (`ld x5, 0(x6)`) e `v[k+1]` a offset 8 byte (`ld x7, 8(x6)`).
- Si scrivono i valori in memoria invertendoli di posizione (`sd x7, 0(x6)` e `sd x5, 8(x6)`).
- Si ritorna al chiamante (`jalr`).

Perché scartare le altre: L'opzione (A) esegue uno shift di 4 posizioni (`slli x6, x11, 4`), che moltiplica l'indice per 16, sfalsando completamente il calcolo dell'indirizzo in memoria. L'opzione (C) utilizza istruzioni `lw` / `sw` (che caricano e salvano solo 4 byte) e offset incrementati di 4; questo sarebbe corretto per array di normali `int`, ma fallisce gravemente troncando a metà i `long long int` a 64 bit richiesti dal frammento C.

42. **Domanda:** Si consideri una CPU le cui fasi impiegano: 150ps per il fetch, 350ps per la decodifica, 500ps per la ALU, 400ps per la memoria e 200ps per il write-back. Il massimo incremento di prestazioni atteso con la pipeline è:

- (A) di 3 volte
- (B) di 2.8 volte
- (C) di 4 volte
- (D) di 3.2 volte
- (E) nessuna delle altre risposte

Risposta corretta: (D)

Perché è corretta: Calcoliamo il tempo di esecuzione sequenziale di una singola istruzione sommando i tempi delle fasi: $150 + 350 + 500 + 400 + 200 = 1600$ ps. In un'architettura a pipeline, il tempo del ciclo di clock globale è vincolato dallo stadio più lento (il collo di bottiglia), che in questo caso è la ALU con 500 ps. Lo speedup (incremento di prestazioni) si calcola dividendo il tempo totale sequenziale per il tempo di clock della pipeline: $1600/500 = 3.2$. L'incremento atteso è di 3.2 volte.

Perché scartare le altre: Le opzioni (A), (B) e (C) sono il risultato di calcoli errati. Ad esempio, (A) deriverebbe se si facesse una stima approssimativa o (B) se il clock fosse calcolato su un'altra fase scorretta.

43. **Domanda:** Si consideri una cache set associative a 8 vie grande 32KB, con blocchi di 64 byte. In quali blocchi può essere mappato l'indirizzo 0x205C0?

- (A) Nei blocchi dal numero 23 al numero 30
- (B) Nei blocchi dal numero 184 al numero 191
- (C) Nel blocco numero 184
- (D) Nei blocchi dal numero 92 al numero 99
- (E) Nessuna delle altre risposte

Risposta corretta: (B)

Perché è corretta: Calcoliamo i parametri della cache. Dimensione totale = 32 KB = 32768 byte. Numero totale di blocchi in cache = $32768/64 = 512$ blocchi. Essendo associativa a 8 vie, il numero di Set è $512/8 = 64$ Set. Convertiamo l'indirizzo in decimale: $0x205C0 = 2 \cdot 16^4 + 5 \cdot 16^2 + 12 \cdot 16 = 131072 + 1280 + 192 = 132544$. L'indirizzo del blocco in memoria è $\lfloor 132544/64 \rfloor = 2071$. Troviamo il Set calcolando il modulo: $2071 \bmod 64 = 23$. Il dato finisce nel Set logico 23. Poiché ogni Set è formato da 8 vie (8 blocchi fisici contigui), il Set 23 inizia dal blocco fisico $23 \times 8 = 184$ e finisce al blocco $184 + 7 = 191$.

Perché scartare le altre: La (A) indica il Set logico, non i blocchi fisici. La (C) indica un solo blocco ignorando l'associatività a 8 vie. La (D) moltiplicherebbe per 4 invece che per 8.

44. **Domanda:** Dato un sistema con indirizzamento a 32 bit e una memoria cache la cui dimensione del blocco (cache line) è di 64 byte, qual è il numero del blocco di memoria in decimale corrispondente all'indirizzo 0x004A1F2C?

- (A) 75900
- (B) 48576
- (C) 18975
- (D) 303600
- (E) Nessuna delle altre risposte

Risposta corretta: (A)

Perché è corretta: Per trovare a quale "blocco di memoria" corrisponde un indirizzo, bisogna semplicemente ignorare i bit di Offset (ovvero effettuare una divisione intera per la grandezza del blocco). Convertiamo l'indirizzo esadecimale in decimale: $0x004A1F2C = 4 \cdot 16^5 + 10 \cdot 16^4 + 1 \cdot 16^3 + 15 \cdot 16^2 + 2 \cdot 16 + 12 = 4194304 + 655360 + 4096 + 3840 + 32 + 12 = 4857644$. Dividiamo questo indirizzo per la dimensione del blocco (64 byte): $\lfloor 4857644/64 \rfloor = 75900$. (L'offset all'interno del blocco è il resto, cioè 44).

Perché scartare le altre: Le altre risposte presentano errori nei calcoli di conversione o omettono la divisione finale per determinare l'indice del blocco.

45. **Domanda:** Si consideri una cache direct mapped grande 16KB con blocchi da 64 byte. In quale blocco di cache andrà la parola all'indirizzo 0x81C0?

- (A) Nel blocco 192
- (B) Nel blocco 7
- (C) Nel blocco 64
- (D) Nel blocco 128
- (E) Nessuna delle altre risposte

Risposta corretta: (B)

Perché è corretta: Troviamo i blocchi totali della cache: $16 \text{ KB} = 16384 \text{ byte}$. Numero di blocchi $= 16384/64 = 256$ blocchi totali. Convertiamo l'indirizzo in decimale: $0x81C0 = 8 \cdot 16^3 + 1 \cdot 16^2 + 12 \cdot 16 = 32768 + 256 + 192 = 33216$. Calcoliamo l'indice del blocco di memoria dividendo per 64 (scartando l'offset): $\lfloor 33216/64 \rfloor = 519$. Calcoliamo in che blocco di cache verrà mappato tramite l'operatore modulo: $519 \bmod 256 = 7$.

Perché scartare le altre: Le opzioni errate derivano tipicamente dal dimenticare la divisione per l'offset (facendo il modulo direttamente sull'indirizzo anziché sull'indice del blocco) o da un'errata conversione di base.

46. **Domanda:** Una CPU presenta un CPI ideale di 5. La cache istruzioni ha un miss rate del 3% e la cache dati del 6%. Una cache miss richiede 80 cicli per essere risolta. Se il 25% delle istruzioni accede alla memoria dati, calcolare il CPI reale.

- (A) 8.6
- (B) 7.4
- (C) 12.2
- (D) 5.24
- (E) Nessuna delle altre risposte

Risposta corretta: (A)

Perché è corretta: Il CPI reale si ottiene sommando al CPI ideale (5) le penalità per i miss sulle istruzioni e sui dati. 1. **Penalità Istruzioni:** Tutte le istruzioni (100%) vengono lette in fase di Fetch. Miss Rate = 0.03. Penalità $= 1.0 \times 0.03 \times 80 = 2.4$ cicli per istruzione. 2. **Penalità Dati:** Solo il 25% delle istruzioni esegue un accesso alla memoria dati. Miss Rate = 0.06. Penalità $= 0.25 \times 0.06 \times 80 = 1.2$ cicli per istruzione. CPI Reale = CPI Ideale + Penalità Istruzioni + Penalità Dati $= 5 + 2.4 + 1.2 = 8.6$.

Perché scartare le altre: (C) calcola assumendo che il 100% delle istruzioni acceda anche alla cache dati ($5 + 2.4 + 4.8 = 12.2$). Le altre opzioni derivano da mancate moltiplicazioni per 80 o da errori nei fattori.

47. **Domanda:** Si consideri una CPU con una pipeline idealmente bilanciata, in cui ciascuna delle 5 fasi di esecuzione impiega esattamente 300ps. Qual è il massimo incremento di prestazioni che ci si aspetta usando la pipeline?

(A) di 4 volte
(B) di 3 volte
(C) di 5 volte
(D) di 6 volte
(E) nessuna delle altre risposte

Risposta corretta: (C)

Perché è corretta: In una pipeline *idealmente bilanciata*, tutti gli stadi hanno esattamente la stessa durata. Il tempo di esecuzione sequenziale di una singola istruzione è la somma dei 5 stadi: $300 \times 5 = 1500$ ps. Il tempo di ciclo della pipeline è dettato dallo stadio più lento, ma essendo tutti uguali, il clock sarà esattamente di 300 ps. Lo speedup (incremento massimo) si ottiene facendo il rapporto tra il tempo senza pipeline e il tempo con pipeline: $1500/300 = 5$. In generale, per una pipeline perfettamente bilanciata a N stadi, lo speedup teorico massimo è esattamente N .

Perché scartare le altre: Le altre opzioni (4, 3, 6) non corrispondono al numero di stadi della pipeline indicata, fallendo l'applicazione della formula dello speedup ideale.

48. **Domanda:** Si consideri una cache set associative a 2 vie grande 4KB, con blocchi da 32 byte. Dove sarà mappata la parola all'indirizzo 0x41E8?

(A) Nel blocco numero 30 o nel blocco numero 31
(B) Nel blocco 15 o 16
(C) Nel blocco 60 o 61
(D) Nel blocco 30 e basta
(E) Nessuna delle altre risposte

Risposta corretta: (A)

Perché è corretta: Procediamo per passi. 1) Troviamo il numero totale di blocchi fisici: Dimensione 4 KB = 4096 byte. Blocchi totali = $4096/32 = 128$ blocchi. 2) Essendo associativa a 2 vie, raggruppiamo i blocchi in *Set*: $128/2 = 64$ Set totali. 3) Convertiamo l'indirizzo esadecimale 0x41E8 in decimale: $4 \cdot 16^3 + 1 \cdot 16^2 + 14 \cdot 16 + 8 = 16384 + 256 + 224 + 8 = 16872_{10}$. 4) Calcoliamo l'indice del blocco di memoria originario dividendo per la dimensione del blocco (32): $\lfloor 16872/32 \rfloor = 527$. 5) Troviamo il Set di destinazione con l'operatore modulo: $527 \bmod 64 = 15$. Il dato finirà nel **Set 15**. 6) Poiché ogni Set è composto da 2 vie contigue, il Set 15 ingloba fisicamente i blocchi della cache numero $15 \times 2 = 30$ e $15 \times 2 + 1 = 31$.

Perché scartare le altre: La (B) indica il numero del Set logico (15) e il successivo (16), non i blocchi fisici che lo compongono. La (C) userebbe un moltiplicatore errato. La (D) dimentica che la cache è a 2 vie, affermando che il blocco possibile è solo uno.

49. **Domanda:** Una cache fully associative grande 64KB ha blocchi da 256 byte. Il blocco di memoria corrispondente all'indirizzo 0xFFE0 verrà scritto:

- (A) In un qualsiasi blocco della cache determinato dalla politica di rimpiazzo
- (B) Nel blocco 255
- (C) Nel blocco 64
- (D) Nei primi 256 blocchi
- (E) Nessuna delle altre risposte

Risposta corretta: (A)

Perché è corretta: La domanda è teorica e a trabocchetto. Per definizione, in una memoria cache **completamente associativa (Fully Associative)** non esiste alcun indice vincolante. L'intera cache agisce come un unico, gigantesco Set. Di conseguenza, un blocco di memoria può essere salvato in **qualsiasi blocco fisico libero** della cache, indipendentemente dal suo indirizzo. Nel caso la cache fosse piena, il blocco da sovrascrivere viene scelto unicamente dall'algoritmo di rimpiazzo (es. LRU o FIFO).

Perché scartare le altre: Fornire un numero di blocco specifico (B, C) o un range limitato (D) è una proprietà delle cache Direct Mapped o Set-Associative, il che contraddice la natura stessa della Fully Associative. I dati numerici forniti servivano solo a confondere.

50. **Domanda:** Si consideri una cache direct mapped grande 2KB con blocchi da 16 byte. Trovare il blocco in cui viene mappato l'indirizzo 0x1034.

- (A) Nel blocco 52
- (B) Nel blocco 16
- (C) Nel blocco 3
- (D) Nel blocco 19
- (E) Nessuna delle altre risposte

Risposta corretta: (C)

Perché è corretta: 1) Calcoliamo il numero totale di blocchi della cache: Dimensione 2 KB = 2048 byte. Numero di blocchi = $2048/16 = 128$ blocchi totali. 2) Convertiamo l'indirizzo esadecimale 0x1034 in decimale: $1 \cdot 16^3 + 0 + 3 \cdot 16 + 4 = 4096 + 48 + 4 = 4148_{10}$. 3) Ricaviamo l'indice del blocco in memoria RAM ignorando l'offset: $\lfloor 4148/16 \rfloor = 259$. 4) Calcoliamo il blocco di destinazione nella cache con il modulo: $259 \bmod 128 = 3$. L'indirizzo atterrà quindi nel **blocco numero 3**.

Perché scartare le altre: Le opzioni errate derivano da calcoli aritmetici olografici sbagliati (es. mancata divisione per la grandezza del blocco prima di applicare il modulo, o fallita conversione da Hex a Decimale).

51. **Domanda:** Calcolare il CPI reale di una CPU sapendo che: il CPI ideale è 3, la percentuale di cache miss sulle istruzioni è dell'1.5%, la percentuale di miss sui dati è del 4%, la miss penalty ammonta a 120 cicli e il 40% del codice assembly esegue operazioni di load/store (accesso dati in memoria).

- (A) 4.8
- (B) 8.72
- (C) 5.2
- (D) 6.72
- (E) Nessuna delle altre risposte

Risposta corretta: (D)

Perché è corretta: Appliciamo la formula dei cicli di stallo persi sommandoli al CPI base (3). 1. **Stallo Istruzioni:** Tutte le istruzioni vengono lette in fase di Fetch (100% frequenza). Cicli persi: $1.0 \times 0.015 \times 120 = 1.8$ cicli/istruzione. 2. **Stallo Dati:** Solo il 40% delle istruzioni fa ricorso alla cache dati. Cicli persi: $0.40 \times 0.04 \times 120 = 1.92$ cicli/istruzione. Calcoliamo il totale: CPI Reale = CPI Ideale + Stalli Istr. + Stalli Dati = $3 + 1.8 + 1.92 = \mathbf{6.72}$.

Perché scartare le altre: La (B) calcolerebbe 8.72 ignorando la frequenza del 40% sulle load/store, trattandole come se fossero il 100% ($3 + 1.8 + 4.8 = 9.6$ circa oppure derivata da errori di somma). La (A) e la (C) si limitano a calcoli parziali o messe addizioni del CPI ideale base.

52. **Domanda:** Quale delle seguenti proposte descrive meglio il contenuto di un file oggetto?

- (A) Il file oggetto viene prodotto dal linker e contiene il codice eseguibile pronto per essere eseguito.
- (B) Il file oggetto contiene oltre al codice tradotto, anche informazioni sui simboli e le funzioni utilizzate.
- (C) Il file oggetto è un file di testo che contiene il codice sorgente del programma.
- (D) Nel file oggetto tutti gli indirizzi di memoria sono risolti e pronti per essere eseguiti.
- (E) Nessuna delle precedenti.

Risposta corretta: (B)

Perché è corretta: Il file oggetto viene generato dall'assemblatore e contiene il codice macchina tradotto unito a tabelle fondamentali, come la Symbol Table (informazioni sui simboli e le funzioni utilizzate o esportate) e la Tabella di Rilocazione, essenziali per il successivo processo di linking.

Perché scartare le altre: La prima opzione è errata in quanto il file oggetto è prodotto dal compilatore/assemblatore e non dal linker, e rappresenta un codice intermedio non ancora eseguibile. La terza opzione è falsa perché non è un file di testo col codice sorgente. La quarta opzione è errata poiché gli indirizzi di memoria (specialmente i riferimenti esterni) non sono tutti risolti, ma verranno rilocati e risolti solo successivamente dal linker.

53. **Domanda:** Quale delle seguenti affermazioni descrive correttamente gli interrupts?

- (A) Gli interrupts sono segnali gestiti in maniera sincrona dal processore.
- (B) Gli interrupts causano sempre (o quasi) l'abort del programma in esecuzione.
- (C) Gli interrupts sono causati da una specifica richiesta del sistema.
- (D) Gli interrupts possono essere gestiti tra due istruzioni.
- (E) Nessuna delle precedenti.

Risposta corretta: (D)

Perché è corretta: Gli interrupts possono essere gestiti tra l'esecuzione di due istruzioni. Quando si verifica l'evento, il processore completa l'istruzione corrente, sospende il programma, salva lo stato dei registri e avvia la routine di gestione (ISR). Al termine, riprende l'esecuzione da dove era stata interrotta[cite: 14, 15].

Perché scartare le altre: Gli interrupts sono asincroni, generati da eventi esterni al programma in esecuzione (come un segnale I/O)[cite: 13, 14]. Non causano l'abort del programma ma lo sospendono momentaneamente. Le richieste specifiche del sistema o anomalie sincrone si definiscono eccezioni (o traps), non interrupts.

54. **Domanda:** Sia una memoria cache del tipo direct mapped, questa è grande 16KB con blocchi grandi 32 bytes, in quale blocco di questa cache potremmo trovare il dato con indirizzo 0x65F3AB?

- (A) Nel blocco 0x143 della cache.
- (B) Nel blocco 0x19E della cache.
- (C) Nel blocco 0x21B2 della cache.
- (D) Nel blocco 0x21C4 della cache.
- (E) Nessuna delle precedenti.

Risposta corretta: (E)

Perché è corretta: Calcoliamo i campi dell'indirizzo: ogni blocco è di 32 bytes, quindi l'Offset richiede $\log_2(32) = 5$ bit. La cache ha $16\text{KB}/32\text{B} = 512$ blocchi, richiedendo $\log_2(512) = 9$ bit per l'Indice del blocco[cite: 20, 21]. L'indirizzo binario di 0x65F3AB è 0110 0101 1111 0011 1010 1011. Estraendo i 9 bit dell'indice (dal bit 5 al bit 13), otteniamo 1 1001 1101, che in esadecimale equivale a 0x19D. Dato che 0x19D non è tra le opzioni (A, B, C, D), la risposta corretta è "Nessuna delle precedenti".

Perché scartare le altre: Le opzioni A, B, C e D riportano blocchi errati calcolati in modo inesatto (es. includendo offset o operando con basi logaritmiche errate).

55. **Domanda:** Supponendo un processore che possa eseguire un'istruzione in 3 cicli di clock nel caso migliore, per accedere alla memoria istruzioni impiega 100 cicli di clock ma la frequenza di cache-miss per questa memoria è del 1%. Inoltre il 40% delle istruzioni eseguite accede alla memoria dati almeno una volta. Se l'accesso alla memoria dati impiega 200 cicli di clock e la frequenza di cache-miss per questa memoria è del 5%, quanti cicli di clock in media impiega il processore per eseguire un'istruzione?
- (A) 5 cicli di clock.
 (B) 10 cicli di clock.
 (C) 7 cicli di clock.
 (D) 8 cicli di clock.
 (E) Nessuna delle precedenti.

Risposta corretta: (D)

Perché è corretta: Si applica la formula per il calcolo del tempo medio (CPI Reale). I cicli per istruzione ideali sono $T_{ist} = 3$. La penalità per le istruzioni è il miss rate (1%) moltiplicato per il costo del miss (100 cicli) = $0.01 \times 100 = 1$ ciclo. La penalità per i dati si applica solo al 40% delle istruzioni, con un miss rate del 5% e costo di 200 cicli: $0.40 \times (0.05 \times 200) = 0.40 \times 10 = 4$ cicli. Sommando: $3 + 1 + 4 = 8$ cicli medi per istruzione.

Perché scartare le altre: Le altre risposte sono figlie di errori nell'applicazione della formula, tipicamente dimenticandosi di calcolare la probabilità di accesso alla memoria dati (40%) o valutando male i costi.

56. **Domanda:** Viene dato un frammento C e una traduzione ARM incompleta. Quali sono le istruzioni da inserire al posto di X1 e X2 per completare correttamente la traduzione?

```
long long int supreamSum (long long int * a, long long int * b,
                          short n, short m) {
    short i = 0;
    long long int sum = 0;
    while (i < n && i < m) {
        sum += a[i] * b[i++];
    }
    return sum;
}
```

- (A) X1: `ldr r2, [r4, #8]` | X2: `pop {r4, r5, r6, pc}`
 (B) X1: `ldr r2, [r4, #8]!` | X2: `pop {r4, r5, r6, pc}`
 (C) X1: `ldr r2, [r4, #4]` | X2: `pop {r4, r5, r6, lr}`
 (D) X1: `ldr r2, [r4, #4]!` | X2: `pop {r4, r5, r6, pc}`
 (E) Nessuna delle precedenti.

Risposta corretta: (B)

Perché è corretta: Il codice lavora su `long long int`, i cui elementi occupano 8 byte. Nel ciclo per scorrere l'array `b`, il compilatore deve aggiornare il puntatore caricando contemporaneamente il valore e spostandosi avanti di 8 byte per l'iterazione successiva. L'istruzione in X1 utilizza l'indirizzamento pre-indexed con writeback (indicato col suffisso `!`): `ldr r2, [r4, #8]!` [cite: 59, 60, 61]. Alla fine della funzione (X2), i registri salvati in testa con `push` devono essere estratti e l'indirizzo di ritorno va messo nel Program Counter (`pc`) per concludere automaticamente la chiamata: `pop {r4, r5, r6, pc}`.

Perché scartare le altre: L'opzione A manca del writeback (`!`), congelando il puntatore in un loop letale. Le opzioni C e D usano incrementi di 4 byte, errati poiché i dati sono a 64 bit (8 byte). Inoltre, `pop` nel `lr` (C) richiederebbe un'istruzione aggiuntiva per saltare indietro.

57. **Domanda:** Viene dato un frammento C e una traduzione Intel x86-64 incompleta. Quale istruzione deve essere inserita al posto di X1 per completare correttamente la traduzione?

```
int mystery (int a[], int n, int b[], int m) {
    int res = 1;
    for (int i=0; i<n && (res<a[i] || res<b[i]) && i<m; i++) {
        res *= a[i] + b[i/2];
    }
    return res;
}
```

- (A) X1: `addl %edi, %eax`
- (B) X1: `imull %edi, %eax`
- (C) X1: `addl %edi, %r8d`
- (D) X1: `imull %edi, %r8d`
- (E) Nessuna delle precedenti.

Risposta corretta: (D)

Perché è corretta: Nel frammento assembly analizzato, la somma $a[i] + b[i/2]$ è stata appena calcolata nel registro `%edi`. L'operazione richiesta per aggiornare `res` è una moltiplicazione (`*`). La variabile di accumulo `res` risiede nel registro `%r8d` (in quanto inizializzata a 1 all'esterno del ciclo e restituita in `%eax` alla fine). L'istruzione corretta per moltiplicare i due e salvare il risultato in `%r8d` è `imull %edi, %r8d` [cite: 68, 69, 70].

Perché scartare le altre: (A) e (C) effettuano una banale addizione ignorando l'operatore prodotto (`*`) del codice C. L'opzione (B) accumulerebbe la moltiplicazione nel registro `%eax` che in questo contesto interno è adoperato per altri scopi, distruggendo la variabile `res` che vive in `%r8d`.

58. **Domanda:** Quale tipo di hazard si potrebbe verificare durante l'esecuzione di queste istruzioni in un processore con pipeline a 5 stadi (IF, ID, EX, MEM, WB)?

```
sub x4, x5, x6
add x7, x5, x8
add x9, x6, x10
add x11, x5, x12
sub x12, x0, x0
jpz x4, label
```

- (A) Hazard di controllo.
- (B) Hazard strutturale.
- (C) Hazard di dati.
- (D) Nessuna delle precedenti.

Risposta corretta: (A)

Perché è corretta: L'ultima riga è un'istruzione di salto condizionato (`jpz x4, label`) il cui comportamento altererà il normale flusso di fetch delle istruzioni successive a seconda del valore del registro `x4`. Poiché la destinazione non è immediatamente predicibile, si forma nella pipeline un **Hazard di Controllo** che il processore dovrà gestire ritardando le operazioni finché l'indirizzo target non diventerà noto [cite: 75, 76, 77].

Perché scartare le altre: Non c'è alcun hazard strutturale, non essendoci collisione su risorse condivise. Benché `jpz` legga `x4` modificato dal primo `sub`, la distanza in pipeline (5 istruzioni intermedie) è ampiamente sufficiente a far completare la fase di WB per quel registro, per cui al momento dell'ingresso della `jpz` non si verifica un Hazard sui Dati.

59. **Domanda:** Quale delle seguenti affermazioni descrive un vantaggio per l'uso di un bus asincrono rispetto a un bus sincrono?

- (A) Un bus asincrono è molto più semplice da progettare rispetto a un bus sincrono.
- (B) Un bus sincrono è più robusto al clock drift rispetto a un bus asincrono.
- (C) Un bus asincrono opera anche tra dispositivi con frequenze e fasi di clock differenti.
- (D) Un bus sincrono usa dei segnali di controllo più complessi rispetto a un bus asincrono.
- (E) Nessuna delle precedenti.

Risposta corretta: (C)

Perché è corretta: Il più grande pregio dei bus asincroni (e svantaggio di quelli sincroni) è il non fare affidamento su un unico clock globale, bensì su protocolli di **handshaking**. Questo consente a svariate periferiche eterogenee, con interfacce lente o con frequenze di clock differenti rispetto alla CPU, di comunicare fluidamente adattandosi alle diverse tempistiche.

Perché scartare le altre: I bus asincroni richiedono protocolli a più "mani" e risultano fisicamente più complessi da progettare (A). I bus sincroni sono più rigidi, richiedono controlli semplici ma non gestiscono agilmente dispositivi multi-frequenza e sono limitati su lunghe distanze (B, D).

60. **Domanda:** Sia una cache del tipo set associative con 8 vie, questa permette di archiviare 128 kB di dati su blocchi della dimensione di 64 bytes. Considerando che vogliamo accedere al dato con indirizzo `0x1FA58B` e che non lo troviamo nella cache, dopo averlo recuperato dalla memoria principale in quale blocco tra quelli elencati potremmo inserire questo dato?

- (A) Nel blocco numero 256 della cache.
- (B) Nel blocco numero 1023 della cache.
- (C) Nel blocco numero 512 della cache.
- (D) Nel blocco numero 1204 della cache.
- (E) In nessuno dei blocchi elencati.

Risposta corretta: (D)

Perché è corretta: Strutturiamo la cache: per blocchi da 64 bytes servono $\log_2(64) = 6$ bit di Offset. I blocchi totali sono $128\text{kB}/64 = 2048$. Essendo a 8 vie, i Set sono $2048/8 = 256$ Set. Questo richiede $\log_2(256) = 8$ bit di indice. Convertiamo l'indirizzo in binario: 0001 1111 1010 0101 1000 1011. L'indice si trova nei bit da 6 a 13 e corrisponde a 10010110, ovvero il Set 150 decimale. All'interno del Set 150 ci sono 8 vie fisiche contigue che vanno dal blocco numero $150 \times 8 = 1200$ al blocco $1200 + 7 = 1207$ [cite: 93, 94]. Poiché il numero 1204 ricade in questo preciso range di slot fisici a disposizione per quel set, la risposta corretta è la D.

Perché scartare le altre: Le opzioni A, B e C elencano numeri di blocco situati in zone di memoria della cache diametralmente lontane, assegnati a set differenti derivanti da probabili malinterpretazioni nel calcolo del modulo o delle basi.

61. **Domanda:** Supponendo un processore che ha la facoltà di dividere l'esecuzione di una istruzione in più operazioni distinte, supponendo che la fetch dell'istruzione impieghi 300ps, la decode impieghi 200ps, l'esecuzione impieghi 400ps, l'accesso alla memoria impieghi 500ps e la scrittura del risultato impieghi 100ps. Quale è il possibile, teorico, fattore di miglioramento se strutturassimo il processore con una pipeline a 5 stadi (IF, ID, EX, MEM, WB)?

- (A) 2.5
- (B) 3.0
- (C) 4.5
- (D) 5.0
- (E) Nessuna delle precedenti.

Risposta corretta: (B)

Perché è corretta: Il tempo totale senza pipeline è dato dalla somma dei ritardi di ciascuno stadio: $300 + 200 + 400 + 500 + 100 = 1500$ ps. Quando si implementa la pipeline, la durata del singolo ciclo di clock è determinata unicamente dal collo di bottiglia, ovvero lo stadio più lento, che in questo caso impiega 500 ps. Il potenziale fattore di miglioramento (Speedup) teorico è pari al tempo sequenziale diviso il nuovo tempo di ciclo: $1500/500 = 3.0$.

Perché scartare le altre: (D) varrebbe 5.0 unicamente se la pipeline fosse perfetta e idealmente bilanciata in stadi tutti della medesima durata (cosa falsa qui). Le altre opzioni derivano da somme errate o rapporti inversi.

62. **Domanda:** Viene dato un frammento C e una traduzione ARM incompleta. Quali sono le due istruzioni da inserire al posto dei puntini per completare correttamente la traduzione?

```
void vectops (short *a, short *b, short *c, short n) {
    short i = 0;
    while (i < n) {
        if (b[i] > c[i]) a[i++] = b[i] - c[i];
        else             a[i++] = c[i] - b[i];
    }
}
```

- (A) `suble ip, ip, lr; strhgt ip, [ro]`
- (B) `suble ip, lr, ip; strhle ip, [ro]`
- (C) `suble ip, lr, ip; strhgt ip, [ro]`
- (D) `suble ip, ip, lr; strhle ip, [ro]`
- (E) Nessuna delle precedenti.

Risposta corretta: (D)

Perché è corretta: L'analisi del codice assembly rivela che ARM ottimizza l'`if/else` apponendo suffissi di predizione. Se $b[i] > c[i]$, ARM ha già calcolato correttamente tramite `subgt` e `strhgt`. La riga da compilare è il ramo `else`, e va eseguita solo nel caso logico opposto, ovvero quando $b[i] \leq c[i]$ (Less or Equal). La sottrazione C per l'`else` richiede di prendere $c[i]$ (mappato in `ip`) sottrargli $b[i]$ (`lr`) e salvare nel registro `ip`: `suble ip, ip, lr`. Infine lo store *condizionale* deposita il risultato nell'array dest: `strhle ip, [ro]`.

Perché scartare le altre: Nelle risposte A e C si tenta un'illogica inversione del salvataggio unendo suffissi `le` a suffissi `gt` (che farebbero abortire l'esecuzione dell'istruzione se i flag di Less-Equal fossero accesi). La risposta B sottrae in ordine inverso `lr - ip` scambiando di fatto $b[i] - c[i]$, errato per l'`else`.

63. **Domanda:** Viene dato un frammento C e una traduzione Intel x86-64 incompleta. Quali sono le due istruzioni da inserire al posto di X1 e X2 per completare correttamente la traduzione?

```
int vectops (int a[], int n, int b, int c) {
    int i=0, prod=1;
    while (i++ < n) {
        if (a[i] > b)      a[i] = b;
        else if (a[i] > c) a[i] = c;
        prod *= a[i];
    }
    return prod;
}
```

- (A) X1: `imull 4(%r8), %edi` | X2: `movl %edx, 4(%rax)`
- (B) X1: `imull 4(%r8), %edi` | X2: `movl %ecx, 4(%rax)`
- (C) X1: `addl 4(%r8), %edi` | X2: `movl %edx, 4(%rax)`
- (D) X1: `addl 4(%r8), %edi` | X2: `movl %ecx, 4(%rax)`
- (E) Nessuna delle precedenti.

Risposta corretta: (A)

Perché è corretta: Rilevando la semantica C e i blocchi del loop tradotti in Assembly x86, si deduce che manca la porzione del prodotto iterativo `prod *= a[i]` (eseguita sempre a fine blocco) e l'assegnamento del ramo `if a[i] = b`. Il prodotto logico (`prod`) è stoccato in `%edi`. L'operazione corretta è la moltiplicazione CISC di un dato caricato da memoria con offset: `imull 4(%r8), %edi` in X1[cite: 140, 142]. Invece in X2, corrispondente alla gestione dell'assegnamento `if` (prima del `jmp L4`), dobbiamo spostare il valore scalare di `b` (in `%edx`) nella cella `a[i]` (puntata da `rax` offset 4): `movl %edx, 4(%rax)`.

Perché scartare le altre: Le opzioni (C) e (D) usano addizioni (`addl`) rimpiazzando l'operatore matematico del ciclo (il prodotto). L'opzione (B) utilizza in X2 il registro `%ecx`, assegnando di fatto `c` e non `b` quando viene rilevata la condizione `a[i] > b`.

15 Preparazione all'Esame Orale: Domande Teoriche e Approfondimenti

In questa sezione vengono affrontate in maniera discorsiva e dettagliata le principali domande poste frequentemente all'esame orale. Queste risposte costituiscono un'ottima base per ripassare i concetti e dimostrare padronanza tecnica.

15.1 1. Spiegare la gerarchia della memoria

Il problema storico dell'architettura dei calcolatori è conciliare la necessità di avere memorie **capienti, veloci ed economiche**. Fisicamente, una memoria non può soddisfare tutti e tre i requisiti: memorie veloci (come la SRAM) costano tantissimo e occupano molto spazio sul chip (bassa densità), mentre memorie capienti (come dischi o DRAM) sono lente ed economiche.

Per risolvere il problema si utilizza una **gerarchia**. Si sfrutta il *Principio di Località* per dare all'utente l'illusione di possedere una memoria grande quanto il livello più capiente (e lento) ma veloce quanto il livello più rapido (vicino al processore). La gerarchia si compone tipicamente di:

1. **Registri della CPU:** Costruiti in SRAM, i più veloci in assoluto ma limitati a poche decine o centinaia di byte.
2. **Cache (L1, L2, L3):** Costruite in SRAM, vicinissime o integrate nel processore. Ordine di grandezza dei Kilobyte o Megabyte. Latenza di pochi nanosecondi.
3. **Memoria Principale (RAM):** Costruita in DRAM (condensatore + transistor), molto densa ed economica. Ordine dei Gigabyte, ma latenze nell'ordine delle decine o centinaia di nanosecondi.
4. **Memorie di Massa (Hard Disk, SSD):** Archiviazione permanente, non volatile, ordine dei Terabyte. Tempi di accesso nell'ordine dei millisecondi (milioni di volte più lenti della CPU).

15.2 2. Definire i principi di località spaziale e temporale

Il successo dell'intera gerarchia di memoria, e in primis della cache, si fonda su due assunti statistici verificati sul comportamento reale dei programmi:

- **Località Temporale:** Se un programma ha appena fatto accesso a una determinata cella di memoria, è altamente probabile che *a brevissimo tempo* faccia di nuovo accesso a quella stessa cella. Un esempio tipico sono le variabili contatore all'interno di un ciclo `for`, che vengono lette e sovrascritte ad ogni iterazione.
- **Località Spaziale:** Se un programma fa accesso a una locazione di memoria, è altamente probabile che nei passi immediatamente successivi richieda l'accesso alle celle di memoria *fisicamente adiacenti*. Un classico esempio è lo scorrimento di un array o l'esecuzione sequenziale delle istruzioni del programma in `.text`.

Grazie alla prima, manteniamo in cache i dati usati da poco. Grazie alla seconda, quando si verifica un *miss*, non portiamo in cache la singola word richiesta, ma preleviamo un intero **blocco** (linea di cache) che comprende la word richiesta e quelle adiacenti.

15.3 3. Le varie tipologie di cache e la lettura Set-Associativa

Esistono tre architetture hardware per mappare la memoria nella cache:

1. **Direct Mapped (A mappatura diretta):** Ogni blocco della RAM ha *un solo* blocco specifico nella cache in cui può essere salvato (calcolato tramite l'operazione $\text{Indirizzo} \bmod \text{NumeroBlocchiCache}$). Se due indirizzi competono per lo stesso slot, si sfrattano a vicenda creando molti conflitti (*conflict miss*).
2. **Fully Associative (Completamente associativa):** Un blocco può essere posizionato in *qualsiasi* riga libera della cache. Abbassa drasticamente i miss, ma per cercare il dato bisogna confrontare contemporaneamente in hardware tutti i *Tag* della cache. Estremamente costosa, impraticabile per grandi dimensioni.
3. **Set Associativa a N vie:** Un compromesso eccellente. La cache è divisa in "Insiemi" (*Set*). Un blocco RAM è vincolato a finire in uno specifico Set (tramite operazione modulo), ma *all'interno* di quel Set può essere collocato in una qualsiasi delle *N* vie a disposizione.

Lettura da una Set-Associativa: Quando la CPU cerca un dato, frammenta l'indirizzo generato in tre parti: *Tag*, *Indice* e *Offset*.

- L'**Indice** viene usato per selezionare lo specifico *Set* della cache.
- All'interno di quel Set vi sono *N* blocchi. La circuiteria estrae *tutti gli N Tag* dei blocchi presenti e li confronta **in parallelo** (simultaneamente) con il *Tag* derivato dall'indirizzo richiesto.
- Se c'è una coincidenza e il bit di validità (*V*) è 1, si ha un *Hit*. L'uscita del comparatore logico piloterà un Multiplexer che selezionerà esattamente la via giusta, restituendo infine la specifica word calcolata tramite l'**Offset**.

15.4 4. Il concetto di Pipeline: vantaggi, stadi ed Hazard

La **Pipeline** è una tecnica architetturale che aumenta il **throughput** del processore, applicando il principio della catena di montaggio di Henry Ford. Invece di eseguire un'intera istruzione (es. 800ps) prima di prelevare la successiva, l'esecuzione viene sminuzzata in stadi bilanciati (solitamente 5, di pari durata, es. 200ps). In questo modo, ad ogni ciclo di clock (200ps), una nuova istruzione entra e un'istruzione terminata esce, permettendo alla CPU di lavorare su 5 istruzioni in parallelo in fasi diverse.

Gli **Stadi RISC classici** sono: **IF** (Instruction Fetch), **ID** (Instruction Decode & Register Read), **EX** (Execute / ALU), **MEM** (Memory Access), **WB** (Write-Back nel registro).

I problemi di sovrapposizione si chiamano **Hazard**:

- **Hazard Strutturali**: Due istruzioni tentano di usare fisicamente la stessa risorsa nello stesso ciclo (es. leggere un dato e prelevare un'istruzione usando l'unica memoria). Si risolve duplicando l'hardware (es. Cache Istruzioni e Cache Dati separate).
- **Hazard sui Dati (Data Hazard)**: Un'istruzione in fase *EX* ha bisogno del risultato di una *add* che la precede ma che non è ancora arrivata alla fine per scriverlo (*WB*). Si risolve con il **Forwarding** (la ALU pesca il risultato appena finito e lo ricicla in ingresso) o, nei peggiori casi (dopo una *Load*), si stallerà la pipeline inserendo delle **bolle** (Nop) per aspettare che la memoria consegni il dato.
- **Hazard di Controllo (Branch Hazard)**: Dopo una *beq*, la CPU non sa subito da quale riga prelevare la successiva (non ha ancora calcolato l'uguaglianza e il target). Si risolve con la **Branch Prediction**: hardware specifico prova a indovinare statisticamente l'esito. Se sbaglia, svuota la pipeline spreco di cicli (*Flush*).

15.5 5. Differenza tra Interrupt e Traps

Entrambe causano una transizione imprevista del flusso verso il Sistema Operativo (passando a modalità *Supervisor*), ma le origini e le tempistiche sono opposte.

- Le **Eccezioni Interne (Traps / Faults)** sono generate **dall'interno** del processore a causa dell'esecuzione di una singola e specifica istruzione problematica (es. divisione per zero, page fault, istruzione illegale o *ecall*). Sono eventi del tutto **sincroni**: avvengono esattamente su quella istruzione.
- Gli **Interrupts** sono interruzioni hardware generate **dall'esterno** del processore (es. una tastiera, la scheda di rete, un timer, il disco fisso). Sono totalmente scorrelate dall'istruzione che la CPU sta processando e sono eventi **asincroni**.

15.6 6. Polling vs I/O Asincrono e Bus

L'I/O Polling (Attesa Attiva): La CPU dialoga con le periferiche (molto lente) attraverso registri Memory Mapped. Nel *polling*, la CPU entra in un ciclo infinito in cui legge ripetutamente un *Registro di Stato* della periferica (**while (!ready);**), attendendo che l'hardware setti il bit di disponibilità per poi inviare il dato. Sebbene semplice da programmare, genera un enorme spreco di preziosi cicli di CPU (fino al 100% di CPU Load inutilmente). Per questo motivo si delega la comunicazione agli *Interrupts* o al *DMA* (Direct Memory Access).

Bus sincroni vs asincroni:

- I **Bus Sincroni** operano comandati da un clock globale (tipicamente il bus Processore-Memoria). Sono velocissimi, ma impongono che ogni entità collegata rispetti una specifica tempistica: non si prestano a periferiche eterogenee di varia velocità.
- I **Bus Asincroni** non hanno un clock condiviso (es. bus di I/O o di sistema estesi). La sincronizzazione avviene con un protocollo di **Handshaking** a quattro mani (Richiesta, Conferma di pronto, Ricevuta) adattando dinamicamente le tempistiche della comunicazione alle capacità della singola periferica.

15.7 7. Spiegare lo schema del processore (RISC-V) e la beq

Lo schema del Datapath di base (a singolo ciclo) unisce gli stadi logici visti per la pipeline in un unico grande circuito combinatorio che viene alimentato e termina in un singolo colpo di clock. Tutto inizia dal **PC (Program Counter)**, che indica quale istruzione prelevare dalla **Memoria Istruzioni**. Un sommatore isolato ne ricalcola subito il $+4$. L'istruzione generata spinge l'**Unità di Controllo** ad asserire segnali (0/1) per pilotare i vari MUX (Multiplexer) del datapath, scegliendo cosa deve scorrere nei fili.

I Segnali di Controllo per la Branch (beq): Per la `beq x5, x6, offset`, la CPU deve confrontare `x5` e `x6` e saltare se sono uguali. L'Unità di Controllo leggerà l'Opcode (codice operativo) e configurerà la CPU così:

- **Branch = 1:** Attiva il potenziale Multiplexer di ricalcolo del PC che si trova alla fine del datapath.
- **ALUOp = 01:** Ordina al controllore secondario della ALU di eseguire una **sottrazione**.
- **ALUSrc = 0:** Indica che il secondo operando che entra in ALU *non* deve essere l'offset, ma deve essere letto dal banco dei registri (`x6`).
- **RegWrite = 0 / MemWrite = 0:** La *branch* non modifica né registri generali né la memoria dati. Quei circuiti vanno tenuti spenti per sicurezza.

Contemporaneamente, l'addizionatore in alto calcola l'indirizzo di offset (esteso in segno e *shiftato* per 2). Infine: se la ALU ha calcolato $x5 - x6 = 0$, la sua uscita hardware speciale **Zero** diventa 1. Essendo una porta logica AND attivata su **Branch** e **Zero**, solo in questo scenario il Mux finale devierà il prossimo PC sull'indirizzo appena calcolato, "prendendo" di fatto il salto.

15.8 8. Istruzione lea in Assembly Intel x86-64

L'istruzione **Load Effective Address (lea)** sfrutta una particolarità CISC di Intel. Benché abbia la sintassi di un'operazione verso la RAM (ad es. `leaq (%rsi, %rdi, 4), %rax`), *non tocca mai la memoria*. Passa invece gli operandi all'*Address Generation Unit* hardware della CPU, facendogli eseguire un'operazione aritmetica complessa (in questo caso $rsi + rdi \times 4$) e riversa puramente il risultato aritmetico calcolato all'interno del registro di destinazione (`%rax`). A differenza dell'istruzione `add`, `lea` non "sporca" nessuno dei due registri in ingresso ed è usatissima dal compilatore per operazioni aritmetiche ottimizzate o incrementi.

15.9 9. Toolchain: fasi ed esecuzione

Il processo di trasformazione dal linguaggio C all'eseguibile passa tramite 4 fasi della "catena degli strumenti":

1. **Preprocessore:** Risolve direttive testuali (`#include`, `#define`), incollando e sostituendo testo puro.
2. **Compilatore:** Analizza la logica di alto livello e traduce in linguaggio macchina mnemonico (**Assembly**, file `.s`).
3. **Assemblatore (Assembler):** Sostituisce le istruzioni pseudo (`mv, li, j`) in istruzioni vere. Codifica le istruzioni mnemonico in vero **Linguaggio Macchina binario**, producendo i blocchi `.data` e `.text`. Restituisce un File Oggetto non eseguibile (file `.o`), contenente "buchi" non risolti (Tabella di Rilocazione) e le interfacce per comunicare all'esterno (Symbol Table).
4. **Linker:** Assembla e amalgama più file oggetto insieme alle librerie (Statiche o Dinamiche). Assegna a tutti uno spazio di memoria assoluto (sostituendo gli indirizzi relativi) ed esegue il *patching* (Rilocazione) rimpiazzando i "buchi" con i veri indirizzi fisici prima di sfornare l'eseguibile (`.exe` o `a.out`).

15.10 10. Write-back in Assembly ARM (e implementazione in RISC-V)

Una delle peculiarità dell'istruzione di Load e Store in ARM è l'utilizzo dell'operatore **Write-Back** (espresso col suffisso del punto esclamativo `!`, o tramite post-indexing). Un'istruzione come `ldr r3, [r0, #4]!` estrae il valore dalla memoria all'indirizzo `r0 + 4`, e in un singolo colpo di clock aggiorna *anche* il registro di base `r0`, che ora conterrà proprio il valore `r0+4`.

Domanda Bonus: Come implementereste l'equivalente ARM in RISC-V?

L'architettura RISC-V si rifiuta di fare troppe cose nello stesso colpo di clock (per mantenere stadi rapidi). RISC-V permette un'unica operazione di Write-Back alla volta per istruzione (scrivendo, in questo caso, sul registro `r3`). Poiché in ARM stiamo aggiornando ben *due registri in contemporanea* (la destinazione `r3` col dato RAM e la base `r0` col nuovo offset calcolato dalla ALU), un'implementazione RISC-V esatta richiederebbe **due istruzioni sequenziali distinte**:

```
ld    x3, 4(x0)           // Carico il dato in x3 (Equivalente al LDR semplice)
addi  x0, x0, 4           // Incremento separatamente il registro base x0
```

Per averlo nativo, la microarchitettura RISC-V dovrebbe essere pesantemente modificata, inserendo due reti di feedback distinte per il *Write-Back stage* e costringendo il banco dei registri ad accettare *due scritture* simultanee per ciclo, complicando il datapath a dismisura e rendendolo più costoso.